UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Facultad de Ingeniería en Sistemas de Información

Aseguramiento de la Calidad de Software

ENTREGA 2

DISEÑO E IMPLEMENTACIÓN DE PRUEBAS QA

BancoERP

Sistema bancario ERP · banca interna, portal e integración ACH

Fases del STLC: diseño de pruebas y preparación para la ejecución

Modalidad: equipo de hasta 4 estudiantes · Valor: 5.00 puntos netos

Integrantes del equipo

No.	Nombre completo	Carné	Rol QA
1	Luis Alejandro Contreras Paredez	[CARNE]	Lider QA
2	[NOMBRE COMPLETO INTEGRANTE 2]	[CARNE]	Analista de Pruebas
3	[NOMBRE COMPLETO INTEGRANTE 3]	[CARNE]	Tester Funcional
4	[NOMBRE COMPLETO INTEGRANTE 4]	[CARNE]	Tester de Seguridad e Integracion

Equipo XX · Docente: [NOMBRE DEL CATEDRÁTICO]

Guatemala, 6 de octubre de 2026

1. Información del proyecto

Elemento	Definición
Sistema	BancoERP · React/Vite, APIs ASP.NET Core .NET 10, PostgreSQL 16 y MongoDB 8.
Base de continuidad	Entrega 1: E1_MasterTestPlan_EquipoXX.docx, RTM de 24 requerimientos REQ-01 a REQ-24; BT, riesgos y hallazgos conservados.
Versión examinada	Rama main; HEAD 0453d06 más cambios locales preexistentes. E1 analizaba 28f3faf; no se presume que los hallazgos antiguos sigan iguales.
Versión del documento	2.0 · 06-10-2026 · Definiciones cerradas por el equipo QA
Alcance	Diseño formal, RTM actualizada, matriz de datos, mock masivo, colección API y alistamiento. La ejecución funcional completa corresponde a la fase posterior.
Resultado del diseño	24 requerimientos inventariados; 53 condiciones; 139 casos; 139 con oráculo de E1, técnico o aprobado por el equipo QA y 0 bloqueados por definición.
Resultado de preparación	15 comprobaciones realizadas; dictamen APTO CON RESTRICCIONES; observación de autorización ACH pendiente.
Referencia API previa	Actividad de clase del 03-10-2026 (plan de pruebas de la Public API): apoyo técnico para REQ-19/20; sus 35 casos no sustituyen la RTM global de E1 ni se contabilizan como ejecutados en este entregable.

1.1 Objetivo y alcance del diseño

Responder cómo se probará cada requerimiento, qué técnica selecciona sus entradas y qué datos y entorno permiten ejecutarlo. Los resultados esperados se fijan antes de la ejecución. Los casos diseñados no se presentan como aprobados; las comprobaciones efectivamente realizadas se identifican RD-01 a RD-15 y MOCK/GATE en sus reportes. No se declara aprobación de la suite formal completa.

La Entrega 1 dejó requerimientos sin un resultado esperado verificable (umbral de bloqueo, descuadre de caja, límite diario, secretos ACH, ACH saliente, compuerta de vulnerabilidades e interfaz). El equipo QA los cerró en la reunión de diseño del 06-10-2026, con aprobación del Líder QA como responsable de requerimientos del proyecto. Esas reglas son el criterio de aceptación vigente de esta entrega; no demuestran que el software ya las cumpla. Se conserva el criterio original de E1 y se añade el vigente; las diferencias de implementación se registran como defectos.

1.2 Registro de decisiones aprobadas

ID / REQ	Regla vigente
D-01 · REQ-02	5 fallos consecutivos; bloqueo de 30 minutos; desbloqueo temporal al vencer; login correcto reinicia contador.
D-02 · REQ-03, REQ-04, REQ-05	Contraseña >=8 caracteres, mayúscula, minúscula y número; impedir últimas 5. Recuperación de un solo uso por 2 horas; token en respuesta exclusivamente en QA académico.
D-03 · REQ-09	Registrar cualquier diferencia; |diferencia|<=Q10 permite cierre sin justificación; |diferencia|>Q10 requiere justificación; clasificar FALTANTE/SOBRANTE/CUADRE EXACTO.
D-04 · REQ-11, REQ-12, REQ-16	Aplicar y acumular límite diario de AHORRO en retiro, transferencia interna y portal; rechazar excedente sin efectos. La divergencia actual es defecto, no excepción aprobada.
D-05 · REQ-06, REQ-17	Configuración ACH requiere JWT y permiso administrativo; no devolver contraseñas; persistirlas cifradas con clave fuera de la base y del repositorio. Precisión de diseño: Usar permiso efectivo existente catalogos.configuracion.editar como permiso administrativo de configuración; falta de JWT=401 y falta de permiso=403.
D-06 · REQ-23	Bloquear liberación por vulnerabilidades Altas o Críticas; registrar Medias para seguimiento. La regla local ejecutable no equivale a integración CI remota realizada.
D-07 · REQ-24	Login y navegación principal a 1366x768 y 390x844, temas claro y oscuro, sin superposición ni scroll horizontal.

ID / REQ	Regla vigente
D-08 · REQ-18	Timeout 15 segundos; ante timeout ERROR y requiere conciliación, sin débito local ni reintento automático. HTTP500 o JSON inválido: ERROR, saldo intacto y mensaje controlado. Confirmación positiva válida: un único débito por código. Precisión de diseño: Contrato objetivo: timeout HTTP504 y message='Tiempo de espera agotado; requiere conciliacion.'; falla HTTP500 externa HTTP502 y message='Banco externo no disponible.'; JSON inválido externo HTTP502 y message='Respuesta invalida del banco externo.'; confirmación válida HTTP200 y estadoNombre=ENVIADA; duplicado HTTP409, message='La transferencia ya fue procesada.' y delta saldo=0. Son oráculos de diseño aprobados, no respuestas verificadas del código actual.
D-09 · REQ-07	Actualizar el inventario vigente a las 20 constantes de EventosSeguridad.cs y diseñar un escenario por evento. El criterio original de E1 (21 eventos) se conserva con nota de cambio, porque el código fuente vigente define 20.

No quedan casos bloqueados por definición de negocio. Decisiones_Aprobadas.json conserva la fecha, el responsable y el alcance de cada decisión.

1.3 Guía de navegación y trazabilidad

Cadena exacta: REQ-nn → CP-REQnn-cc → TC-REQnn-sss → DT-TC-REQnn-sss. El caso declara técnica, prioridad, precondiciones, pasos, entrada y oráculo. El Excel permite filtrar y recorrer ambos sentidos; este Word conserva los vínculos y las fichas completas.

Contenido	Ubicación
2. RTM	Inventario E1, condiciones/casos y bloqueos.
3. Diseño y casos	Aplicación de cinco técnicas y repositorio formal completo.
4. Datos	Fixtures, volumen, estructuras y preparación.
5. Readiness	Resultados técnicos observados, capturas, aislamiento y uso del paquete.
6. Reflexión	Respuestas fundamentadas a las cuatro preguntas.
Anexos	Consultas SQL, inventario de archivos y referencias.

2. RTM actualizada

2.1 Requerimientos originales y criterios

Se preservan literalmente la descripción y el criterio de aceptación de la RTM de E1. Los pendientes de E1 se conservan como historia y se resuelven mediante el criterio vigente aprobado mostrado debajo. La prioridad del caso usa Alta/Media/Baja; la prioridad Crítica de E1 se conserva en el Excel y se traduce a Alta para el campo solicitado del caso.

REQ-01 · Autenticacion de usuario interno con emision de JWT y refresh token

Fuente: BT-01 · Riesgo: R-02 · Prioridad E1: Alta

Criterio original E1: Con credenciales validas responde 200 con token, refresh y permisos; con invalidas responde 401 sin emitir token

Criterio vigente: Con credenciales validas responde 200 con token, refresh y permisos; con invalidas responde 401 sin emitir token

Cobertura de diseño: 4 casos; 0 bloqueados. Casos con oráculo definido; ejecución funcional pendiente.

REQ-02 · Bloqueo de usuario por intentos fallidos consecutivos

Fuente: BT-01 · Riesgo: R-02 · Prioridad E1: Alta

Criterio original E1: PENDIENTE DE DEFINICION: el umbral N no esta especificado por el negocio

Criterio vigente: 5 fallos consecutivos; bloqueo de 30 minutos; desbloqueo temporal al vencer; login correcto reinicia contador.

Cobertura de diseño: 5 casos; 0 bloqueados. Casos con oráculo definido; ejecución funcional pendiente.

REQ-03 · Politica de composicion e historial de contrasenas

Fuente: BT-01 · Riesgo: R-02 · Prioridad E1: Media

Criterio original E1: Rechaza contrasenas de menos de 8 caracteres o sin mayuscula, minuscula o digito; rechaza las ultimas 5 usadas

Criterio vigente: Contraseña >=8 caracteres, mayúscula, minúscula y número; impedir últimas 5. Recuperación de un solo uso por 2 horas; token en respuesta exclusivamente en QA académico.

Cobertura de diseño: 7 casos; 0 bloqueados. Casos con oráculo definido; ejecución funcional pendiente.

REQ-04 · Recuperacion de contrasena mediante token temporal

Fuente: BT-01 · Riesgo: R-02 · Prioridad E1: Media

Criterio original E1: Genera token de un solo uso con expiracion; token usado o vencido es rechazado

Criterio vigente: Contraseña >=8 caracteres, mayúscula, minúscula y número; impedir últimas 5. Recuperación de un solo uso por 2 horas; token en respuesta exclusivamente en QA académico.

Cobertura de diseño: 3 casos; 0 bloqueados. Casos con oráculo definido; ejecución funcional pendiente.

REQ-05 · Rotacion de refresh token y revocacion de sesiones

Fuente: BT-01 · Riesgo: R-02 · Prioridad E1: Alta

Criterio original E1: El refresh token anterior queda invalido tras la rotacion; el cambio de contrasena revoca toda sesion activa

Criterio vigente: Contraseña >=8 caracteres, mayúscula, minúscula y número; impedir últimas 5. Recuperación de un solo uso por 2 horas; token en respuesta exclusivamente en QA académico.

Cobertura de diseño: 5 casos; 0 bloqueados. Casos con oráculo definido; ejecución funcional pendiente.

REQ-06 · Autorizacion granular por permisos efectivos

Fuente: BT-01, BT-03 · Riesgo: R-01 · Prioridad E1: Critica

Criterio original E1: Toda peticion sin token responde 401 y toda peticion sin el permiso requerido responde 403

Criterio vigente: Configuración ACH requiere JWT y permiso administrativo; no devolver contraseñas; persistirlas cifradas con clave fuera de la base y del repositorio.

Cobertura de diseño: 4 casos; 0 bloqueados. Casos con oráculo definido; ejecución funcional pendiente.

REQ-07 · Bitacora de eventos de seguridad

Fuente: BT-03 · Riesgo: R-07 · Prioridad E1: Alta

Criterio original E1: Cada uno de los 21 eventos catalogados se persiste con usuario, IP, agente, resultado y fecha

Criterio vigente: Actualizar el inventario vigente a las 20 constantes de EventosSeguridad.cs y diseñar un escenario por evento. El criterio original de E1 (21 eventos) se conserva con nota de cambio, porque el código fuente vigente define 20.

Cobertura de diseño: 21 casos; 0 bloqueados. Casos con oráculo definido; ejecución funcional pendiente.

REQ-08 · Apertura de sesion de caja con fondo inicial

Fuente: BT-01, BT-09 · Riesgo: R-06 · Prioridad E1: Alta

Criterio original E1: Crea una sesion activa para el operador; sin caja abierta las operaciones de ventanilla se rechazan con 422

Criterio vigente: Crea una sesion activa para el operador; sin caja abierta las operaciones de ventanilla se rechazan con 422

Cobertura de diseño: 5 casos; 0 bloqueados. Casos con oráculo definido; ejecución funcional pendiente.

REQ-09 · Cierre de caja con cuadre de efectivo

Fuente: BT-09 · Riesgo: R-06 · Prioridad E1: Alta

Criterio original E1: PENDIENTE DE DEFINICION: no se especifica el comportamiento ante descuadre

Criterio vigente: Registrar cualquier diferencia; |diferencia|<=Q10 permite cierre sin justificación; |diferencia|>Q10 requiere justificación; clasificar FALTANTE/SOBRANTE/CUADRE EXACTO.

Cobertura de diseño: 7 casos; 0 bloqueados. Casos con oráculo definido; ejecución funcional pendiente.

REQ-10 · Deposito en ventanilla

Fuente: BT-02 · Riesgo: R-03 · Prioridad E1: Critica

Criterio original E1: Acredita el monto, devuelve referencia y saldo nuevo, y genera el asiento contable correspondiente

Criterio vigente: Acredita el monto, devuelve referencia y saldo nuevo, y genera el asiento contable correspondiente

Cobertura de diseño: 3 casos; 0 bloqueados. Casos con oráculo definido; ejecución funcional pendiente.

REQ-11 · Retiro en ventanilla con validacion de saldo y limite diario

Fuente: BT-02 · Riesgo: R-03, R-04 · Prioridad E1: Critica

Criterio original E1: Rechaza si el saldo disponible es insuficiente, si deja el saldo bajo el minimo o si excede el limite diario en cuentas AHORRO

Criterio vigente: Aplicar y acumular límite diario de AHORRO en retiro, transferencia interna y portal; rechazar excedente sin efectos. La divergencia actual es defecto, no excepción aprobada.

Cobertura de diseño: 7 casos; 0 bloqueados. Casos con oráculo definido; ejecución funcional pendiente.

REQ-12 · Transferencia interna en ventanilla

Fuente: BT-02 · Riesgo: R-04 · Prioridad E1: Critica

Criterio original E1: Debita origen, acredita destino y rechaza si origen y destino coinciden o el saldo es insuficiente

Criterio vigente: Aplicar y acumular límite diario de AHORRO en retiro, transferencia interna y portal; rechazar excedente sin efectos. La divergencia actual es defecto, no excepción aprobada.

Cobertura de diseño: 3 casos; 0 bloqueados. Casos con oráculo definido; ejecución funcional pendiente.

REQ-13 · Consulta paginada de historial de movimientos

Fuente: BT-01 · Riesgo: R-08 · Prioridad E1: Media

Criterio original E1: Devuelve los movimientos de la cuenta ordenados por fecha descendente, con maximo 100 por pagina

Criterio vigente: Devuelve los movimientos de la cuenta ordenados por fecha descendente, con maximo 100 por pagina

Cobertura de diseño: 8 casos; 0 bloqueados. Casos con oráculo definido; ejecución funcional pendiente.

REQ-14 · Apertura de cuenta AHORRO o MONETARIA

Fuente: BT-02 · Riesgo: R-04 · Prioridad E1: Alta

Criterio original E1: Crea la cuenta solo si el cliente esta ACTIVO, aplicando saldo minimo y limite diario segun el tipo

Criterio vigente: Crea la cuenta solo si el cliente esta ACTIVO, aplicando saldo minimo y limite diario segun el tipo

Cobertura de diseño: 3 casos; 0 bloqueados. Casos con oráculo definido; ejecución funcional pendiente.

REQ-15 · Consulta de cuentas y movimientos propios en el portal

Fuente: BT-01 · Riesgo: R-05 · Prioridad E1: Critica

Criterio original E1: El cliente solo obtiene informacion de las cuentas asociadas a su propio identificador de cliente

Criterio vigente: El cliente solo obtiene informacion de las cuentas asociadas a su propio identificador de cliente

Cobertura de diseño: 6 casos; 0 bloqueados. Casos con oráculo definido; ejecución funcional pendiente.

REQ-16 · Transferencia desde el portal del cliente

Fuente: BT-01 · Riesgo: R-04, R-05 · Prioridad E1: Critica

Criterio original E1: Valida propiedad de la cuenta origen, estado ACTIVO de cliente y cuentas, saldo disponible, saldo minimo y limite diario

Criterio vigente: Aplicar y acumular límite diario de AHORRO en retiro, transferencia interna y portal; rechazar excedente sin efectos. La divergencia actual es defecto, no excepción aprobada.

Cobertura de diseño: 3 casos; 0 bloqueados. Casos con oráculo definido; ejecución funcional pendiente.

REQ-17 · Administracion de configuracion de bancos ACH

Fuente: BT-01 · Riesgo: R-01 · Prioridad E1: Critica

Criterio original E1: PENDIENTE DE DEFINICION: no se especifica la proteccion de las credenciales de integracion

Criterio vigente: Configuración ACH requiere JWT y permiso administrativo; no devolver contraseñas; persistirlas cifradas con clave fuera de la base y del repositorio.

Cobertura de diseño: 3 casos; 0 bloqueados. Casos con oráculo definido; ejecución funcional pendiente.

REQ-18 · Envio de transferencia ACH saliente

Fuente: BT-01 · Riesgo: R-09 · Prioridad E1: Critica

Criterio original E1: PENDIENTE DE DEFINICION: no se especifica el manejo de indisponibilidad del banco destino

Criterio vigente: Timeout 15 segundos; ante timeout ERROR y requiere conciliación, sin débito local ni reintento automático. HTTP500 o JSON inválido: ERROR, saldo intacto y mensaje controlado. Confirmación positiva válida: un único débito por código.

Cobertura de diseño: 5 casos; 0 bloqueados. Casos con oráculo definido; ejecución funcional pendiente.

REQ-19 · Recepcion idempotente de ACH entrante

Fuente: BT-05 · Riesgo: R-10 · Prioridad E1: Critica

Criterio original E1: Un codigo de confirmacion ya procesado es rechazado sin acreditar fondos por segunda vez

Criterio vigente: Un codigo de confirmacion ya procesado es rechazado sin acreditar fondos por segunda vez

Cobertura de diseño: 14 casos; 0 bloqueados. Casos con oráculo definido; ejecución funcional pendiente.

REQ-20 · Autenticacion de banco externo con rol BANCO_EXTERNO

Fuente: BT-05, BT-07 · Riesgo: R-01 · Prioridad E1: Alta

Criterio original E1: Solo usuarios con el rol y el permiso especifico obtienen token; usuarios internos son rechazados

Criterio vigente: Solo usuarios con el rol y el permiso especifico obtienen token; usuarios internos son rechazados

Cobertura de diseño: 4 casos; 0 bloqueados. Casos con oráculo definido; ejecución funcional pendiente.

REQ-21 · Generacion automatica de asientos contables

Fuente: BT-02 · Riesgo: R-03 · Prioridad E1: Alta

Criterio original E1: Cada transaccion genera su asiento por partida doble contra el plan de cuentas

Criterio vigente: Cada transaccion genera su asiento por partida doble contra el plan de cuentas

Cobertura de diseño: 5 casos; 0 bloqueados. Casos con oráculo definido; ejecución funcional pendiente.

REQ-22 · Restriccion de origenes mediante CORS

Fuente: BT-01 · Riesgo: R-01 · Prioridad E1: Media

Criterio original E1: Solo los origenes declarados en la configuracion pueden consumir las APIs desde un navegador

Criterio vigente: Solo los origenes declarados en la configuracion pueden consumir las APIs desde un navegador

Cobertura de diseño: 5 casos; 0 bloqueados. Casos con oráculo definido; ejecución funcional pendiente.

REQ-23 · Despliegue reproducible con contenedores e integracion continua

Fuente: BT-08 · Riesgo: R-12 · Prioridad E1: Media

Criterio original E1: El pipeline construye, levanta el entorno y ejecuta el smoke funcional sin errores

Criterio vigente: Bloquear liberación por vulnerabilidades Altas o Críticas; registrar Medias para seguimiento. La regla local ejecutable no equivale a integración CI remota realizada.

Cobertura de diseño: 5 casos; 0 bloqueados. Casos con oráculo definido; ejecución funcional pendiente.

REQ-24 · Interfaz usable y responsive con modo claro y oscuro

Fuente: BT-04, BT-09 · Riesgo: R-13 · Prioridad E1: Baja

Criterio original E1: PENDIENTE DE DEFINICION: no existen criterios de usabilidad ni mockups de referencia

Criterio vigente: Login y navegación principal a 1366x768 y 390x844, temas claro y oscuro, sin superposición ni scroll horizontal.

Cobertura de diseño: 4 casos; 0 bloqueados. Casos con oráculo definido; ejecución funcional pendiente.

2.2 Vinculación 1:N entre requerimiento, condición y caso

Cada fila representa una condición; varios casos pueden derivarse de ella. El Excel RTM expande el último nivel a una fila por caso y conserva criterio, riesgo, fuente, dato, técnica y estado.

REQ	Condición	Casos asociados	Estado
REQ-01	CP-REQ01-01	TC-REQ01-001 TC-REQ01-002	Diseñado
REQ-01	CP-REQ01-02	TC-REQ01-003 TC-REQ01-004	Diseñado
REQ-02	CP-REQ02-01	TC-REQ02-001 TC-REQ02-002	Diseñado
REQ-02	CP-REQ02-02	TC-REQ02-003	Diseñado
REQ-02	CP-REQ02-03	TC-REQ02-004 TC-REQ02-005	Diseñado
REQ-03	CP-REQ03-01	TC-REQ03-001 TC-REQ03-002 TC-REQ03-003 TC-REQ03-004 TC-REQ03-005 TC-REQ03-006	Diseñado
REQ-03	CP-REQ03-02	TC-REQ03-007	Diseñado
REQ-04	CP-REQ04-01	TC-REQ04-001 TC-REQ04-002	Diseñado
REQ-04	CP-REQ04-02	TC-REQ04-003	Diseñado
REQ-05	CP-REQ05-01	TC-REQ05-001 TC-REQ05-005	Diseñado
REQ-05	CP-REQ05-02	TC-REQ05-002	Diseñado
REQ-06	CP-REQ06-01	TC-REQ06-001 TC-REQ06-002	Diseñado
REQ-06	CP-REQ06-02	TC-REQ06-003 TC-REQ06-004	Diseñado
REQ-07	CP-REQ07-01	TC-REQ07-001 TC-REQ07-002	Diseñado
REQ-07	CP-REQ07-02	TC-REQ07-003	Diseñado
REQ-08	CP-REQ08-01	TC-REQ08-001 TC-REQ08-002 TC-REQ08-003	Diseñado
REQ-08	CP-REQ08-02	TC-REQ08-004 TC-REQ08-005	Diseñado
REQ-09	CP-REQ09-01	TC-REQ09-001	Diseñado
REQ-09	CP-REQ09-02	TC-REQ09-002 TC-REQ09-003 TC-REQ09-004 TC-REQ09-005 TC-REQ09-006 TC-REQ09-007	Diseñado
REQ-10	CP-REQ10-01	TC-REQ10-001 TC-REQ10-002 TC-REQ10-003	Diseñado
REQ-11	CP-REQ11-01	TC-REQ11-001 TC-REQ11-002 TC-REQ11-003	Diseñado
REQ-11	CP-REQ11-02	TC-REQ11-004 TC-REQ11-005 TC-REQ11-006	Diseñado
REQ-11	CP-REQ11-03	TC-REQ11-007	Diseñado

REQ	Condición	Casos asociados	Estado
REQ-12	CP-REQ12-01	TC-REQ12-001 TC-REQ12-002	Diseñado
REQ-12	CP-REQ12-02	TC-REQ12-003	Diseñado
REQ-13	CP-REQ13-01	TC-REQ13-001 TC-REQ13-002 TC-REQ13-003 TC-REQ13-004 TC-REQ13-005	Diseñado
REQ-13	CP-REQ13-02	TC-REQ13-006 TC-REQ13-007 TC-REQ13-008	Diseñado
REQ-14	CP-REQ14-01	TC-REQ14-001 TC-REQ14-002	Diseñado
REQ-14	CP-REQ14-02	TC-REQ14-003	Diseñado
REQ-15	CP-REQ15-01	TC-REQ15-001 TC-REQ15-003 TC-REQ15-006	Diseñado
REQ-15	CP-REQ15-02	TC-REQ15-002	Diseñado
REQ-16	CP-REQ16-01	TC-REQ16-001	Diseñado
REQ-16	CP-REQ16-02	TC-REQ16-002 TC-REQ16-003	Diseñado
REQ-17	CP-REQ17-01	TC-REQ17-001 TC-REQ17-003	Diseñado
REQ-17	CP-REQ17-02	TC-REQ17-002	Diseñado
REQ-18	CP-REQ18-01	TC-REQ18-001 TC-REQ18-003 TC-REQ18-004	Diseñado
REQ-18	CP-REQ18-02	TC-REQ18-002 TC-REQ18-005	Diseñado
REQ-19	CP-REQ19-01	TC-REQ19-001	Diseñado
REQ-19	CP-REQ19-02	TC-REQ19-002 TC-REQ19-003	Diseñado
REQ-19	CP-REQ19-03	TC-REQ19-004 TC-REQ19-005 TC-REQ19-006 TC-REQ19-007 TC-REQ19-008	Diseñado
REQ-19	CP-REQ19-04	TC-REQ19-009 TC-REQ19-010 TC-REQ19-011 TC-REQ19-012 TC-REQ19-013 TC-REQ19-014	Diseñado
REQ-20	CP-REQ20-01	TC-REQ20-001 TC-REQ20-002 TC-REQ20-003	Diseñado
REQ-20	CP-REQ20-02	TC-REQ20-004	Diseñado
REQ-21	CP-REQ21-01	TC-REQ21-001 TC-REQ21-003 TC-REQ21-004	Diseñado
REQ-21	CP-REQ21-02	TC-REQ21-002 TC-REQ21-005	Diseñado

REQ	Condición	Casos asociados	Estado
REQ-22	CP-REQ22-01	TC-REQ22-001 TC-REQ22-002 TC-REQ22-003	Diseñado
REQ-23	CP-REQ23-01	TC-REQ23-001	Diseñado
REQ-23	CP-REQ23-02	TC-REQ23-002 TC-REQ23-003 TC-REQ23-004 TC-REQ23-005	Diseñado
REQ-24	CP-REQ24-01	TC-REQ24-001 TC-REQ24-002 TC-REQ24-003 TC-REQ24-004	Diseñado
REQ-07	CP-REQ07-04	TC-REQ07-004 TC-REQ07-005 TC-REQ07-006 TC-REQ07-007 TC-REQ07-008 TC-REQ07-009 TC-REQ07-010 TC-REQ07-011 TC-REQ07-012 TC-REQ07-013 TC-REQ07-014 TC-REQ07-015 TC-REQ07-016 TC-REQ07-017 TC-REQ07-018 TC-REQ07-019 TC-REQ07-020 TC-REQ07-021	Diseñado
REQ-05	CP-REQ05-03	TC-REQ05-003 TC-REQ05-004	Diseñado
REQ-15	CP-REQ15-03	TC-REQ15-004 TC-REQ15-005	Diseñado
REQ-22	CP-REQ22-02	TC-REQ22-004 TC-REQ22-005	Diseñado

2.3 Control de cobertura y brechas

Cobertura de inventario: 24/24 requerimientos tienen al menos una condición y un caso asociado (100%). Esto demuestra trazabilidad, no suficiencia ni aceptación. Diseño con oráculos definidos: 139/139 casos con oráculo de E1, técnico o aprobado por el equipo QA; las precondiciones de cada caso todavía deben prepararse. Cero casos bloqueados por definición. La preparación de fixtures por caso y la ejecución integral siguen siendo actividades separadas de la cobertura de diseño.

Registro de bloqueos de definición: 0. Las ocho brechas originales se cerraron con D-01 a D-09; no se eliminan los defectos funcionales abiertos por ese cierre.

La ausencia de casos huérfanos se valida programáticamente: cada TC tiene una REQ existente, una CP de esa misma REQ y un DT único. El resumen está en Validacion_Artefactos.json. H-E2-01 queda resuelto como corrección de inventario: 21 en E1 frente a 20 constantes vigentes. Catalogo_Eventos_QA.json vincula los 20 códigos a escenarios y puntos de emisión; esto prueba cobertura de diseño, no ejecución de los eventos.

3. Diseño formal y repositorio de casos

3.1 Partición de equivalencia

Se elige un representante por clase con comportamiento esperado común y se separan los factores que cambian el oráculo. Las particiones no sustituyen las fronteras ni los riesgos críticos; las cuatro técnicas de caja negra de este apartado se basan en ISTQB CTFL 4.0.1, §4.2 [R1].

Dominio	Clases válidas	Clases inválidas / adversas	Casos representativos
Login	Usuario activo y credencial correcta	Contraseña errónea; campos vacíos; inactivo	TC-REQ01-001/002/003; TC-REQ20-003
Monto ACH	Positivo, ≤2 decimales y dentro de numeric(18,2)	≤0; 3 decimales; supera máximo	TC-REQ19-001; 004–007
Moneda ACH	Q y GTQ se normalizan al mismo dominio	USD	TC-REQ19-008 y Pairwise
Cuenta portal	Propia y ACTIVA (cliente A y cliente B)	Origen ajeno; cuenta ajena; sin JWT; JWT sin permiso de portal	TC-REQ15-001 a 006; TC-REQ16-001
Refresh token	Vigente y no usado	Vacío; inexistente; ya rotado; revocado por logout	TC-REQ05-001, 003, 004, 005
Origen CORS	http://localhost:13000 exacto	Origen no listado; prefijo engañoso 13000.evil.invalid	TC-REQ22-001 a 005
Asiento por tipo de cuenta	AHORRO→2101; MONETARIA→2102; caja→1101	Plan 1101 inactivo	TC-REQ21-001 a 005
Tipos de cuenta	AHORRO, MONETARIA del catálogo activo	Cliente INACTIVO	TC-REQ14-001/002/003

3.2 Análisis de valores límite

Se aísla una regla por prueba: el límite diario se anula en los casos del saldo mínimo para que una validación anterior no oculte la frontera buscada. La unidad monetaria de frontera es Q0.01; una entrada con tres decimales constituye una clase inválida adicional.

Regla	Frontera / vecinos	Oráculo y casos
Contraseña ≥8 caracteres	7 / 8 / 9	Rechaza 7; acepta 8 y 9 si cumplen composición. TC-REQ03-001/002/003.
Fondo inicial ≥0	−0.01 / 0 / 0.01	422 / 200 / 200. TC-REQ08-001/002/003.
Depósito >0	−0.01 / 0 / 0.01	400 / 400 / 200. TC-REQ10-001/002/003.
Límite diario 500	499.99 / 500 / 500.01	200 / 200 / 422. TC-REQ11-001/002/003.
Saldo mínimo 100, saldo=1000	Retiro 899.99 / 900 / 900.01	Saldos 100.01 / 100 / rechazo. TC-REQ11-004/005/006.
Tamaño paginado	0 / 1 / 99 / 100 / 101	0 normaliza a 20; 101 limita a 100. TC-REQ13-001 a 005.
Puerto del origen CORS	13000 permitido / 13001 vecino	Allow-Origin presente / ausente. TC-REQ22-003/004.
Monto ACH numeric(18,2)	0, −0.01, 0.001, 10^16	Rechazos explícitos. No se simula que el máximo cabe sobre un saldo positivo: aceptación del máximo requeriría cuenta a cero y revisión del riesgo de overflow.

3.3 Tabla de decisión de depósito

Reglas minimizadas por orden de evaluación. “—” significa que el factor no cambia esa acción; no significa que se hayan probado todas las combinaciones. Esta tabla evita repetir estados irrelevantes una vez que un control anterior rechaza la operación.

Regla	JWT	Permiso	Caja	Monto> 0	Acción / casos
TD-DEP-01	No	—	—	—	401; sin lectura ni escritura TC-REQ06-001 / 002
TD-DEP-02	Sí	No	—	—	403; sin operación TC-REQ06-003
TD-DEP-03	Sí	Sí	No	—	422; sin operación TC-REQ08-004
TD-DEP-04	Sí	Sí	Sí	No	400; monto <=0, sin operación TC-REQ10-001 / 002
TD-DEP-05	Sí	Sí	Sí	Sí	200; operación y partida doble TC-REQ06-004 / TC-REQ10-003

3.4 Transición de estados

Se verifica el estado posterior y la ausencia de efectos en transiciones rechazadas. La revocación y la idempotencia se observan en persistencia; un HTTP aislado no demuestra que no hubo doble acreditación.

ID	Estado inicial	Evento	Estado final / oráculo	Casos
S-01	Activo / intentos=3	Contraseña errónea	Activo / intentos=4; 401; aún desbloqueado	TC-REQ02-001
S-02	Activo / intentos=4	Contraseña errónea	Bloqueado / intentos=5; 401; revocación sesiones	TC-REQ02-002
S-03	Bloqueado	Contraseña correcta	Bloqueado; 423; no token	TC-REQ02-003
S-04	Refresh vigente R1	Rotar	R1 revocado / R2 vigente; R1 posterior:401	TC-REQ05-001
S-05	Caja cerrada	Abrir fondo válido	Caja abierta; 200; UUID sesión	TC-REQ08-002 / 003
S-06	Caja abierta	Abrir otra vez	Caja abierta original; 422; una sesión	TC-REQ08-005
S-07	Caja abierta	Cerrar con cuadre	Caja cerrada; 200; diferencia cero	TC-REQ09-001
S-08	ACH no procesada	POST válido	APROBADA; 201; delta=monto	TC-REQ19-001
S-09	ACH APROBADA	Repetir código	APROBADA sin cambio; 409; delta=0	TC-REQ19-002
S-10	Sesión activa con R1	Logout y refresh con R1	Sesión cerrada; logout 200; refresh 401	TC-REQ05-005

3.5 Pairwise verificable

Para ACH entrante se combinan cuatro factores binarios válidos: moneda {Q, GTQ}, tipo {omitido, entrante}, referencia {omitida, E2 combinatorio} y monto {0.01, 100.00}. El producto completo contiene 2^4=16 filas. El algoritmo greedy determinista seleccionó seis; cubre C(4,2)×2×2=24 pares, verificados por enumeración independiente. Reducción: 62.5% de filas para este modelo. No se incluyen datos inválidos en Pairwise: se prueban por separado. [R2] define la cobertura por interacciones; no garantiza detectar fallos de tres o más factores.

Fila	Moneda	Tipo	Referencia	Monto	Caso
PW-01	Q	<omitido>	<omitido>	0.01	TC-REQ19-009
PW-02	Q	entrante	E2 combinatorio	100.00	TC-REQ19-010
PW-03	GTQ	<omitido>	<omitido>	100.00	TC-REQ19-011

Fila	Moneda	Tipo	Referencia	Monto	Caso
PW-04	GTQ	entrante	E2 combinatorio	0.01	TC-REQ19-012
PW-05	Q	<omitido>	E2 combinatorio	0.01	TC-REQ19-013
PW-06	Q	entrante	<omitido>	0.01	TC-REQ19-014

Evidencia del algoritmo y de los pares: Pairwise_Cobertura.json; generación en scripts/disenar.py y validación en scripts/validar.py. Cada fila restaura el saldo destino a Q1000.00 y usa un código único; no se deja que la ejecución anterior cambie el resultado esperado de la siguiente.

3.6 Convenciones de ejecución de las fichas

Cada ficha trae pasos numerados y ejecutables: preparación con el comando o SQL exacto, autenticación del perfil, petición con método, URL, encabezados y cuerpo literales, verificación de la respuesta y verificación en base de datos. Abreviaturas usadas en los pasos:

Abreviatura	Significado
psql-QA	docker compose -f 03_Evidencias_y_MockData/docker-compose.qa.yml exec -T postgres psql -U e2qa -d bancoerp_e2_qa -c "<SQL>". Solo opera sobre la base bancoerp_e2_qa.
<token>	JWT obtenido en el paso de autenticación del mismo caso (campo token en la API interna, accessToken en la Public API).
<inicio>	Valor de SELECT now() registrado antes de la acción; delimita los registros nuevos que crea el caso.
<referencia>, <id>, <R1>	Valores devueltos por un paso anterior del mismo caso; se capturan en ejecución y no se publican.
preparar_escenario.py	Restaura AH000000000001 (1000.00, mínimo 100.00, límite 500.00, retirado 0), MO000000000002 (500.00) y AH000000000003 (1000.00).

Los importes se escriben como decimales exactos. En cuerpos JSON de API interna convertir monto/fondoInicial/efectivoContado al número decimal exacto, conforme a OpenAPI; la API pública admite texto numérico. Los campos “perfil”, “intentos_previos”, “concurrencia” y los marcadores de fixture son instrucciones de preparación, no propiedades del request. Tokens, UUID de sesiones y referencias generadas se capturan en ejecución: su forma y relación se comprueban sin inventar un valor fijo.

PC-F indica: cuentas restauradas desde MockData_Sintetica.json, caja del cajero ABIERTA con Q2000.00, permisos efectivos y plan contable activo. Los tests se ejecutan aislados y registran sus conteos previos; el histórico masivo se usa para consultas y queda fuera del oráculo de contabilidad de las nuevas operaciones. Todos los casos tienen oráculo definido. Los identificadores {{rolCajeroId}}, {{permisoDepositoId}} y tokens se resuelven por las consultas/pasos exactos de su ficha; son valores generados por el propio entorno QA. La colección contiene plantillas, no un “Run All” autónomo.

Los casos de bitácora pueden compartir ejecución y evidencia con el caso funcional de la misma acción cuando coincidan datos y precondiciones. Sus aserciones de evento siguen identificadas por separado; no es necesario repetir una solicitud solo por existir dos IDs de cobertura.

3.7 Fichas completas de casos de prueba

REQ-01 · Autenticacion de usuario interno con emision de JWT y refresh token

TC-REQ01-001 · Login por username válido

Trazabilidad	REQ-01 → CP-REQ01-01 → TC-REQ01-001 Dato: DT-TC-REQ01-001
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	qa_cajero activo, desbloqueado, intentos_fallidos=0; no requiere Bearer.
Interfaz	POST http://localhost:15141/api/auth/login
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0 WHERE username='qa_cajero'; 2. En psql-QA registrar el instante de inicio: SELECT now(); 3. Enviar POST http://localhost:15141/api/auth/login con Content-Type: application/json, sin encabezado Authorization y cuerpo {"username": "qa_cajero", "password": "E2Qa2026*"}. 4. Verificar en la respuesta: HTTP 200; token y refreshToken no vacíos; usuario.username=qa_cajero; permisos contiene transacciones.depositos.crear. 5. En psql-QA ejecutar: SELECT bloqueado, bloqueado_hasta, intentos_fallidos, (SELECT count(*) FROM seguridad.sesiones s WHERE s.usuario_id=u.id AND s.estado AND s.fecha_inicio>=<inicio>) AS sesiones_nuevas FROM seguridad.usuarios u WHERE username='qa_cajero'; Comprobar: se crea sesión activa.
Datos de entrada	{"username": "qa_cajero", "password": "E2Qa2026*"}
Resultado esperado	HTTP 200; token y refreshToken no vacíos; usuario.username=qa_cajero; permisos contiene transacciones.depositos.crear; se crea sesión activa.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ01-002 · Contraseña incorrecta

Trazabilidad	REQ-01 → CP-REQ01-01 → TC-REQ01-002 Dato: DT-TC-REQ01-002
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	qa_cajero con intentos_fallidos=0; sin token.
Interfaz	POST http://localhost:15141/api/auth/login
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0 WHERE username='qa_cajero'; 2. En psql-QA registrar el instante de inicio: SELECT now(); 3. Enviar POST http://localhost:15141/api/auth/login con Content-Type: application/json, sin encabezado Authorization y cuerpo {"username": "qa_cajero", "password": "Incorrecta9*"}. 4. Verificar en la respuesta: HTTP 401; message="Credenciales incorrectas. Intento 1 de 5."; no token. 5. En psql-QA ejecutar: SELECT bloqueado, bloqueado_hasta, intentos_fallidos, (SELECT count(*) FROM seguridad.sesiones s WHERE s.usuario_id=u.id AND s.estado AND s.fecha_inicio>=<inicio>) AS sesiones_nuevas FROM seguridad.usuarios u WHERE username='qa_cajero'; Comprobar: intentos_fallidos=1. 6. Restaurar: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0 WHERE username='qa_cajero';
Datos de entrada	{"username": "qa_cajero", "password": "Incorrecta9*"}
Resultado esperado	HTTP 401; message="Credenciales incorrectas. Intento 1 de 5."; no token; intentos_fallidos=1.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ01-003 · Campos obligatorios vacíos

Trazabilidad	REQ-01 → CP-REQ01-02 → TC-REQ01-003 Dato: DT-TC-REQ01-003
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	Entorno QA E2 activo; datos sintéticos cargados; autenticación y permisos del perfil indicado en la entrada.
Interfaz	POST http://localhost:15141/api/auth/login
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0 WHERE username='qa_cajero'; 2. En psql-QA registrar el instante de inicio: SELECT now(); 3. Enviar POST http://localhost:15141/api/auth/login con Content-Type: application/json, sin encabezado Authorization y cuerpo {"username": "", "password": ""}. 4. Verificar en la respuesta: HTTP 400; message="Usuario/correo y contrasena son requeridos.". 5. En psql-QA ejecutar: SELECT bloqueado, bloqueado_hasta, intentos_fallidos, (SELECT count(*) FROM seguridad.sesiones s WHERE s.usuario_id=u.id AND s.estado AND s.fecha_inicio>=<inicio>) AS sesiones_nuevas FROM seguridad.usuarios u WHERE username='qa_cajero'; Comprobar: ninguna sesión nueva.
Datos de entrada	{"username": "", "password": ""}
Resultado esperado	HTTP 400; message="Usuario/correo y contrasena son requeridos."; ninguna sesión nueva.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ01-004 · Email tiene precedencia sobre username

Trazabilidad	REQ-01 → CP-REQ01-02 → TC-REQ01-004 Dato: DT-TC-REQ01-004
Técnica / prioridad	Tabla de decisión · Alta Estado: Diseñado; no ejecutado
Precondiciones	Entorno QA E2 activo; datos sintéticos cargados; autenticación y permisos del perfil indicado en la entrada.
Interfaz	POST http://localhost:15141/api/auth/login
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0 WHERE username='qa_cajero'; 2. En psql-QA registrar el instante de inicio: SELECT now(); 3. Enviar POST http://localhost:15141/api/auth/login con Content-Type: application/json, sin encabezado Authorization y cuerpo {"email": "qa_cliente_a@example.invalid", "username": "qa_cajero", "password": "E2Qa2026*"}. 4. Verificar en la respuesta: HTTP 200; usuario.username=qa_cliente_a; no autentica qa_cajero. 5. En psql-QA ejecutar: SELECT bloqueado, bloqueado_hasta, intentos_fallidos, (SELECT count(*) FROM seguridad.sesiones s WHERE s.usuario_id=u.id AND s.estado AND s.fecha_inicio>=<inicio>) AS sesiones_nuevas FROM seguridad.usuarios u WHERE username='qa_cajero'; Comprobar que no hay registros nuevos ni cambios respecto al estado del paso 1.
Datos de entrada	{"email": "qa_cliente_a@example.invalid", "username": "qa_cajero", "password": "E2Qa2026*"}
Resultado esperado	HTTP 200; usuario.username=qa_cliente_a; no autentica qa_cajero.
Base / bloqueo	AuthDtos.LoginRequest.Identificador; caracteriza precedencia técnica, pendiente de ratificación funcional H-01.

REQ-02 · Bloqueo de usuario por intentos fallidos consecutivos

TC-REQ02-001 · Umbral técnico N=5: intento 4

Trazabilidad	REQ-02 → CP-REQ02-01 → TC-REQ02-001 Dato: DT-TC-REQ02-001
Técnica / prioridad	Valores límite + Transición de estados · Alta Estado: Diseñado; no ejecutado
Precondiciones	Restablecer qa_bloqueo a estado activo y 3 intentos; Security__MaxFailedAccessAttempts=5.
Interfaz	POST http://localhost:15141/api/auth/login
Pasos	1. En psql-QA: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=3 WHERE username='qa_bloqueo'; 2. En psql-QA registrar el instante de inicio: SELECT now(); 3. Enviar POST http://localhost:15141/api/auth/login con Content-Type: application/json, sin encabezado Authorization y cuerpo {"username": "qa_bloqueo", "password": "Erronea9*"}. 4. Verificar en la respuesta: HTTP 401; message="Credenciales incorrectas. Intento 4 de 5.". 5. En psql-QA ejecutar: SELECT bloqueado, bloqueado_hasta, intentos_fallidos, (SELECT count(*) FROM seguridad.sesiones s WHERE s.usuario_id=u.id AND s.estado AND s.fecha_inicio>=<inicio>) AS sesiones_nuevas FROM seguridad.usuarios u WHERE username='qa_bloqueo'; Comprobar: bloqueado=false. 6. Restaurar: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo';
Datos de entrada	{"username": "qa_bloqueo", "password": "Erronea9*", "intentos_previos": 3}
Resultado esperado	HTTP 401; message="Credenciales incorrectas. Intento 4 de 5."; bloqueado=false.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-01. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ02-002 · Umbral técnico N=5: intento 5

Trazabilidad	REQ-02 → CP-REQ02-01 → TC-REQ02-002 Dato: DT-TC-REQ02-002
Técnica / prioridad	Valores límite + Transición de estados · Alta Estado: Diseñado; no ejecutado
Precondiciones	Restablecer qa_bloqueo a estado activo y 4 intentos; Security__MaxFailedAccessAttempts=5.
Interfaz	POST http://localhost:15141/api/auth/login
Pasos	1. En psql-QA: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=4 WHERE username='qa_bloqueo'; 2. En psql-QA registrar el instante de inicio: SELECT now(); 3. Enviar POST http://localhost:15141/api/auth/login con Content-Type: application/json, sin encabezado Authorization y cuerpo {"username": "qa_bloqueo", "password": "Erronea9*"}. 4. Verificar en la respuesta: HTTP 401; message="Credenciales incorrectas. El usuario se encuentra bloqueado. Por favor comuniquese con soporte.". 5. En psql-QA ejecutar: SELECT bloqueado, bloqueado_hasta, intentos_fallidos, (SELECT count(*) FROM seguridad.sesiones s WHERE s.usuario_id=u.id AND s.estado AND s.fecha_inicio>=<inicio>) AS sesiones_nuevas FROM seguridad.usuarios u WHERE username='qa_bloqueo'; Comprobar: bloqueado=true; sesiones activas del usuario=0. 6. Restaurar: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo';
Datos de entrada	{"username": "qa_bloqueo", "password": "Erronea9*", "intentos_previos": 4}
Resultado esperado	HTTP 401; message="Credenciales incorrectas. El usuario se encuentra bloqueado. Por favor comuniquese con soporte."; bloqueado=true; sesiones activas del usuario=0.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-01. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ02-003 · Login correcto posterior al bloqueo

Trazabilidad	REQ-02 → CP-REQ02-02 → TC-REQ02-003 Dato: DT-TC-REQ02-003
Técnica / prioridad	Transición de estados · Alta Estado: Diseñado; no ejecutado
Precondiciones	qa_bloqueo bloqueado=true, bloqueado_hasta=NOW()+INTERVAL 30 minutes; contraseña correcta.
Interfaz	POST http://localhost:15141/api/auth/login
Pasos	1. En psql-QA: UPDATE seguridad.usuarios SET bloqueado=true, intentos_fallidos=5, bloqueado_hasta=now()+interval '30 minutes' WHERE username='qa_bloqueo'; 2. En psql-QA registrar el instante de inicio: SELECT now(); 3. Enviar POST http://localhost:15141/api/auth/login con Content-Type: application/json, sin encabezado Authorization y cuerpo {"username": "qa_bloqueo", "password": "E2Qa2026*"}. 4. Verificar en la respuesta: HTTP 423; message="El usuario se encuentra bloqueado. Por favor comuniquese con soporte."; sin token. 5. En psql-QA ejecutar: SELECT bloqueado, bloqueado_hasta, intentos_fallidos, (SELECT count(*) FROM seguridad.sesiones s WHERE s.usuario_id=u.id AND s.estado AND s.fecha_inicio>=<inicio>) AS sesiones_nuevas FROM seguridad.usuarios u WHERE username='qa_bloqueo'; Comprobar que no hay registros nuevos ni cambios respecto al estado del paso 1. 6. Restaurar: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo';
Datos de entrada	{"username": "qa_bloqueo", "password": "E2Qa2026*"}
Resultado esperado	HTTP 423; message="El usuario se encuentra bloqueado. Por favor comuniquese con soporte."; sin token.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-01. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ02-004 · Desbloqueo al vencer los 30 minutos

Trazabilidad	REQ-02 → CP-REQ02-03 → TC-REQ02-004 Dato: DT-TC-REQ02-004
Técnica / prioridad	Transición de estados + Valores límite · Alta Estado: Diseñado; no ejecutado
Precondiciones	qa_bloqueo con cinco fallos previos, bloqueado=true y bloqueado_hasta fijado a NOW()-INTERVAL 1 second en QA; contraseña correcta E2Qa2026*.
Interfaz	POST http://localhost:15141/api/auth/login
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; 2. Enviar 5 veces POST http://localhost:15141/api/auth/login con cuerpo {"username":"qa_bloqueo","password":"Erronea9*"}; la quinta respuesta informa el bloqueo (HTTP 401). 3. En psql-QA verificar: SELECT bloqueado, bloqueado_hasta-now() FROM seguridad.usuarios WHERE username='qa_bloqueo'; esperar bloqueado=true e intervalo de 30 min ± 2 s. 4. En psql-QA (solo QA) adelantar el vencimiento: UPDATE seguridad.usuarios SET bloqueado_hasta=now()-interval '1 second' WHERE username='qa_bloqueo'; 5. Enviar POST http://localhost:15141/api/auth/login con cuerpo {"username":"qa_bloqueo","password":"E2Qa2026*"}. 6. Verificar en la respuesta: HTTP 200; token y refreshToken no vacíos. 7. En psql-QA ejecutar: SELECT bloqueado, bloqueado_hasta, intentos_fallidos FROM seguridad.usuarios WHERE username='qa_bloqueo'; esperar false, NULL y 0. 8. Restaurar: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo';
Datos de entrada	{"username": "qa_bloqueo", "password": "E2Qa2026*"}
Resultado esperado	HTTP 200; token y refreshToken no vacíos; bloqueado=false, bloqueado_hasta=NULL e intentos_fallidos=0; una sesión activa nueva.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-01. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ02-005 · No desbloquear antes del vencimiento

Trazabilidad	REQ-02 → CP-REQ02-03 → TC-REQ02-005 Dato: DT-TC-REQ02-005
Técnica / prioridad	Valores límite + Transición de estados · Alta Estado: Diseñado; no ejecutado
Precondiciones	qa_bloqueo bloqueado=true; bloqueado_hasta al menos 60 segundos después de iniciar la petición; reloj UTC del servidor registrado. El caso comprueba el lado anterior del límite; el posterior se cubre con TC-REQ02-004.
Interfaz	POST http://localhost:15141/api/auth/login
Pasos	1. En psql-QA: UPDATE seguridad.usuarios SET bloqueado=true, intentos_fallidos=5, bloqueado_hasta=now()+interval '5 minutes' WHERE username='qa_bloqueo'; (vence después de la petición). 2. En psql-QA registrar el instante de inicio: SELECT now(); 3. Enviar POST http://localhost:15141/api/auth/login con Content-Type: application/json, sin encabezado Authorization y cuerpo {"username": "qa_bloqueo", "password": "E2Qa2026*"}. 4. Verificar en la respuesta: HTTP 423; message="El usuario se encuentra bloqueado. Por favor comuniquese con soporte.". 5. En psql-QA ejecutar: SELECT bloqueado, bloqueado_hasta, intentos_fallidos, (SELECT count(*) FROM seguridad.sesiones s WHERE s.usuario_id=u.id AND s.estado AND s.fecha_inicio>=<inicio>) AS sesiones_nuevas FROM seguridad.usuarios u WHERE username='qa_bloqueo'; Comprobar: bloqueado=true y ninguna sesión nueva. 6. Restaurar: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo';
Datos de entrada	{"username": "qa_bloqueo", "password": "E2Qa2026*"}
Resultado esperado	HTTP 423; message="El usuario se encuentra bloqueado. Por favor comuniquese con soporte."; bloqueado=true y ninguna sesión nueva.
Base / bloqueo	Criterios aprobados por el equipo QA el 06-10-2026

REQ-03 · Politica de composicion e historial de contrasenas

TC-REQ03-001 · Composición de contraseña: Abcde1*

Trazabilidad	REQ-03 → CP-REQ03-01 → TC-REQ03-001 Dato: DT-TC-REQ03-001
Técnica / prioridad	Valores límite · Media Estado: Diseñado; no ejecutado
Precondiciones	Usuario qa_bloqueo activo, desbloqueado, con token; contraseña actual E2Qa2026*; cada variante usa usuario restaurado y contraseña nueva ausente del historial.
Interfaz	POST http://localhost:15141/api/auth/cambiar-password
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; DELETE FROM seguridad.password_historial WHERE usuario_id=(SELECT id FROM seguridad.usuarios WHERE username='qa_bloqueo'); 2. Guardar <hash_previo>: SELECT password_hash FROM seguridad.usuarios WHERE username='qa_bloqueo'; 3. Autenticar qa_bloqueo con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_bloqueo","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Enviar POST http://localhost:15141/api/auth/cambiar-password con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"passwordActual": "E2Qa2026*", "passwordNueva": "Abcde1*"}. 5. Verificar en la respuesta: HTTP 400; message="La contrasena debe tener al menos 8 caracteres.". 6. En psql-QA ejecutar: SELECT password_hash=<hash_previo> AS sin_cambio, (SELECT count(*) FROM seguridad.sesiones s WHERE s.usuario_id=u.id AND s.estado) AS sesiones_activas FROM seguridad.usuarios u WHERE username='qa_bloqueo'; Comprobar: hash y sesiones sin cambios. 7. Restaurar: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo';
Datos de entrada	{"passwordActual": "E2Qa2026*", "passwordNueva": "Abcde1*"}
Resultado esperado	HTTP 400; message="La contrasena debe tener al menos 8 caracteres."; hash y sesiones sin cambios.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-02. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ03-002 · Composición de contraseña: Abcdef1*

Trazabilidad	REQ-03 → CP-REQ03-01 → TC-REQ03-002 Dato: DT-TC-REQ03-002
Técnica / prioridad	Valores límite · Media Estado: Diseñado; no ejecutado
Precondiciones	Usuario qa_bloqueo activo, desbloqueado, con token; contraseña actual E2Qa2026*; cada variante usa usuario restaurado y contraseña nueva ausente del historial.
Interfaz	POST http://localhost:15141/api/auth/cambiar-password
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; DELETE FROM seguridad.password_historial WHERE usuario_id=(SELECT id FROM seguridad.usuarios WHERE username='qa_bloqueo'); 2. Guardar <hash_previo>: SELECT password_hash FROM seguridad.usuarios WHERE username='qa_bloqueo'; 3. Autenticar qa_bloqueo con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_bloqueo","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Enviar POST http://localhost:15141/api/auth/cambiar-password con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"passwordActual": "E2Qa2026*", "passwordNueva": "Abcdef1*"}. 5. Verificar en la respuesta: HTTP 200; message="Operacion realizada correctamente.". 6. En psql-QA ejecutar: SELECT password_hash=<hash_previo> AS sin_cambio, (SELECT count(*) FROM seguridad.sesiones s WHERE s.usuario_id=u.id AND s.estado) AS sesiones_activas FROM seguridad.usuarios u WHERE username='qa_bloqueo'; Comprobar: hash actualizado; sesiones previas revocadas. 7. Restaurar: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo';
Datos de entrada	{"passwordActual": "E2Qa2026*", "passwordNueva": "Abcdef1*"}
Resultado esperado	HTTP 200; message="Operacion realizada correctamente."; hash actualizado; sesiones previas revocadas.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-02. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ03-003 · Composición de contraseña: Abcdefg1*

Trazabilidad	REQ-03 → CP-REQ03-01 → TC-REQ03-003 Dato: DT-TC-REQ03-003
Técnica / prioridad	Valores límite · Media Estado: Diseñado; no ejecutado
Precondiciones	Usuario qa_bloqueo activo, desbloqueado, con token; contraseña actual E2Qa2026*; cada variante usa usuario restaurado y contraseña nueva ausente del historial.
Interfaz	POST http://localhost:15141/api/auth/cambiar-password
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; DELETE FROM seguridad.password_historial WHERE usuario_id=(SELECT id FROM seguridad.usuarios WHERE username='qa_bloqueo'); 2. Guardar <hash_previo>: SELECT password_hash FROM seguridad.usuarios WHERE username='qa_bloqueo'; 3. Autenticar qa_bloqueo con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_bloqueo","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Enviar POST http://localhost:15141/api/auth/cambiar-password con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"passwordActual": "E2Qa2026*", "passwordNueva": "Abcdefg1*"}. 5. Verificar en la respuesta: HTTP 200; message="Operacion realizada correctamente.". 6. En psql-QA ejecutar: SELECT password_hash=<hash_previo> AS sin_cambio, (SELECT count(*) FROM seguridad.sesiones s WHERE s.usuario_id=u.id AND s.estado) AS sesiones_activas FROM seguridad.usuarios u WHERE username='qa_bloqueo'; Comprobar: hash actualizado; sesiones previas revocadas. 7. Restaurar: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo';
Datos de entrada	{"passwordActual": "E2Qa2026*", "passwordNueva": "Abcdefg1*"}
Resultado esperado	HTTP 200; message="Operacion realizada correctamente."; hash actualizado; sesiones previas revocadas.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-02. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ03-004 · Composición de contraseña: abcdef1*

Trazabilidad	REQ-03 → CP-REQ03-01 → TC-REQ03-004 Dato: DT-TC-REQ03-004
Técnica / prioridad	Partición de equivalencia · Media Estado: Diseñado; no ejecutado
Precondiciones	Usuario qa_bloqueo activo, desbloqueado, con token; contraseña actual E2Qa2026*; cada variante usa usuario restaurado y contraseña nueva ausente del historial.
Interfaz	POST http://localhost:15141/api/auth/cambiar-password
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; DELETE FROM seguridad.password_historial WHERE usuario_id=(SELECT id FROM seguridad.usuarios WHERE username='qa_bloqueo'); 2. Guardar <hash_previo>: SELECT password_hash FROM seguridad.usuarios WHERE username='qa_bloqueo'; 3. Autenticar qa_bloqueo con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_bloqueo","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Enviar POST http://localhost:15141/api/auth/cambiar-password con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"passwordActual": "E2Qa2026*", "passwordNueva": "abcdef1*"}. 5. Verificar en la respuesta: HTTP 400; message="La contrasena debe incluir mayusculas, minusculas y numeros.". 6. En psql-QA ejecutar: SELECT password_hash=<hash_previo> AS sin_cambio, (SELECT count(*) FROM seguridad.sesiones s WHERE s.usuario_id=u.id AND s.estado) AS sesiones_activas FROM seguridad.usuarios u WHERE username='qa_bloqueo'; Comprobar: hash y sesiones sin cambios. 7. Restaurar: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo';
Datos de entrada	{"passwordActual": "E2Qa2026*", "passwordNueva": "abcdef1*"}
Resultado esperado	HTTP 400; message="La contrasena debe incluir mayusculas, minusculas y numeros."; hash y sesiones sin cambios.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-02. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ03-005 · Composición de contraseña: ABCDEF1*

Trazabilidad	REQ-03 → CP-REQ03-01 → TC-REQ03-005 Dato: DT-TC-REQ03-005
Técnica / prioridad	Valores límite · Media Estado: Diseñado; no ejecutado
Precondiciones	Usuario qa_bloqueo activo, desbloqueado, con token; contraseña actual E2Qa2026*; cada variante usa usuario restaurado y contraseña nueva ausente del historial.
Interfaz	POST http://localhost:15141/api/auth/cambiar-password
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; DELETE FROM seguridad.password_historial WHERE usuario_id=(SELECT id FROM seguridad.usuarios WHERE username='qa_bloqueo'); 2. Guardar <hash_previo>: SELECT password_hash FROM seguridad.usuarios WHERE username='qa_bloqueo'; 3. Autenticar qa_bloqueo con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_bloqueo","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Enviar POST http://localhost:15141/api/auth/cambiar-password con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"passwordActual": "E2Qa2026*", "passwordNueva": "ABCDEF1*"}. 5. Verificar en la respuesta: HTTP 400; message="La contrasena debe incluir mayusculas, minusculas y numeros.". 6. En psql-QA ejecutar: SELECT password_hash=<hash_previo> AS sin_cambio, (SELECT count(*) FROM seguridad.sesiones s WHERE s.usuario_id=u.id AND s.estado) AS sesiones_activas FROM seguridad.usuarios u WHERE username='qa_bloqueo'; Comprobar: hash y sesiones sin cambios. 7. Restaurar: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo';
Datos de entrada	{"passwordActual": "E2Qa2026*", "passwordNueva": "ABCDEF1*"}
Resultado esperado	HTTP 400; message="La contrasena debe incluir mayusculas, minusculas y numeros."; hash y sesiones sin cambios.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-02. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ03-006 · Composición de contraseña: Abcdefg*

Trazabilidad	REQ-03 → CP-REQ03-01 → TC-REQ03-006 Dato: DT-TC-REQ03-006
Técnica / prioridad	Partición de equivalencia · Media Estado: Diseñado; no ejecutado
Precondiciones	Usuario qa_bloqueo activo, desbloqueado, con token; contraseña actual E2Qa2026*; cada variante usa usuario restaurado y contraseña nueva ausente del historial.
Interfaz	POST http://localhost:15141/api/auth/cambiar-password
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; DELETE FROM seguridad.password_historial WHERE usuario_id=(SELECT id FROM seguridad.usuarios WHERE username='qa_bloqueo'); 2. Guardar <hash_previo>: SELECT password_hash FROM seguridad.usuarios WHERE username='qa_bloqueo'; 3. Autenticar qa_bloqueo con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_bloqueo","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Enviar POST http://localhost:15141/api/auth/cambiar-password con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"passwordActual": "E2Qa2026*", "passwordNueva": "Abcdefg*"}. 5. Verificar en la respuesta: HTTP 400; message="La contrasena debe incluir mayusculas, minusculas y numeros.". 6. En psql-QA ejecutar: SELECT password_hash=<hash_previo> AS sin_cambio, (SELECT count(*) FROM seguridad.sesiones s WHERE s.usuario_id=u.id AND s.estado) AS sesiones_activas FROM seguridad.usuarios u WHERE username='qa_bloqueo'; Comprobar: hash y sesiones sin cambios. 7. Restaurar: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo';
Datos de entrada	{"passwordActual": "E2Qa2026*", "passwordNueva": "Abcdefg*"}
Resultado esperado	HTTP 400; message="La contrasena debe incluir mayusculas, minusculas y numeros."; hash y sesiones sin cambios.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-02. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ03-007 · Reutilización de una contraseña reciente

Trazabilidad	REQ-03 → CP-REQ03-02 → TC-REQ03-007 Dato: DT-TC-REQ03-007
Técnica / prioridad	Partición de equivalencia · Media Estado: Diseñado; no ejecutado
Precondiciones	qa_bloqueo restaurado; hash de Anterior8* entre las últimas 5 contraseñas; token válido.
Interfaz	POST http://localhost:15141/api/auth/cambiar-password
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; DELETE FROM seguridad.password_historial WHERE usuario_id=(SELECT id FROM seguridad.usuarios WHERE username='qa_bloqueo'); 2. En psql-QA sembrar el historial: INSERT INTO seguridad.password_historial(id,usuario_id,password_hash,created_at) VALUES (gen_random_uuid(), (SELECT id FROM seguridad.usuarios WHERE username='qa_bloqueo'),crypt('Anterior8*', gen_salt('bf',10)),now()); 3. Guardar <hash_previo>: SELECT password_hash FROM seguridad.usuarios WHERE username='qa_bloqueo'; 4. Autenticar qa_bloqueo con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_bloqueo","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 5. Enviar POST http://localhost:15141/api/auth/cambiar-password con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"passwordActual": "E2Qa2026*", "passwordNueva": "Anterior8*"}. 6. Verificar en la respuesta: HTTP 400; message="No puedes reutilizar una contrasena reciente.". 7. En psql-QA ejecutar: SELECT password_hash=<hash_previo> AS sin_cambio, (SELECT count(*) FROM seguridad.sesiones s WHERE s.usuario_id=u.id AND s.estado) AS sesiones_activas FROM seguridad.usuarios u WHERE username='qa_bloqueo'; Comprobar: password_hash sin cambios. 8. Restaurar: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo';
Datos de entrada	{"passwordActual": "E2Qa2026*", "passwordNueva": "Anterior8*"}
Resultado esperado	HTTP 400; message="No puedes reutilizar una contrasena reciente."; password_hash sin cambios.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-02. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

REQ-04 · Recuperacion de contrasena mediante token temporal

TC-REQ04-001 · Token de recuperación usado por segunda vez

Trazabilidad	REQ-04 → CP-REQ04-01 → TC-REQ04-001 Dato: DT-TC-REQ04-001
Técnica / prioridad	Transición de estados · Media Estado: Diseñado; no ejecutado
Precondiciones	Entorno QA E2 activo; datos sintéticos cargados; autenticación y permisos del perfil indicado en la entrada.
Interfaz	POST http://localhost:15141/api/auth/restablecer-password
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; 2. Enviar POST http://localhost:15141/api/auth/solicitar-recuperacion-password con cuerpo {"identificador":"qa_bloqueo@example.invalid"}; verificar HTTP 200 y guardar tokenRecuperacion como <tokenRecuperacion> (sin registrarlo en evidencia). 3. Enviar POST http://localhost:15141/api/auth/restablecer-password con cuerpo {"token":"<tokenRecuperacion>","passwordNueva":"Recupera9*"}; verificar HTTP 200. 4. Enviar de nuevo POST http://localhost:15141/api/auth/restablecer-password con el mismo cuerpo; verificar HTTP 400 y message="Token invalido o expirado.". 5. En psql-QA ejecutar: SELECT usado FROM seguridad.tokens_recuperacion WHERE usuario_id=(SELECT id FROM seguridad.usuarios WHERE username='qa_bloqueo') ORDER BY created_at DESC LIMIT 1; esperar true. Comprobar con un login usando Recupera9* (HTTP 200) que solo hubo una actualización. 6. Restaurar: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo';
Datos de entrada	{"token": "{{tokenRecuperacion}}", "passwordNueva": "Recupera9*"}
Resultado esperado	Primer restablecimiento HTTP 200; segundo con el mismo token HTTP 400 y message="Token invalido o expirado."; token usado=true; no segunda actualización.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-02. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ04-002 · Token vencido

Trazabilidad	REQ-04 → CP-REQ04-01 → TC-REQ04-002 Dato: DT-TC-REQ04-002
Técnica / prioridad	Valores límite + Transición de estados · Media Estado: Diseñado; no ejecutado
Precondiciones	Token QA real creado en esta BD; fijar su expiración a un segundo antes de la solicitud; no usar tokens productivos.
Interfaz	POST http://localhost:15141/api/auth/restablecer-password
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; 2. Solicitar token con POST http://localhost:15141/api/auth/solicitar-recuperacion-password y cuerpo {"identificador":"qa_bloqueo@example.invalid"}; guardar tokenRecuperacion. 3. Guardar <hash_previo>: SELECT password_hash FROM seguridad.usuarios WHERE username='qa_bloqueo'; 4. Enviar POST http://localhost:15141/api/auth/restablecer-password con Content-Type: application/json, sin encabezado Authorization y cuerpo {"token": "{{tokenRecuperacion}}", "passwordNueva": "Recupera9*"}. 5. Verificar en la respuesta: HTTP 400; message="Token invalido o expirado.". 6. En psql-QA ejecutar: SELECT password_hash=<hash_previo> AS sin_cambio, (SELECT count(*) FROM seguridad.sesiones s WHERE s.usuario_id=u.id AND s.estado) AS sesiones_activas FROM seguridad.usuarios u WHERE username='qa_bloqueo'; Comprobar: contraseña sin cambios. 7. Restaurar: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo';
Datos de entrada	{"token": "{{tokenRecuperacion}}", "passwordNueva": "Recupera9*"}
Resultado esperado	HTTP 400; message="Token invalido o expirado."; contraseña sin cambios.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-02. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ04-003 · Token académico de recuperación: canal y vigencia aprobados

Trazabilidad	REQ-04 → CP-REQ04-02 → TC-REQ04-003 Dato: DT-TC-REQ04-003
Técnica / prioridad	Tabla de decisión · Media Estado: Diseñado; no ejecutado
Precondiciones	Entorno QA E2 activo; datos sintéticos cargados; autenticación y permisos del perfil indicado en la entrada.
Interfaz	POST http://localhost:15141/api/auth/solicitar-recuperacion-password
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; 2. En psql-QA registrar el instante de inicio: SELECT now(); 3. Enviar POST http://localhost:15141/api/auth/solicitar-recuperacion-password sin Authorization y cuerpo {"identificador":"qa_bloqueo@example.invalid"}. 4. Verificar en la respuesta: HTTP 200; tokenRecuperacion no vacío; expiraEn entre <inicio>+2 h y <fin>+2 h (tolerancia 2 s). 5. En psql-QA ejecutar: SELECT token_hash<>'<tokenRecuperacion>' AS hash_distinto, usado, fecha_expiracion FROM seguridad.tokens_recuperacion WHERE usuario_id=(SELECT id FROM seguridad.usuarios WHERE username='qa_bloqueo') ORDER BY created_at DESC LIMIT 1; esperar hash_distinto=true y usado=false. 6. Enviar POST http://localhost:15141/api/auth/restablecer-password con {"token":"<tokenRecuperacion>","passwordNueva":"Recupera9*"}; verificar HTTP 200 (el segundo uso se cubre en TC-REQ04-001). 7. Restaurar: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo';
Datos de entrada	{"identificador": "qa_bloqueo@example.invalid"}
Resultado esperado	HTTP 200; tokenRecuperacion no vacío únicamente en QA académico; expiraEn entre inicio+2h y fin+2h (tolerancia 2 s); seguridad.tokens_recuperacion guarda hash distinto del token y usado=false; nunca se registra el token íntegro en evidencia.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-02. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

REQ-05 · Rotacion de refresh token y revocacion de sesiones

TC-REQ05-001 · Rotación e invalidez del refresh anterior

Trazabilidad	REQ-05 → CP-REQ05-01 → TC-REQ05-001 Dato: DT-TC-REQ05-001
Técnica / prioridad	Transición de estados · Alta Estado: Diseñado; no ejecutado
Precondiciones	Entorno QA E2 activo; datos sintéticos cargados; autenticación y permisos del perfil indicado en la entrada.
Interfaz	POST http://localhost:15141/api/auth/refresh-token
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0 WHERE username='qa_cajero'; 2. Enviar POST http://localhost:15141/api/auth/login con cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; guardar refreshToken como R1. 3. Enviar POST http://localhost:15141/api/auth/refresh-token con cuerpo {"refreshToken":"<R1>"}; verificar HTTP 200 y guardar el nuevo refreshToken como R2. 4. Comprobar que R2 es distinto de R1. 5. Enviar POST http://localhost:15141/api/auth/refresh-token con cuerpo {"refreshToken":"<R1>"}. 6. Verificar en la respuesta: HTTP 401; message="Refresh token invalido o expirado.". No registrar R1 ni R2 en la evidencia.
Datos de entrada	{"refreshToken": "{{refreshTokenCajero}}"}
Resultado esperado	Primer refresh HTTP 200 con refresh nuevo distinto; reutilización del anterior HTTP 401 y message="Refresh token invalido o expirado.".
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-02. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ05-002 · Cambio de contraseña revoca todas las sesiones

Trazabilidad	REQ-05 → CP-REQ05-02 → TC-REQ05-002 Dato: DT-TC-REQ05-002
Técnica / prioridad	Transición de estados · Alta Estado: Diseñado; no ejecutado
Precondiciones	Entorno QA E2 activo; datos sintéticos cargados; autenticación y permisos del perfil indicado en la entrada.
Interfaz	POST http://localhost:15141/api/auth/cambiar-password
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; DELETE FROM seguridad.password_historial WHERE usuario_id=(SELECT id FROM seguridad.usuarios WHERE username='qa_bloqueo'); 2. Enviar dos veces POST http://localhost:15141/api/auth/login con cuerpo {"username":"qa_bloqueo","password":"E2Qa2026*"}; guardar los JWT como J1 y J2. 3. En psql-QA verificar: SELECT count(*) FROM seguridad.sesiones WHERE usuario_id=(SELECT id FROM seguridad.usuarios WHERE username='qa_bloqueo') AND estado; esperar 2. 4. Enviar POST http://localhost:15141/api/auth/cambiar-password con Authorization: Bearer <J1> y cuerpo {"passwordActual":"E2Qa2026*","passwordNueva":"SesionNueva9*"}; verificar HTTP 200. 5. En psql-QA repetir la consulta de sesiones activas; esperar 0. 6. Enviar GET http://localhost:15141/api/auth/me con Authorization: Bearer <J1> y luego con <J2>; verificar HTTP 401 en ambos. 7. Restaurar: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo';
Datos de entrada	{"perfil": "qa_bloqueo", "passwordActual": "E2Qa2026*", "passwordNueva": "SesionNueva9*"}
Resultado esperado	Cambio HTTP 200; seguridad.sesiones activas del usuario=0; GET /api/auth/me con cada JWT anterior HTTP 401.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-02. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ05-003 · Refresh token vacío

Trazabilidad	REQ-05 → CP-REQ05-03 → TC-REQ05-003 Dato: DT-TC-REQ05-003
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	API interna QA activa; no requiere Bearer. qa_cajero activo y desbloqueado.
Interfaz	POST http://localhost:15141/api/auth/refresh-token
Pasos	1. En psql-QA registrar el conteo previo: SELECT count(*) FROM seguridad.sesiones s JOIN seguridad.usuarios u ON u.id=s.usuario_id WHERE u.username='qa_cajero' AND s.estado; 2. Enviar POST http://localhost:15141/api/auth/refresh-token con Content-Type: application/json, sin Authorization y cuerpo {"refreshToken": ""}. 3. Verificar HTTP 400 y cuerpo exacto {"message":"Refresh token requerido."}; confirmar que no aparecen las propiedades token ni refreshToken. 4. Repetir la consulta del paso 1 y comprobar que el conteo es idéntico.
Datos de entrada	{"refreshToken": ""}
Resultado esperado	HTTP 400; message="Refresh token requerido."; sin token ni refreshToken en el cuerpo; el número de sesiones activas de qa_cajero no cambia.
Base / bloqueo	Criterios aprobados por el equipo QA el 06-10-2026

TC-REQ05-004 · Refresh token inexistente

Trazabilidad	REQ-05 → CP-REQ05-03 → TC-REQ05-004 Dato: DT-TC-REQ05-004
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	API interna QA activa; el valor E2-QA-TOKEN-INEXISTENTE-0001 nunca fue emitido (su hash no existe en seguridad.sesiones).
Interfaz	POST http://localhost:15141/api/auth/refresh-token
Pasos	1. En psql-QA registrar el conteo previo: SELECT count(*) FROM seguridad.sesiones s JOIN seguridad.usuarios u ON u.id=s.usuario_id WHERE u.username='qa_cajero' AND s.estado; 2. Enviar POST http://localhost:15141/api/auth/refresh-token con Content-Type: application/json, sin Authorization y cuerpo {"refreshToken": "E2-QA-TOKEN-INEXISTENTE-0001"}. 3. Verificar HTTP 401 y cuerpo exacto {"message":"Refresh token invalido o expirado."}. 4. Repetir la consulta del paso 1 y comprobar que el conteo es idéntico.
Datos de entrada	{"refreshToken": "E2-QA-TOKEN-INEXISTENTE-0001"}
Resultado esperado	HTTP 401; message="Refresh token invalido o expirado."; sin token ni refreshToken; ninguna sesión rotada ni creada.
Base / bloqueo	Criterios aprobados por el equipo QA el 06-10-2026

TC-REQ05-005 · Refresh token reutilizado después de logout

Trazabilidad	REQ-05 → CP-REQ05-01 → TC-REQ05-005 Dato: DT-TC-REQ05-005
Técnica / prioridad	Transición de estados · Alta Estado: Diseñado; no ejecutado
Precondiciones	qa_cajero activo y desbloqueado; sin sesiones de prueba previas abiertas.
Interfaz	POST http://localhost:15141/api/auth/refresh-token
Pasos	1. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; guardar token (JWT) y refreshToken (R1) en variables, sin registrarlos en evidencia. 2. Enviar POST http://localhost:15141/api/auth/logout con Authorization: Bearer <JWT> y cuerpo {"refreshToken": "<R1>"}; verificar HTTP 200. 3. Enviar POST http://localhost:15141/api/auth/refresh-token con cuerpo {"refreshToken": "<R1>"}; verificar HTTP 401 y message="Refresh token invalido o expirado.". 4. En psql-QA ejecutar: SELECT estado, fecha_cierre FROM seguridad.sesiones s JOIN seguridad.usuarios u ON u.id=s.usuario_id WHERE u.username='qa_cajero' ORDER BY fecha_inicio DESC LIMIT 1; esperar estado=false y fecha_cierre no nula.
Datos de entrada	{"refreshToken": "{{refreshTokenCajero}}"}
Resultado esperado	Logout HTTP 200; refresh posterior con el mismo valor HTTP 401 y message="Refresh token invalido o expirado."; la sesión queda estado=false con fecha_cierre no nula.
Base / bloqueo	Criterios aprobados por el equipo QA el 06-10-2026

REQ-06 · Autorizacion granular por permisos efectivos

TC-REQ06-001 · Acceso anónimo a clientes

Trazabilidad	REQ-06 → CP-REQ06-01 → TC-REQ06-001 Dato: DT-TC-REQ06-001
Técnica / prioridad	Tabla de decisión · Alta Estado: Diseñado; no ejecutado
Precondiciones	No enviar token; ausencia de cookies. Oráculo REQ-06 de Entrega 1 incluso si el código actual responde 200.
Interfaz	GET http://localhost:15141/api/clientes
Pasos	1. Enviar GET http://localhost:15141/api/clientes sin encabezado Authorization. 2. Verificar en la respuesta: HTTP 401; no lista de clientes ni configuración ACH en el cuerpo.
Datos de entrada	{"Authorization": "ausente"}
Resultado esperado	HTTP 401; no lista de clientes ni configuración ACH en el cuerpo.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-05. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ06-002 · Acceso anónimo a ach/bancos

Trazabilidad	REQ-06 → CP-REQ06-01 → TC-REQ06-002 Dato: DT-TC-REQ06-002
Técnica / prioridad	Tabla de decisión · Alta Estado: Diseñado; no ejecutado
Precondiciones	No enviar token; ausencia de cookies. Oráculo REQ-06 de Entrega 1 incluso si el código actual responde 200.
Interfaz	GET http://localhost:15141/api/ach/bancos
Pasos	1. Enviar GET http://localhost:15141/api/ach/bancos sin encabezado Authorization. 2. Verificar en la respuesta: HTTP 401; no lista de clientes ni configuración ACH en el cuerpo.
Datos de entrada	{"Authorization": "ausente"}
Resultado esperado	HTTP 401; no lista de clientes ni configuración ACH en el cuerpo.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-05. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ06-003 · JWT válido sin permiso de depósito

Trazabilidad	REQ-06 → CP-REQ06-02 → TC-REQ06-003 Dato: DT-TC-REQ06-003
Técnica / prioridad	Tabla de decisión · Alta Estado: Diseñado; no ejecutado
Precondiciones	Entorno QA E2 activo; datos sintéticos cargados; autenticación y permisos del perfil indicado en la entrada.
Interfaz	POST http://localhost:15141/api/transacciones/deposito
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA registrar el instante de inicio: SELECT now(); 3. Autenticar qa_sin_permiso con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_sin_permiso","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Enviar POST http://localhost:15141/api/transacciones/deposito con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"numeroCuenta": "AH000000000001", "monto": 10.00}. 5. Verificar en la respuesta: HTTP 403. 6. En psql-QA ejecutar: SELECT c.numero_cuenta, c.saldo, c.saldo_disponible, c.monto_retirado_hoy, (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.created_at>=<inicio>) AS movs, (SELECT count(*) FROM contabilidad.asientos a JOIN transacciones.movimientos m ON m.id=a.movimiento_id WHERE m.cuenta_id=c.id AND a.created_at>=<inicio>) AS asientos FROM cuentas.cuentas c WHERE c.numero_cuenta IN ('AH000000000001','MO000000000002','AH000000000003') ORDER BY 1; Comprobar: saldo, movimientos y asientos sin cambios. 7. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_sin_permiso", "numeroCuenta": "AH000000000001", "monto": "10.00"}
Resultado esperado	HTTP 403; saldo, movimientos y asientos sin cambios.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-05. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ06-004 · Permiso efectivo autoriza depósito

Trazabilidad	REQ-06 → CP-REQ06-02 → TC-REQ06-004 Dato: DT-TC-REQ06-004
Técnica / prioridad	Tabla de decisión · Alta Estado: Diseñado; no ejecutado
Precondiciones	PC-F: restaurar las cuentas implicadas a los valores de MockData_Sintetica.json; caja del cajero abierta con fondo Q2000.00; usuario qa_cajero con permisos efectivos. Ejecutar cada variante desde esa misma línea base.
Interfaz	POST http://localhost:15141/api/transacciones/deposito
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; 3. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Abrir caja con POST http://localhost:15141/api/cajas/abrir, Bearer de qa_cajero y cuerpo {"fondoInicial": 2000.00}; verificar HTTP 200. 5. En psql-QA registrar el instante de inicio: SELECT now(); 6. Enviar POST http://localhost:15141/api/transacciones/deposito con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"numeroCuenta": "AH000000000001", "monto": 10.00}. 7. Verificar en la respuesta: HTTP 200; saldoNuevo=1010.00; referencia no vacía. 8. En psql-QA ejecutar: SELECT c.numero_cuenta, c.saldo, c.saldo_disponible, c.monto_retirado_hoy, (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.created_at>=<inicio>) AS movs, (SELECT count(*) FROM contabilidad.asientos a JOIN transacciones.movimientos m ON m.id=a.movimiento_id WHERE m.cuenta_id=c.id AND a.created_at>=<inicio>) AS asientos FROM cuentas.cuentas c WHERE c.numero_cuenta IN ('AH000000000001','MO000000000002','AH000000000003') ORDER BY 1; Comprobar: un movimiento DEPOSITO y partida doble por Q10.00. 9. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_cajero", "numeroCuenta": "AH000000000001", "monto": "10.00"}
Resultado esperado	HTTP 200; saldoNuevo=1010.00; referencia no vacía; un movimiento DEPOSITO y partida doble por Q10.00.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-05. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

REQ-07 · Bitacora de eventos de seguridad

TC-REQ07-001 · Persistencia de evento LOGIN_EXITOSO

Trazabilidad	REQ-07 → CP-REQ07-01 → TC-REQ07-001 Dato: DT-TC-REQ07-001
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	qa_bloqueo restaurado; registrar conteo previo del evento.
Interfaz	POST http://localhost:15141/api/auth/login
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; También UPDATE seguridad.usuarios SET intentos_fallidos=0 WHERE username='qa_externo'; 2. En psql-QA registrar el instante de inicio: SELECT now(); Conteo previo: SELECT count(*) FROM seguridad.eventos_seguridad WHERE evento='LOGIN_EXITOSO'; 3. Enviar POST http://localhost:15141/api/auth/login con Content-Type: application/json, sin encabezado Authorization, User-Agent: E2-QA/1.0 y cuerpo {"username": "qa_bloqueo", "password": "E2Qa2026*"}. 4. Verificar en la respuesta: Login HTTP 200. 5. En psql-QA ejecutar: SELECT count(*), bool_and(ip_address IS NOT NULL), max(user_agent), bool_and(exitoso) FROM seguridad.eventos_seguridad WHERE evento='LOGIN_EXITOSO' AND created_at>=<inicio>; Comprobar: incremento de un evento LOGIN_EXITOSO con usuario QA, IP no vacía, user_agent=E2-QA/1.0, exitoso=true y fecha entre inicio/fin de petición. 6. Restaurar: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo';
Datos de entrada	{"username": "qa_bloqueo", "password": "E2Qa2026*", "User-Agent": "E2-QA/1.0"}
Resultado esperado	Login HTTP 200; incremento de un evento LOGIN_EXITOSO con usuario QA, IP no vacía, user_agent=E2-QA/1.0, exitoso=true y fecha entre inicio/fin de petición.
Base / bloqueo	D-09: inventario técnico de 20 eventos reconciliado con EventosSeguridad.cs.

TC-REQ07-002 · Persistencia de evento LOGIN_FALLIDO

Trazabilidad	REQ-07 → CP-REQ07-01 → TC-REQ07-002 Dato: DT-TC-REQ07-002
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	qa_bloqueo restaurado; registrar conteo previo del evento.
Interfaz	POST http://localhost:15141/api/auth/login
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; También UPDATE seguridad.usuarios SET intentos_fallidos=0 WHERE username='qa_externo'; 2. En psql-QA registrar el instante de inicio: SELECT now(); Conteo previo: SELECT count(*) FROM seguridad.eventos_seguridad WHERE evento='LOGIN_FALLIDO'; 3. Enviar POST http://localhost:15141/api/auth/login con Content-Type: application/json, sin encabezado Authorization, User-Agent: E2-QA/1.0 y cuerpo {"username": "qa_bloqueo", "password": "Erronea9*"}. 4. Verificar en la respuesta: Login HTTP 401. 5. En psql-QA ejecutar: SELECT count(*), bool_and(ip_address IS NOT NULL), max(user_agent), bool_and(exitoso) FROM seguridad.eventos_seguridad WHERE evento='LOGIN_FALLIDO' AND created_at>=<inicio>; Comprobar: incremento de un evento LOGIN_FALLIDO con usuario QA, IP no vacía, user_agent=E2-QA/1.0, exitoso=false y fecha entre inicio/fin de petición. 6. Restaurar: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo';
Datos de entrada	{"username": "qa_bloqueo", "password": "Erronea9*", "User-Agent": "E2-QA/1.0"}
Resultado esperado	Login HTTP 401; incremento de un evento LOGIN_FALLIDO con usuario QA, IP no vacía, user_agent=E2-QA/1.0, exitoso=false y fecha entre inicio/fin de petición.
Base / bloqueo	D-09: inventario técnico de 20 eventos reconciliado con EventosSeguridad.cs.

TC-REQ07-003 · Reconciliación del catálogo vigente de 20 eventos

Trazabilidad	REQ-07 → CP-REQ07-02 → TC-REQ07-003 Dato: DT-TC-REQ07-003
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	Entorno QA E2 activo; datos sintéticos cargados; autenticación y permisos del perfil indicado en la entrada.
Pasos	1. Desde la raíz del repositorio ejecutar: grep -c "public const string" Backend/BancoERP.Domain/Constants/EventosSeguridad.cs; esperar 20. 2. Listar los usos: grep -rn "EventosSeguridad\." Backend/BancoERP.Application/Services; cada una de las 20 constantes debe aparecer al menos una vez. 3. Abrir 03_Evidencias_y_MockData/Catalogo_Eventos_QA.json y comprobar que eventos_vigentes=20, que los 20 códigos son únicos y que cada uno tiene un ID de caso existente y al menos un punto de emisión. 4. Ejecutar python3 scripts/validar.py y verificar el control "20 eventos únicos con casos y puntos de emisión" = OK.
Datos de entrada	{"archivo": "Backend/BancoERP.Domain/Constants/EventosSeguridad.cs", "eventos_vigentes": 20, "eventos_E1": 21}
Resultado esperado	El parser obtiene exactamente 20 códigos únicos; cada código tiene escenario de prueba y punto de emisión identificado; cero eventos sin caso y cero casos con códigos ausentes de la fuente. Registrar la actualización del inventario E1 de 21 a 20 conservando el valor original como historia.
Base / bloqueo	D-09: inventario técnico de 20 eventos reconciliado con EventosSeguridad.cs.

TC-REQ07-004 · Evento LOGOUT

Trazabilidad	REQ-07 → CP-REQ07-04 → TC-REQ07-004 Dato: DT-TC-REQ07-004
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	Crear sesión qa_bloqueo; capturar su refresh/JWT en memoria. Medir conteo después de la preparación, justo antes de la acción objetivo; usar header User-Agent: E2-QA/2.0.
Interfaz	POST http://localhost:15141/api/auth/logout
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; También UPDATE seguridad.usuarios SET intentos_fallidos=0 WHERE username='qa_externo'; 2. Autenticar qa_bloqueo con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_bloqueo","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token> y refreshToken como {{refreshTokenBloqueo}}. 3. En psql-QA registrar el instante de inicio: SELECT now(); Conteo previo: SELECT count(*) FROM seguridad.eventos_seguridad WHERE evento='LOGOUT'; 4. Enviar POST http://localhost:15141/api/auth/logout con Content-Type: application/json, Authorization: Bearer <token>, User-Agent: E2-QA/2.0 y cuerpo {"refreshToken": "{{refreshTokenBloqueo}}"}. 5. Verificar en la respuesta: HTTP 200. 6. En psql-QA ejecutar: SELECT count(*), bool_and(ip_address IS NOT NULL), max(user_agent), bool_and(exitoso) FROM seguridad.eventos_seguridad WHERE evento='LOGOUT' AND created_at>=<inicio>; Comprobar: el conteo de seguridad.eventos_seguridad para evento=LOGOUT aumenta exactamente 1; exitoso=true; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción. 7. Restaurar: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo';
Datos de entrada	{"perfil": "qa_bloqueo", "refreshToken": "{{refreshTokenBloqueo}}"}
Resultado esperado	HTTP 200; el conteo de seguridad.eventos_seguridad para evento=LOGOUT aumenta exactamente 1; exitoso=true; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción.
Base / bloqueo	D-09: inventario técnico de 20 eventos reconciliado con EventosSeguridad.cs.

TC-REQ07-005 · Evento CAMBIO_PASSWORD

Trazabilidad	REQ-07 → CP-REQ07-04 → TC-REQ07-005 Dato: DT-TC-REQ07-005
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	qa_bloqueo restaurado; clave nueva fuera del historial. Medir conteo después de la preparación, justo antes de la acción objetivo; usar header User-Agent: E2-QA/2.0.
Interfaz	POST http://localhost:15141/api/auth/cambiar-password
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; También UPDATE seguridad.usuarios SET intentos_fallidos=0 WHERE username='qa_externo'; 2. En psql-QA registrar el instante de inicio: SELECT now(); Conteo previo: SELECT count(*) FROM seguridad.eventos_seguridad WHERE evento='CAMBIO_PASSWORD'; 3. Autenticar qa_bloqueo con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_bloqueo","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Enviar POST http://localhost:15141/api/auth/cambiar-password con Content-Type: application/json, Authorization: Bearer <token>, User-Agent: E2-QA/2.0 y cuerpo {"passwordActual": "E2Qa2026*", "passwordNueva": "CambioEvento9*"}. 5. Verificar en la respuesta: HTTP 200. 6. En psql-QA ejecutar: SELECT count(*), bool_and(ip_address IS NOT NULL), max(user_agent), bool_and(exitoso) FROM seguridad.eventos_seguridad WHERE evento='CAMBIO_PASSWORD' AND created_at>=<inicio>; Comprobar: el conteo de seguridad.eventos_seguridad para evento=CAMBIO_PASSWORD aumenta exactamente 1; exitoso=true; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción. 7. Restaurar: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo';
Datos de entrada	{"perfil": "qa_bloqueo", "passwordActual": "E2Qa2026*", "passwordNueva": "CambioEvento9*"}
Resultado esperado	HTTP 200; el conteo de seguridad.eventos_seguridad para evento=CAMBIO_PASSWORD aumenta exactamente 1; exitoso=true; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción.
Base / bloqueo	D-09: inventario técnico de 20 eventos reconciliado con EventosSeguridad.cs.

TC-REQ07-006 · Evento CAMBIO_PASSWORD_FALLIDO

Trazabilidad	REQ-07 → CP-REQ07-04 → TC-REQ07-006 Dato: DT-TC-REQ07-006
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	qa_bloqueo restaurado con token. Medir conteo después de la preparación, justo antes de la acción objetivo; usar header User-Agent: E2-QA/2.0.
Interfaz	POST http://localhost:15141/api/auth/cambiar-password
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; También UPDATE seguridad.usuarios SET intentos_fallidos=0 WHERE username='qa_externo'; 2. En psql-QA registrar el instante de inicio: SELECT now(); Conteo previo: SELECT count(*) FROM seguridad.eventos_seguridad WHERE evento='CAMBIO_PASSWORD_FALLIDO'; 3. Autenticar qa_bloqueo con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_bloqueo","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Enviar POST http://localhost:15141/api/auth/cambiar-password con Content-Type: application/json, Authorization: Bearer <token>, User-Agent: E2-QA/2.0 y cuerpo {"passwordActual": "E2Qa2026*", "passwordNueva": "corta"}. 5. Verificar en la respuesta: HTTP 400. 6. En psql-QA ejecutar: SELECT count(*), bool_and(ip_address IS NOT NULL), max(user_agent), bool_and(exitoso) FROM seguridad.eventos_seguridad WHERE evento='CAMBIO_PASSWORD_FALLIDO' AND created_at>=<inicio>; Comprobar: el conteo de seguridad.eventos_seguridad para evento=CAMBIO_PASSWORD_FALLIDO aumenta exactamente 1; exitoso=false; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción. 7. Restaurar: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo';
Datos de entrada	{"perfil": "qa_bloqueo", "passwordActual": "E2Qa2026*", "passwordNueva": "corta"}
Resultado esperado	HTTP 400; el conteo de seguridad.eventos_seguridad para evento=CAMBIO_PASSWORD_FALLIDO aumenta exactamente 1; exitoso=false; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción.
Base / bloqueo	D-09: inventario técnico de 20 eventos reconciliado con EventosSeguridad.cs.

TC-REQ07-007 · Evento RESET_PASSWORD

Trazabilidad	REQ-07 → CP-REQ07-04 → TC-REQ07-007 Dato: DT-TC-REQ07-007
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	JWT qa_admin con permiso seguridad.usuarios.reset_password; usuario objetivo qa_bloqueo. Medir conteo después de la preparación, justo antes de la acción objetivo; usar header User-Agent: E2-QA/2.0.
Interfaz	POST http://localhost:15141/api/seguridad/usuarios/4d00b5fc-9b29-5274-bf79-4d712ff0c72b/reset-password
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; También UPDATE seguridad.usuarios SET intentos_fallidos=0 WHERE username='qa_externo'; 2. En psql-QA registrar el instante de inicio: SELECT now(); Conteo previo: SELECT count(*) FROM seguridad.eventos_seguridad WHERE evento='RESET_PASSWORD'; 3. Autenticar qa_admin con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_admin","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Enviar POST http://localhost:15141/api/seguridad/usuarios/4d00b5fc-9b29-5274-bf79-4d712ff0c72b/reset-password con Content-Type: application/json, Authorization: Bearer <token>, User-Agent: E2-QA/2.0 y cuerpo {"passwordNueva": "ResetEvento9*", "debeCambiarPassword": true}. 5. Verificar en la respuesta: HTTP 200. 6. En psql-QA ejecutar: SELECT count(*), bool_and(ip_address IS NOT NULL), max(user_agent), bool_and(exitoso) FROM seguridad.eventos_seguridad WHERE evento='RESET_PASSWORD' AND created_at>=<inicio>; Comprobar: el conteo de seguridad.eventos_seguridad para evento=RESET_PASSWORD aumenta exactamente 1; exitoso=true; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción. 7. Restaurar: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo';
Datos de entrada	{"perfil": "qa_admin", "passwordNueva": "ResetEvento9*", "debeCambiarPassword": true}
Resultado esperado	HTTP 200; el conteo de seguridad.eventos_seguridad para evento=RESET_PASSWORD aumenta exactamente 1; exitoso=true; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción.
Base / bloqueo	D-09: inventario técnico de 20 eventos reconciliado con EventosSeguridad.cs.

TC-REQ07-008 · Evento SOLICITUD_RECUPERACION_PASSWORD

Trazabilidad	REQ-07 → CP-REQ07-04 → TC-REQ07-008 Dato: DT-TC-REQ07-008
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	qa_bloqueo activo restaurado; sin token en solicitud. Medir conteo después de la preparación, justo antes de la acción objetivo; usar header User-Agent: E2-QA/2.0.
Interfaz	POST http://localhost:15141/api/auth/solicitar-recuperacion-password
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; También UPDATE seguridad.usuarios SET intentos_fallidos=0 WHERE username='qa_externo'; 2. En psql-QA registrar el instante de inicio: SELECT now(); Conteo previo: SELECT count(*) FROM seguridad.eventos_seguridad WHERE evento='SOLICITUD_RECUPERACION_PASSWORD'; 3. Enviar POST http://localhost:15141/api/auth/solicitar-recuperacion-password con Content-Type: application/json, sin encabezado Authorization, User-Agent: E2-QA/2.0 y cuerpo {"identificador": "qa_bloqueo@example.invalid"}. 4. Verificar en la respuesta: HTTP 200. 5. En psql-QA ejecutar: SELECT count(*), bool_and(ip_address IS NOT NULL), max(user_agent), bool_and(exitoso) FROM seguridad.eventos_seguridad WHERE evento='SOLICITUD_RECUPERACION_PASSWORD' AND created_at>=<inicio>; Comprobar: el conteo de seguridad.eventos_seguridad para evento=SOLICITUD_RECUPERACION_PASSWORD aumenta exactamente 1; exitoso=true; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción. 6. Restaurar: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo';
Datos de entrada	{"identificador": "qa_bloqueo@example.invalid"}
Resultado esperado	HTTP 200; el conteo de seguridad.eventos_seguridad para evento=SOLICITUD_RECUPERACION_PASSWORD aumenta exactamente 1; exitoso=true; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción.
Base / bloqueo	D-09: inventario técnico de 20 eventos reconciliado con EventosSeguridad.cs.

TC-REQ07-009 · Evento USUARIO_CREADO

Trazabilidad	REQ-07 → CP-REQ07-04 → TC-REQ07-009 Dato: DT-TC-REQ07-009
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	Username/correo de entrada inexistentes; admin con permiso crear. Medir conteo después de la preparación, justo antes de la acción objetivo; usar header User-Agent: E2-QA/2.0.
Interfaz	POST http://localhost:15141/api/seguridad/usuarios
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; También UPDATE seguridad.usuarios SET intentos_fallidos=0 WHERE username='qa_externo'; 2. En psql-QA confirmar que no existe: SELECT count(*) FROM seguridad.usuarios WHERE username='qa_evento_nuevo'; esperar 0. 3. En psql-QA registrar el instante de inicio: SELECT now(); Conteo previo: SELECT count(*) FROM seguridad.eventos_seguridad WHERE evento='USUARIO_CREADO'; 4. Autenticar qa_admin con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_admin","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 5. Enviar POST http://localhost:15141/api/seguridad/usuarios con Content-Type: application/json, Authorization: Bearer <token>, User-Agent: E2-QA/2.0 y cuerpo {"username": "qa_evento_nuevo", "correo": "qa_evento_nuevo@example.invalid", "nombres": "Evento", "apellidos": "Sintetico", "password": "EventoNuevo9*", "telefono": null, "fotografiaUrl": null, "rolesIds": []}. 6. Verificar en la respuesta: HTTP 201. 7. En psql-QA ejecutar: SELECT count(*), bool_and(ip_address IS NOT NULL), max(user_agent), bool_and(exitoso) FROM seguridad.eventos_seguridad WHERE evento='USUARIO_CREADO' AND created_at>=<inicio>; Comprobar: el conteo de seguridad.eventos_seguridad para evento=USUARIO_CREADO aumenta exactamente 1; exitoso=true; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al UUID del nuevo usuario creado.
Datos de entrada	{"perfil": "qa_admin", "username": "qa_evento_nuevo", "correo": "qa_evento_nuevo@example.invalid", "nombres": "Evento", "apellidos": "Sintetico", "password": "EventoNuevo9*", "telefono": null, "fotografiaUrl": null, "rolesIds": []}
Resultado esperado	HTTP 201; el conteo de seguridad.eventos_seguridad para evento=USUARIO_CREADO aumenta exactamente 1; exitoso=true; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al UUID del nuevo usuario creado.
Base / bloqueo	D-09: inventario técnico de 20 eventos reconciliado con EventosSeguridad.cs.

TC-REQ07-010 · Evento USUARIO_EDITADO

Trazabilidad	REQ-07 → CP-REQ07-04 → TC-REQ07-010 Dato: DT-TC-REQ07-010
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	qa_bloqueo existente; admin con permiso editar. Medir conteo después de la preparación, justo antes de la acción objetivo; usar header User-Agent: E2-QA/2.0.
Interfaz	PUT http://localhost:15141/api/seguridad/usuarios/4d00b5fc-9b29-5274-bf79-4d712ff0c72b
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; También UPDATE seguridad.usuarios SET intentos_fallidos=0 WHERE username='qa_externo'; 2. En psql-QA registrar el instante de inicio: SELECT now(); Conteo previo: SELECT count(*) FROM seguridad.eventos_seguridad WHERE evento='USUARIO_EDITADO'; 3. Autenticar qa_admin con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_admin","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Enviar PUT http://localhost:15141/api/seguridad/usuarios/4d00b5fc-9b29-5274-bf79-4d712ff0c72b con Content-Type: application/json, Authorization: Bearer <token>, User-Agent: E2-QA/2.0 y cuerpo {"username": "qa_bloqueo", "correo": "qa_bloqueo@example.invalid", "nombres": "Evento", "apellidos": "Editado", "telefono": null, "fotografiaUrl": null, "clienteId": null, "empleadoId": null}. 5. Verificar en la respuesta: HTTP 200. 6. En psql-QA ejecutar: SELECT count(*), bool_and(ip_address IS NOT NULL), max(user_agent), bool_and(exitoso) FROM seguridad.eventos_seguridad WHERE evento='USUARIO_EDITADO' AND created_at>=<inicio>; Comprobar: el conteo de seguridad.eventos_seguridad para evento=USUARIO_EDITADO aumenta exactamente 1; exitoso=true; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción. 7. Restaurar: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo';
Datos de entrada	{"perfil": "qa_admin", "username": "qa_bloqueo", "correo": "qa_bloqueo@example.invalid", "nombres": "Evento", "apellidos": "Editado", "telefono": null, "fotografiaUrl": null, "clienteId": null, "empleadoId": null}
Resultado esperado	HTTP 200; el conteo de seguridad.eventos_seguridad para evento=USUARIO_EDITADO aumenta exactamente 1; exitoso=true; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción.
Base / bloqueo	D-09: inventario técnico de 20 eventos reconciliado con EventosSeguridad.cs.

TC-REQ07-011 · Evento USUARIO_BLOQUEADO

Trazabilidad	REQ-07 → CP-REQ07-04 → TC-REQ07-011 Dato: DT-TC-REQ07-011
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	qa_bloqueo desbloqueado; admin con permiso bloquear. Medir conteo después de la preparación, justo antes de la acción objetivo; usar header User-Agent: E2-QA/2.0.
Interfaz	PATCH http://localhost:15141/api/seguridad/usuarios/4d00b5fc-9b29-5274-bf79-4d712ff0c72b/bloqueo
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; También UPDATE seguridad.usuarios SET intentos_fallidos=0 WHERE username='qa_externo'; 2. En psql-QA registrar el instante de inicio: SELECT now(); Conteo previo: SELECT count(*) FROM seguridad.eventos_seguridad WHERE evento='USUARIO_BLOQUEADO'; 3. Autenticar qa_admin con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_admin","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Enviar PATCH http://localhost:15141/api/seguridad/usuarios/4d00b5fc-9b29-5274-bf79-4d712ff0c72b/bloqueo con Content-Type: application/json, Authorization: Bearer <token>, User-Agent: E2-QA/2.0 y cuerpo {"bloqueado": true}. 5. Verificar en la respuesta: HTTP 200. 6. En psql-QA ejecutar: SELECT count(*), bool_and(ip_address IS NOT NULL), max(user_agent), bool_and(exitoso) FROM seguridad.eventos_seguridad WHERE evento='USUARIO_BLOQUEADO' AND created_at>=<inicio>; Comprobar: el conteo de seguridad.eventos_seguridad para evento=USUARIO_BLOQUEADO aumenta exactamente 1; exitoso=true; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción. 7. Restaurar: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo';
Datos de entrada	{"perfil": "qa_admin", "bloqueado": true}
Resultado esperado	HTTP 200; el conteo de seguridad.eventos_seguridad para evento=USUARIO_BLOQUEADO aumenta exactamente 1; exitoso=true; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción.
Base / bloqueo	D-09: inventario técnico de 20 eventos reconciliado con EventosSeguridad.cs.

TC-REQ07-012 · Evento USUARIO_DESBLOQUEADO

Trazabilidad	REQ-07 → CP-REQ07-04 → TC-REQ07-012 Dato: DT-TC-REQ07-012
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	qa_bloqueo bloqueado; admin con permiso bloquear. Medir conteo después de la preparación, justo antes de la acción objetivo; usar header User-Agent: E2-QA/2.0.
Interfaz	PATCH http://localhost:15141/api/seguridad/usuarios/4d00b5fc-9b29-5274-bf79-4d712ff0c72b/bloqueo
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; También UPDATE seguridad.usuarios SET intentos_fallidos=0 WHERE username='qa_externo'; 2. En psql-QA: UPDATE seguridad.usuarios SET bloqueado=true WHERE username='qa_bloqueo'; 3. En psql-QA registrar el instante de inicio: SELECT now(); Conteo previo: SELECT count(*) FROM seguridad.eventos_seguridad WHERE evento='USUARIO_DESBLOQUEADO'; 4. Autenticar qa_admin con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_admin","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 5. Enviar PATCH http://localhost:15141/api/seguridad/usuarios/4d00b5fc-9b29-5274-bf79-4d712ff0c72b/bloqueo con Content-Type: application/json, Authorization: Bearer <token>, User-Agent: E2-QA/2.0 y cuerpo {"bloqueado": false}. 6. Verificar en la respuesta: HTTP 200. 7. En psql-QA ejecutar: SELECT count(*), bool_and(ip_address IS NOT NULL), max(user_agent), bool_and(exitoso) FROM seguridad.eventos_seguridad WHERE evento='USUARIO_DESBLOQUEADO' AND created_at>=<inicio>; Comprobar: el conteo de seguridad.eventos_seguridad para evento=USUARIO_DESBLOQUEADO aumenta exactamente 1; exitoso=true; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción. 8. Restaurar: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo';
Datos de entrada	{"perfil": "qa_admin", "bloqueado": false}
Resultado esperado	HTTP 200; el conteo de seguridad.eventos_seguridad para evento=USUARIO_DESBLOQUEADO aumenta exactamente 1; exitoso=true; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción.
Base / bloqueo	D-09: inventario técnico de 20 eventos reconciliado con EventosSeguridad.cs.

TC-REQ07-013 · Evento ASIGNACION_ROL

Trazabilidad	REQ-07 → CP-REQ07-04 → TC-REQ07-013 Dato: DT-TC-REQ07-013
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	Resolver rolCajeroId mediante SELECT id FROM seguridad.roles WHERE codigo=CAJERO; admin con permiso asignar roles. Medir conteo después de la preparación, justo antes de la acción objetivo; usar header User-Agent: E2-QA/2.0.
Interfaz	PUT http://localhost:15141/api/seguridad/usuarios/4d00b5fc-9b29-5274-bf79-4d712ff0c72b/roles
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; También UPDATE seguridad.usuarios SET intentos_fallidos=0 WHERE username='qa_externo'; 2. Resolver el UUID: SELECT id FROM seguridad.roles WHERE codigo='CAJERO'; o SELECT id FROM seguridad.permisos WHERE codigo='transacciones.depositos.crear'; y asignarlo a la variable indicada en el cuerpo. 3. En psql-QA registrar el instante de inicio: SELECT now(); Conteo previo: SELECT count(*) FROM seguridad.eventos_seguridad WHERE evento='ASIGNACION_ROL'; 4. Autenticar qa_admin con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_admin","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 5. Enviar PUT http://localhost:15141/api/seguridad/usuarios/4d00b5fc-9b29-5274-bf79-4d712ff0c72b/roles con Content-Type: application/json, Authorization: Bearer <token>, User-Agent: E2-QA/2.0 y cuerpo {"rolesIds": ["{{rolCajeroId}}"]}. 6. Verificar en la respuesta: HTTP 200. 7. En psql-QA ejecutar: SELECT count(*), bool_and(ip_address IS NOT NULL), max(user_agent), bool_and(exitoso) FROM seguridad.eventos_seguridad WHERE evento='ASIGNACION_ROL' AND created_at>=<inicio>; Comprobar: el conteo de seguridad.eventos_seguridad para evento=ASIGNACION_ROL aumenta exactamente 1; exitoso=true; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción. 8. Restaurar: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo';
Datos de entrada	{"perfil": "qa_admin", "rolesIds": ["{{rolCajeroId}}"]}
Resultado esperado	HTTP 200; el conteo de seguridad.eventos_seguridad para evento=ASIGNACION_ROL aumenta exactamente 1; exitoso=true; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción.
Base / bloqueo	D-09: inventario técnico de 20 eventos reconciliado con EventosSeguridad.cs.

TC-REQ07-014 · Evento ASIGNACION_PERMISO

Trazabilidad	REQ-07 → CP-REQ07-04 → TC-REQ07-014 Dato: DT-TC-REQ07-014
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	Resolver permisoDepositoId de seguridad.permisos codigo=transacciones.depositos.crear; admin con permiso asignar. Medir conteo después de la preparación, justo antes de la acción objetivo; usar header User-Agent: E2-QA/2.0.
Interfaz	PUT http://localhost:15141/api/seguridad/usuarios/4d00b5fc-9b29-5274-bf79-4d712ff0c72b/permisos
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; También UPDATE seguridad.usuarios SET intentos_fallidos=0 WHERE username='qa_externo'; 2. Resolver el UUID: SELECT id FROM seguridad.roles WHERE codigo='CAJERO'; o SELECT id FROM seguridad.permisos WHERE codigo='transacciones.depositos.crear'; y asignarlo a la variable indicada en el cuerpo. 3. En psql-QA registrar el instante de inicio: SELECT now(); Conteo previo: SELECT count(*) FROM seguridad.eventos_seguridad WHERE evento='ASIGNACION_PERMISO'; 4. Autenticar qa_admin con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_admin","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 5. Enviar PUT http://localhost:15141/api/seguridad/usuarios/4d00b5fc-9b29-5274-bf79-4d712ff0c72b/permisos con Content-Type: application/json, Authorization: Bearer <token>, User-Agent: E2-QA/2.0 y cuerpo {"permisosIds": ["{{permisoDepositoId}}"], "permisosDenegadosIds": []}. 6. Verificar en la respuesta: HTTP 200. 7. En psql-QA ejecutar: SELECT count(*), bool_and(ip_address IS NOT NULL), max(user_agent), bool_and(exitoso) FROM seguridad.eventos_seguridad WHERE evento='ASIGNACION_PERMISO' AND created_at>=<inicio>; Comprobar: el conteo de seguridad.eventos_seguridad para evento=ASIGNACION_PERMISO aumenta exactamente 1; exitoso=true; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción. 8. Restaurar: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo';
Datos de entrada	{"perfil": "qa_admin", "permisosIds": ["{{permisoDepositoId}}"], "permisosDenegadosIds": []}
Resultado esperado	HTTP 200; el conteo de seguridad.eventos_seguridad para evento=ASIGNACION_PERMISO aumenta exactamente 1; exitoso=true; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción.
Base / bloqueo	D-09: inventario técnico de 20 eventos reconciliado con EventosSeguridad.cs.

TC-REQ07-015 · Evento ACH_PUBLIC_LOGIN_OK

Trazabilidad	REQ-07 → CP-REQ07-04 → TC-REQ07-015 Dato: DT-TC-REQ07-015
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	qa_externo activo y desbloqueado con rol/permisos. Medir conteo después de la preparación, justo antes de la acción objetivo; usar header User-Agent: E2-QA/2.0.
Interfaz	POST http://localhost:15254/api/external/auth/login
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; También UPDATE seguridad.usuarios SET intentos_fallidos=0 WHERE username='qa_externo'; 2. En psql-QA registrar el instante de inicio: SELECT now(); Conteo previo: SELECT count(*) FROM seguridad.eventos_seguridad WHERE evento='ACH_PUBLIC_LOGIN_OK'; 3. Enviar POST http://localhost:15254/api/external/auth/login con Content-Type: application/json, sin encabezado Authorization, User-Agent: E2-QA/2.0 y cuerpo {"email": "qa_externo@example.invalid", "password": "E2Qa2026*"}. 4. Verificar en la respuesta: HTTP 200. 5. En psql-QA ejecutar: SELECT count(*), bool_and(ip_address IS NOT NULL), max(user_agent), bool_and(exitoso) FROM seguridad.eventos_seguridad WHERE evento='ACH_PUBLIC_LOGIN_OK' AND created_at>=<inicio>; Comprobar: el conteo de seguridad.eventos_seguridad para evento=ACH_PUBLIC_LOGIN_OK aumenta exactamente 1; exitoso=true; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción.
Datos de entrada	{"email": "qa_externo@example.invalid", "password": "E2Qa2026*"}
Resultado esperado	HTTP 200; el conteo de seguridad.eventos_seguridad para evento=ACH_PUBLIC_LOGIN_OK aumenta exactamente 1; exitoso=true; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción.
Base / bloqueo	D-09: inventario técnico de 20 eventos reconciliado con EventosSeguridad.cs.

TC-REQ07-016 · Evento ACH_PUBLIC_LOGIN_FAIL

Trazabilidad	REQ-07 → CP-REQ07-04 → TC-REQ07-016 Dato: DT-TC-REQ07-016
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	qa_externo restaurado con contador=0. Medir conteo después de la preparación, justo antes de la acción objetivo; usar header User-Agent: E2-QA/2.0.
Interfaz	POST http://localhost:15254/api/external/auth/login
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; También UPDATE seguridad.usuarios SET intentos_fallidos=0 WHERE username='qa_externo'; 2. En psql-QA registrar el instante de inicio: SELECT now(); Conteo previo: SELECT count(*) FROM seguridad.eventos_seguridad WHERE evento='ACH_PUBLIC_LOGIN_FAIL'; 3. Enviar POST http://localhost:15254/api/external/auth/login con Content-Type: application/json, sin encabezado Authorization, User-Agent: E2-QA/2.0 y cuerpo {"email": "qa_externo@example.invalid", "password": "Incorrecta9*"}. 4. Verificar en la respuesta: HTTP 401. 5. En psql-QA ejecutar: SELECT count(*), bool_and(ip_address IS NOT NULL), max(user_agent), bool_and(exitoso) FROM seguridad.eventos_seguridad WHERE evento='ACH_PUBLIC_LOGIN_FAIL' AND created_at>=<inicio>; Comprobar: el conteo de seguridad.eventos_seguridad para evento=ACH_PUBLIC_LOGIN_FAIL aumenta exactamente 1; exitoso=false; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción.
Datos de entrada	{"email": "qa_externo@example.invalid", "password": "Incorrecta9*"}
Resultado esperado	HTTP 401; el conteo de seguridad.eventos_seguridad para evento=ACH_PUBLIC_LOGIN_FAIL aumenta exactamente 1; exitoso=false; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción.
Base / bloqueo	D-09: inventario técnico de 20 eventos reconciliado con EventosSeguridad.cs.

TC-REQ07-017 · Evento ACH_PUBLIC_CTA_LOOKUP

Trazabilidad	REQ-07 → CP-REQ07-04 → TC-REQ07-017 Dato: DT-TC-REQ07-017
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	Token externo; cuenta destino ACTIVA. Medir conteo después de la preparación, justo antes de la acción objetivo; usar header User-Agent: E2-QA/2.0.
Interfaz	GET http://localhost:15254/api/external/ach/cuentas/search/AH000000000001
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; También UPDATE seguridad.usuarios SET intentos_fallidos=0 WHERE username='qa_externo'; 2. En psql-QA registrar el instante de inicio: SELECT now(); Conteo previo: SELECT count(*) FROM seguridad.eventos_seguridad WHERE evento='ACH_PUBLIC_CTA_LOOKUP'; 3. Autenticar qa_externo con POST http://localhost:15254/api/external/auth/login y cuerpo {"email":"qa_externo@example.invalid","password":"E2Qa2026*"}; verificar HTTP 200 y guardar accessToken como <token>. 4. Enviar GET http://localhost:15254/api/external/ach/cuentas/search/AH000000000001 con Authorization: Bearer <token>, User-Agent: E2-QA/2.0. 5. Verificar en la respuesta: HTTP 200. 6. En psql-QA ejecutar: SELECT count(*), bool_and(ip_address IS NOT NULL), max(user_agent), bool_and(exitoso) FROM seguridad.eventos_seguridad WHERE evento='ACH_PUBLIC_CTA_LOOKUP' AND created_at>=<inicio>; Comprobar: el conteo de seguridad.eventos_seguridad para evento=ACH_PUBLIC_CTA_LOOKUP aumenta exactamente 1; exitoso=true; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción.
Datos de entrada	{"perfil": "qa_externo"}
Resultado esperado	HTTP 200; el conteo de seguridad.eventos_seguridad para evento=ACH_PUBLIC_CTA_LOOKUP aumenta exactamente 1; exitoso=true; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción.
Base / bloqueo	D-09: inventario técnico de 20 eventos reconciliado con EventosSeguridad.cs.

TC-REQ07-018 · Evento ACH_PUBLIC_TRX_IN_OK

Trazabilidad	REQ-07 → CP-REQ07-04 → TC-REQ07-018 Dato: DT-TC-REQ07-018
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	Código nuevo; saldo destino restaurado a 1000. Medir conteo después de la preparación, justo antes de la acción objetivo; usar header User-Agent: E2-QA/2.0.
Interfaz	POST http://localhost:15254/api/external/ach/transferencias
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; También UPDATE seguridad.usuarios SET intentos_fallidos=0 WHERE username='qa_externo'; 2. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 3. En psql-QA registrar el instante de inicio: SELECT now(); Conteo previo: SELECT count(*) FROM seguridad.eventos_seguridad WHERE evento='ACH_PUBLIC_TRX_IN_OK'; 4. Autenticar qa_externo con POST http://localhost:15254/api/external/auth/login y cuerpo {"email":"qa_externo@example.invalid","password":"E2Qa2026*"}; verificar HTTP 200 y guardar accessToken como <token>. 5. Enviar POST http://localhost:15254/api/external/ach/transferencias con Content-Type: application/json, Authorization: Bearer <token>, User-Agent: E2-QA/2.0 y cuerpo {"codigo_confirmacion": "E2-EV-ACH_PUBLIC_TRX_IN_OK", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "1.00", "moneda": "Q", "tipo": "entrante"} (en Postman se agrega el sufijo -{{runId}} al código). 6. Verificar en la respuesta: HTTP 201. 7. En psql-QA ejecutar: SELECT count(*), bool_and(ip_address IS NOT NULL), max(user_agent), bool_and(exitoso) FROM seguridad.eventos_seguridad WHERE evento='ACH_PUBLIC_TRX_IN_OK' AND created_at>=<inicio>; Comprobar: el conteo de seguridad.eventos_seguridad para evento=ACH_PUBLIC_TRX_IN_OK aumenta exactamente 1; exitoso=true; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción. 8. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_externo", "codigo_confirmacion": "E2-EV-ACH_PUBLIC_TRX_IN_OK", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "1.00", "moneda": "Q", "tipo": "entrante"}
Resultado esperado	HTTP 201; el conteo de seguridad.eventos_seguridad para evento=ACH_PUBLIC_TRX_IN_OK aumenta exactamente 1; exitoso=true; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción.
Base / bloqueo	D-09: inventario técnico de 20 eventos reconciliado con EventosSeguridad.cs.

TC-REQ07-019 · Evento ACH_PUBLIC_TRX_IN_REJ

Trazabilidad	REQ-07 → CP-REQ07-04 → TC-REQ07-019 Dato: DT-TC-REQ07-019
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	Código nuevo y token externo; monto cero. Medir conteo después de la preparación, justo antes de la acción objetivo; usar header User-Agent: E2-QA/2.0.
Interfaz	POST http://localhost:15254/api/external/ach/transferencias
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; También UPDATE seguridad.usuarios SET intentos_fallidos=0 WHERE username='qa_externo'; 2. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 3. En psql-QA registrar el instante de inicio: SELECT now(); Conteo previo: SELECT count(*) FROM seguridad.eventos_seguridad WHERE evento='ACH_PUBLIC_TRX_IN_REJ'; 4. Autenticar qa_externo con POST http://localhost:15254/api/external/auth/login y cuerpo {"email":"qa_externo@example.invalid","password":"E2Qa2026*"}; verificar HTTP 200 y guardar accessToken como <token>. 5. Enviar POST http://localhost:15254/api/external/ach/transferencias con Content-Type: application/json, Authorization: Bearer <token>, User-Agent: E2-QA/2.0 y cuerpo {"codigo_confirmacion": "E2-EV-ACH_PUBLIC_TRX_IN_REJ", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "0.00", "moneda": "Q", "tipo": "entrante"} (en Postman se agrega el sufijo -{{runId}} al código). 6. Verificar en la respuesta: HTTP 400. 7. En psql-QA ejecutar: SELECT count(*), bool_and(ip_address IS NOT NULL), max(user_agent), bool_and(exitoso) FROM seguridad.eventos_seguridad WHERE evento='ACH_PUBLIC_TRX_IN_REJ' AND created_at>=<inicio>; Comprobar: el conteo de seguridad.eventos_seguridad para evento=ACH_PUBLIC_TRX_IN_REJ aumenta exactamente 1; exitoso=false; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción. 8. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_externo", "codigo_confirmacion": "E2-EV-ACH_PUBLIC_TRX_IN_REJ", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "0.00", "moneda": "Q", "tipo": "entrante"}
Resultado esperado	HTTP 400; el conteo de seguridad.eventos_seguridad para evento=ACH_PUBLIC_TRX_IN_REJ aumenta exactamente 1; exitoso=false; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción.
Base / bloqueo	D-09: inventario técnico de 20 eventos reconciliado con EventosSeguridad.cs.

TC-REQ07-020 · Evento ACH_PUBLIC_TRX_IN_DUP

Trazabilidad	REQ-07 → CP-REQ07-04 → TC-REQ07-020 Dato: DT-TC-REQ07-020
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	Enviar previamente una operación válida de 1 con el mismo código; capturar conteo del evento DUP tras la primera aceptación. Medir conteo después de la preparación, justo antes de la acción objetivo; usar header User-Agent: E2-QA/2.0.
Interfaz	POST http://localhost:15254/api/external/ach/transferencias
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; También UPDATE seguridad.usuarios SET intentos_fallidos=0 WHERE username='qa_externo'; 2. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 3. Enviar una primera vez POST http://localhost:15254/api/external/ach/transferencias con el mismo cuerpo del paso de ejecución y verificar HTTP 201 (crea el código). 4. En psql-QA registrar el instante de inicio: SELECT now(); Conteo previo: SELECT count(*) FROM seguridad.eventos_seguridad WHERE evento='ACH_PUBLIC_TRX_IN_DUP'; 5. Autenticar qa_externo con POST http://localhost:15254/api/external/auth/login y cuerpo {"email":"qa_externo@example.invalid","password":"E2Qa2026*"}; verificar HTTP 200 y guardar accessToken como <token>. 6. Enviar POST http://localhost:15254/api/external/ach/transferencias con Content-Type: application/json, Authorization: Bearer <token>, User-Agent: E2-QA/2.0 y cuerpo {"codigo_confirmacion": "E2-EV-ACH_PUBLIC_TRX_IN_DUP", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "1.00", "moneda": "Q", "tipo": "entrante"} (en Postman se agrega el sufijo -{{runId}} al código). 7. Verificar en la respuesta: HTTP 409. 8. En psql-QA ejecutar: SELECT count(*), bool_and(ip_address IS NOT NULL), max(user_agent), bool_and(exitoso) FROM seguridad.eventos_seguridad WHERE evento='ACH_PUBLIC_TRX_IN_DUP' AND created_at>=<inicio>; Comprobar: el conteo de seguridad.eventos_seguridad para evento=ACH_PUBLIC_TRX_IN_DUP aumenta exactamente 1; exitoso=false; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción. 9. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_externo", "codigo_confirmacion": "E2-EV-ACH_PUBLIC_TRX_IN_DUP", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "1.00", "moneda": "Q", "tipo": "entrante"}
Resultado esperado	HTTP 409; el conteo de seguridad.eventos_seguridad para evento=ACH_PUBLIC_TRX_IN_DUP aumenta exactamente 1; exitoso=false; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción.
Base / bloqueo	D-09: inventario técnico de 20 eventos reconciliado con EventosSeguridad.cs.

TC-REQ07-021 · Evento ACH_PUBLIC_TRX_IN_ERR

Trazabilidad	REQ-07 → CP-REQ07-04 → TC-REQ07-021 Dato: DT-TC-REQ07-021
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	Solo QA: fijar saldo=saldo_disponible=9999999999999999.99 de cuenta destino; nuevo código y monto=1 provocan overflow; restaurar saldo a 1000 al terminar. Medir conteo después de la preparación, justo antes de la acción objetivo; usar header User-Agent: E2-QA/2.0.
Interfaz	POST http://localhost:15254/api/external/ach/transferencias
Pasos	1. En psql-QA restaurar el usuario: UPDATE seguridad.usuarios SET estado=true, bloqueado=false, bloqueado_hasta=NULL, intentos_fallidos=0, password_hash=crypt('E2Qa2026*', gen_salt('bf',10)) WHERE username='qa_bloqueo'; También UPDATE seguridad.usuarios SET intentos_fallidos=0 WHERE username='qa_externo'; 2. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 3. En psql-QA (solo QA) forzar el desbordamiento: UPDATE cuentas.cuentas SET saldo=9999999999999999.99, saldo_disponible=9999999999999999.99 WHERE numero_cuenta='AH000000000001'; 4. En psql-QA registrar el instante de inicio: SELECT now(); Conteo previo: SELECT count(*) FROM seguridad.eventos_seguridad WHERE evento='ACH_PUBLIC_TRX_IN_ERR'; 5. Autenticar qa_externo con POST http://localhost:15254/api/external/auth/login y cuerpo {"email":"qa_externo@example.invalid","password":"E2Qa2026*"}; verificar HTTP 200 y guardar accessToken como <token>. 6. Enviar POST http://localhost:15254/api/external/ach/transferencias con Content-Type: application/json, Authorization: Bearer <token>, User-Agent: E2-QA/2.0 y cuerpo {"codigo_confirmacion": "E2-EV-ACH_PUBLIC_TRX_IN_ERR", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "1.00", "moneda": "Q", "tipo": "entrante"} (en Postman se agrega el sufijo -{{runId}} al código). 7. Verificar en la respuesta: HTTP 500. 8. En psql-QA ejecutar: SELECT count(*), bool_and(ip_address IS NOT NULL), max(user_agent), bool_and(exitoso) FROM seguridad.eventos_seguridad WHERE evento='ACH_PUBLIC_TRX_IN_ERR' AND created_at>=<inicio>; Comprobar: el conteo de seguridad.eventos_seguridad para evento=ACH_PUBLIC_TRX_IN_ERR aumenta exactamente 1; exitoso=false; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción. 9. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_externo", "codigo_confirmacion": "E2-EV-ACH_PUBLIC_TRX_IN_ERR", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "1.00", "moneda": "Q", "tipo": "entrante"}
Resultado esperado	HTTP 500; el conteo de seguridad.eventos_seguridad para evento=ACH_PUBLIC_TRX_IN_ERR aumenta exactamente 1; exitoso=false; ip_address no vacío; user_agent=E2-QA/2.0; created_at entre inicio y fin. usuario_id corresponde al sujeto sintético de la acción.
Base / bloqueo	D-09: inventario técnico de 20 eventos reconciliado con EventosSeguridad.cs.

REQ-08 · Apertura de sesion de caja con fondo inicial

TC-REQ08-001 · Fondo inicial -0.01

Trazabilidad	REQ-08 → CP-REQ08-01 → TC-REQ08-001 Dato: DT-TC-REQ08-001
Técnica / prioridad	Valores límite · Alta Estado: Diseñado; no ejecutado
Precondiciones	qa_cajero con empleado ACTIVO; ninguna caja abierta hoy; restaurar cada variante.
Interfaz	POST http://localhost:15141/api/cajas/abrir
Pasos	1. En psql-QA cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; 2. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 3. En psql-QA registrar el instante de inicio: SELECT now(); 4. Enviar POST http://localhost:15141/api/cajas/abrir con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"fondoInicial": -0.01}. 5. Verificar en la respuesta: HTTP 422; message="El fondo inicial no puede ser negativo.". 6. En psql-QA ejecutar: SELECT estado, fondo_inicial, total_depositos, total_retiros, efectivo_contado, diferencia, justificacion, hora_cierre FROM cajas.sesiones_caja WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND fecha=CURRENT_DATE ORDER BY hora_apertura DESC; Comprobar: no sesión. 7. Al terminar cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA';
Datos de entrada	{"perfil": "qa_cajero", "fondoInicial": "-0.01"}
Resultado esperado	HTTP 422; message="El fondo inicial no puede ser negativo."; no sesión.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ08-002 · Fondo inicial 0.00

Trazabilidad	REQ-08 → CP-REQ08-01 → TC-REQ08-002 Dato: DT-TC-REQ08-002
Técnica / prioridad	Valores límite · Alta Estado: Diseñado; no ejecutado
Precondiciones	qa_cajero con empleado ACTIVO; ninguna caja abierta hoy; restaurar cada variante.
Interfaz	POST http://localhost:15141/api/cajas/abrir
Pasos	1. En psql-QA cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; 2. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 3. En psql-QA registrar el instante de inicio: SELECT now(); 4. Enviar POST http://localhost:15141/api/cajas/abrir con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"fondoInicial": 0.00}. 5. Verificar en la respuesta: HTTP 200; message="Caja abierta. Puede comenzar a operar."; sesionCajaId UUID. 6. En psql-QA ejecutar: SELECT estado, fondo_inicial, total_depositos, total_retiros, efectivo_contado, diferencia, justificacion, hora_cierre FROM cajas.sesiones_caja WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND fecha=CURRENT_DATE ORDER BY hora_apertura DESC; Comprobar: una caja ABIERTA con fondo_inicial=0.00. 7. Al terminar cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA';
Datos de entrada	{"perfil": "qa_cajero", "fondoInicial": "0.00"}
Resultado esperado	HTTP 200; message="Caja abierta. Puede comenzar a operar."; sesionCajaId UUID; una caja ABIERTA con fondo_inicial=0.00.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ08-003 · Fondo inicial 0.01

Trazabilidad	REQ-08 → CP-REQ08-01 → TC-REQ08-003 Dato: DT-TC-REQ08-003
Técnica / prioridad	Valores límite · Alta Estado: Diseñado; no ejecutado
Precondiciones	qa_cajero con empleado ACTIVO; ninguna caja abierta hoy; restaurar cada variante.
Interfaz	POST http://localhost:15141/api/cajas/abrir
Pasos	1. En psql-QA cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; 2. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 3. En psql-QA registrar el instante de inicio: SELECT now(); 4. Enviar POST http://localhost:15141/api/cajas/abrir con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"fondoInicial": 0.01}. 5. Verificar en la respuesta: HTTP 200; message="Caja abierta. Puede comenzar a operar."; sesionCajaId UUID. 6. En psql-QA ejecutar: SELECT estado, fondo_inicial, total_depositos, total_retiros, efectivo_contado, diferencia, justificacion, hora_cierre FROM cajas.sesiones_caja WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND fecha=CURRENT_DATE ORDER BY hora_apertura DESC; Comprobar: una caja ABIERTA con fondo_inicial=0.01. 7. Al terminar cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA';
Datos de entrada	{"perfil": "qa_cajero", "fondoInicial": "0.01"}
Resultado esperado	HTTP 200; message="Caja abierta. Puede comenzar a operar."; sesionCajaId UUID; una caja ABIERTA con fondo_inicial=0.01.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ08-004 · Operación sin caja abierta

Trazabilidad	REQ-08 → CP-REQ08-02 → TC-REQ08-004 Dato: DT-TC-REQ08-004
Técnica / prioridad	Transición de estados · Alta Estado: Diseñado; no ejecutado
Precondiciones	Empleado QA activo; ninguna caja ABIERTA; token válido.
Interfaz	POST http://localhost:15141/api/transacciones/deposito
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; 3. En psql-QA registrar el instante de inicio: SELECT now(); 4. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 5. Enviar POST http://localhost:15141/api/transacciones/deposito con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"numeroCuenta": "AH000000000001", "monto": 10.00}. 6. Verificar en la respuesta: HTTP 422; message="El usuario autenticado debe tener un empleado activo con caja abierta."; cuenta intacta. 7. En psql-QA ejecutar: SELECT c.numero_cuenta, c.saldo, c.saldo_disponible, c.monto_retirado_hoy, (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.created_at>=<inicio>) AS movs, (SELECT count(*) FROM contabilidad.asientos a JOIN transacciones.movimientos m ON m.id=a.movimiento_id WHERE m.cuenta_id=c.id AND a.created_at>=<inicio>) AS asientos FROM cuentas.cuentas c WHERE c.numero_cuenta IN ('AH000000000001','MO000000000002','AH000000000003') ORDER BY 1; Comprobar que no hay registros nuevos ni cambios respecto al estado del paso 1. 8. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_cajero", "numeroCuenta": "AH000000000001", "monto": "10.00"}
Resultado esperado	HTTP 422; message="El usuario autenticado debe tener un empleado activo con caja abierta."; cuenta intacta.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ08-005 · Segunda apertura de caja

Trazabilidad	REQ-08 → CP-REQ08-02 → TC-REQ08-005 Dato: DT-TC-REQ08-005
Técnica / prioridad	Transición de estados · Alta Estado: Diseñado; no ejecutado
Precondiciones	Una caja QA abierta hoy con fondo Q2000.00.
Interfaz	POST http://localhost:15141/api/cajas/abrir
Pasos	1. En psql-QA cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; 2. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 3. Abrir caja con POST http://localhost:15141/api/cajas/abrir, Bearer de qa_cajero y cuerpo {"fondoInicial": 2000.00}; verificar HTTP 200. 4. En psql-QA registrar el instante de inicio: SELECT now(); 5. Enviar POST http://localhost:15141/api/cajas/abrir con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"fondoInicial": 100.00}. 6. Verificar en la respuesta: HTTP 422; message="Ya tiene una sesion de caja abierta hoy". 7. En psql-QA ejecutar: SELECT estado, fondo_inicial, total_depositos, total_retiros, efectivo_contado, diferencia, justificacion, hora_cierre FROM cajas.sesiones_caja WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND fecha=CURRENT_DATE ORDER BY hora_apertura DESC; Comprobar: continúa una sola caja ABIERTA. 8. Al terminar cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA';
Datos de entrada	{"perfil": "qa_cajero", "fondoInicial": "100.00"}
Resultado esperado	HTTP 422; message="Ya tiene una sesion de caja abierta hoy"; continúa una sola caja ABIERTA.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

REQ-09 · Cierre de caja con cuadre de efectivo

TC-REQ09-001 · Cierre con cuadre exacto

Trazabilidad	REQ-09 → CP-REQ09-01 → TC-REQ09-001 Dato: DT-TC-REQ09-001
Técnica / prioridad	Transición de estados · Alta Estado: Diseñado; no ejecutado
Precondiciones	Caja abierta con fondo 2000; total_depositos=0, total_retiros=0.
Interfaz	POST http://localhost:15141/api/cajas/cerrar
Pasos	1. En psql-QA cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; 2. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 3. Abrir caja con POST http://localhost:15141/api/cajas/abrir, Bearer de qa_cajero y cuerpo {"fondoInicial": 2000.00}; verificar HTTP 200. Confirmar total_depositos=0 y total_retiros=0 (sin operaciones en la caja). 4. Enviar POST http://localhost:15141/api/cajas/cerrar con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"efectivoContado": 2000.00}. 5. Verificar en la respuesta: HTTP 200; diferencia=0, resultado="CUADRE EXACTO", efectivoEsperado=2000. 6. En psql-QA ejecutar: SELECT estado, fondo_inicial, total_depositos, total_retiros, efectivo_contado, diferencia, justificacion, hora_cierre FROM cajas.sesiones_caja WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND fecha=CURRENT_DATE ORDER BY hora_apertura DESC; Comprobar: caja CERRADA y hora_cierre no nula. 7. Al terminar cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA';
Datos de entrada	{"perfil": "qa_cajero", "efectivoContado": "2000.00"}
Resultado esperado	HTTP 200; diferencia=0, resultado="CUADRE EXACTO", efectivoEsperado=2000; caja CERRADA y hora_cierre no nula.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-03. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ09-002 · Descuadre de Q10.01 sin justificación

Trazabilidad	REQ-09 → CP-REQ09-02 → TC-REQ09-002 Dato: DT-TC-REQ09-002
Técnica / prioridad	Valores límite + Tabla de decisión · Alta Estado: Diseñado; no ejecutado
Precondiciones	Caja ABIERTA con fondo=2000, depósitos=0, retiros=0; usuario qa_cajero; sin justificación.
Interfaz	POST http://localhost:15141/api/cajas/cerrar
Pasos	1. En psql-QA cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; 2. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 3. Abrir caja con POST http://localhost:15141/api/cajas/abrir, Bearer de qa_cajero y cuerpo {"fondoInicial": 2000.00}; verificar HTTP 200. Confirmar total_depositos=0 y total_retiros=0 (sin operaciones en la caja). 4. Enviar POST http://localhost:15141/api/cajas/cerrar con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"efectivoContado": 2010.01, "justificacion": null}. 5. Verificar en la respuesta: HTTP 422; message="Diferencia de Q10.01 requiere justificacion.". 6. En psql-QA ejecutar: SELECT estado, fondo_inicial, total_depositos, total_retiros, efectivo_contado, diferencia, justificacion, hora_cierre FROM cajas.sesiones_caja WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND fecha=CURRENT_DATE ORDER BY hora_apertura DESC; Comprobar: la caja permanece ABIERTA, sin hora_cierre. 7. Al terminar cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA';
Datos de entrada	{"perfil": "qa_cajero", "efectivoContado": "2010.01", "justificacion": null}
Resultado esperado	HTTP 422; message="Diferencia de Q10.01 requiere justificacion."; la caja permanece ABIERTA, sin hora_cierre.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-03. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ09-003 · Diferencia 10.00 con justificación ausente

Trazabilidad	REQ-09 → CP-REQ09-02 → TC-REQ09-003 Dato: DT-TC-REQ09-003
Técnica / prioridad	Valores límite + Tabla de decisión · Alta Estado: Diseñado; no ejecutado
Precondiciones	Restaurar caja ABIERTA con fondo=2000, depósitos=0 y retiros=0 antes de cada variante; token de qa_cajero.
Interfaz	POST http://localhost:15141/api/cajas/cerrar
Pasos	1. En psql-QA cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; 2. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 3. Abrir caja con POST http://localhost:15141/api/cajas/abrir, Bearer de qa_cajero y cuerpo {"fondoInicial": 2000.00}; verificar HTTP 200. Confirmar total_depositos=0 y total_retiros=0 (sin operaciones en la caja). 4. Enviar POST http://localhost:15141/api/cajas/cerrar con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"efectivoContado": 2010.00, "justificacion": null}. 5. Verificar en la respuesta: HTTP 200; diferencia=10.00; resultado="SOBRANTE". 6. En psql-QA ejecutar: SELECT estado, fondo_inicial, total_depositos, total_retiros, efectivo_contado, diferencia, justificacion, hora_cierre FROM cajas.sesiones_caja WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND fecha=CURRENT_DATE ORDER BY hora_apertura DESC; Comprobar: caja CERRADA; hora_cierre no nula; justificación persistida conforme a la entrada. 7. Al terminar cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA';
Datos de entrada	{"perfil": "qa_cajero", "efectivoContado": "2010.00", "justificacion": null}
Resultado esperado	HTTP 200; diferencia=10.00; resultado="SOBRANTE"; caja CERRADA; hora_cierre no nula; justificación persistida conforme a la entrada.
Base / bloqueo	Criterios aprobados por el equipo QA el 06-10-2026

TC-REQ09-004 · Diferencia -10.00 con justificación ausente

Trazabilidad	REQ-09 → CP-REQ09-02 → TC-REQ09-004 Dato: DT-TC-REQ09-004
Técnica / prioridad	Valores límite + Tabla de decisión · Alta Estado: Diseñado; no ejecutado
Precondiciones	Restaurar caja ABIERTA con fondo=2000, depósitos=0 y retiros=0 antes de cada variante; token de qa_cajero.
Interfaz	POST http://localhost:15141/api/cajas/cerrar
Pasos	1. En psql-QA cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; 2. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 3. Abrir caja con POST http://localhost:15141/api/cajas/abrir, Bearer de qa_cajero y cuerpo {"fondoInicial": 2000.00}; verificar HTTP 200. Confirmar total_depositos=0 y total_retiros=0 (sin operaciones en la caja). 4. Enviar POST http://localhost:15141/api/cajas/cerrar con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"efectivoContado": 1990.00, "justificacion": null}. 5. Verificar en la respuesta: HTTP 200; diferencia=-10.00; resultado="FALTANTE". 6. En psql-QA ejecutar: SELECT estado, fondo_inicial, total_depositos, total_retiros, efectivo_contado, diferencia, justificacion, hora_cierre FROM cajas.sesiones_caja WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND fecha=CURRENT_DATE ORDER BY hora_apertura DESC; Comprobar: caja CERRADA; hora_cierre no nula; justificación persistida conforme a la entrada. 7. Al terminar cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA';
Datos de entrada	{"perfil": "qa_cajero", "efectivoContado": "1990.00", "justificacion": null}
Resultado esperado	HTTP 200; diferencia=-10.00; resultado="FALTANTE"; caja CERRADA; hora_cierre no nula; justificación persistida conforme a la entrada.
Base / bloqueo	Criterios aprobados por el equipo QA el 06-10-2026

TC-REQ09-005 · Diferencia -10.01 con justificación ausente

Trazabilidad	REQ-09 → CP-REQ09-02 → TC-REQ09-005 Dato: DT-TC-REQ09-005
Técnica / prioridad	Valores límite + Tabla de decisión · Alta Estado: Diseñado; no ejecutado
Precondiciones	Restaurar caja ABIERTA con fondo=2000, depósitos=0 y retiros=0 antes de cada variante; token de qa_cajero.
Interfaz	POST http://localhost:15141/api/cajas/cerrar
Pasos	1. En psql-QA cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; 2. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 3. Abrir caja con POST http://localhost:15141/api/cajas/abrir, Bearer de qa_cajero y cuerpo {"fondoInicial": 2000.00}; verificar HTTP 200. Confirmar total_depositos=0 y total_retiros=0 (sin operaciones en la caja). 4. Enviar POST http://localhost:15141/api/cajas/cerrar con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"efectivoContado": 1989.99, "justificacion": null}. 5. Verificar en la respuesta: HTTP 422; message="Diferencia de Q-10.01 requiere justificacion.". 6. En psql-QA ejecutar: SELECT estado, fondo_inicial, total_depositos, total_retiros, efectivo_contado, diferencia, justificacion, hora_cierre FROM cajas.sesiones_caja WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND fecha=CURRENT_DATE ORDER BY hora_apertura DESC; Comprobar: caja ABIERTA y sin hora_cierre. 7. Al terminar cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA';
Datos de entrada	{"perfil": "qa_cajero", "efectivoContado": "1989.99", "justificacion": null}
Resultado esperado	HTTP 422; message="Diferencia de Q-10.01 requiere justificacion."; caja ABIERTA y sin hora_cierre.
Base / bloqueo	Criterios aprobados por el equipo QA el 06-10-2026

TC-REQ09-006 · Diferencia 10.01 con justificación

Trazabilidad	REQ-09 → CP-REQ09-02 → TC-REQ09-006 Dato: DT-TC-REQ09-006
Técnica / prioridad	Valores límite + Tabla de decisión · Alta Estado: Diseñado; no ejecutado
Precondiciones	Restaurar caja ABIERTA con fondo=2000, depósitos=0 y retiros=0 antes de cada variante; token de qa_cajero.
Interfaz	POST http://localhost:15141/api/cajas/cerrar
Pasos	1. En psql-QA cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; 2. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 3. Abrir caja con POST http://localhost:15141/api/cajas/abrir, Bearer de qa_cajero y cuerpo {"fondoInicial": 2000.00}; verificar HTTP 200. Confirmar total_depositos=0 y total_retiros=0 (sin operaciones en la caja). 4. Enviar POST http://localhost:15141/api/cajas/cerrar con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"efectivoContado": 2010.01, "justificacion": "Descuadre sintetico QA"}. 5. Verificar en la respuesta: HTTP 200; diferencia=10.01; resultado="SOBRANTE". 6. En psql-QA ejecutar: SELECT estado, fondo_inicial, total_depositos, total_retiros, efectivo_contado, diferencia, justificacion, hora_cierre FROM cajas.sesiones_caja WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND fecha=CURRENT_DATE ORDER BY hora_apertura DESC; Comprobar: caja CERRADA; hora_cierre no nula; justificación persistida conforme a la entrada. 7. Al terminar cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA';
Datos de entrada	{"perfil": "qa_cajero", "efectivoContado": "2010.01", "justificacion": "Descuadre sintetico QA"}
Resultado esperado	HTTP 200; diferencia=10.01; resultado="SOBRANTE"; caja CERRADA; hora_cierre no nula; justificación persistida conforme a la entrada.
Base / bloqueo	Criterios aprobados por el equipo QA el 06-10-2026

TC-REQ09-007 · Diferencia -10.01 con justificación

Trazabilidad	REQ-09 → CP-REQ09-02 → TC-REQ09-007 Dato: DT-TC-REQ09-007
Técnica / prioridad	Valores límite + Tabla de decisión · Alta Estado: Diseñado; no ejecutado
Precondiciones	Restaurar caja ABIERTA con fondo=2000, depósitos=0 y retiros=0 antes de cada variante; token de qa_cajero.
Interfaz	POST http://localhost:15141/api/cajas/cerrar
Pasos	1. En psql-QA cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; 2. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 3. Abrir caja con POST http://localhost:15141/api/cajas/abrir, Bearer de qa_cajero y cuerpo {"fondoInicial": 2000.00}; verificar HTTP 200. Confirmar total_depositos=0 y total_retiros=0 (sin operaciones en la caja). 4. Enviar POST http://localhost:15141/api/cajas/cerrar con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"efectivoContado": 1989.99, "justificacion": "Descuadre sintetico QA"}. 5. Verificar en la respuesta: HTTP 200; diferencia=-10.01; resultado="FALTANTE". 6. En psql-QA ejecutar: SELECT estado, fondo_inicial, total_depositos, total_retiros, efectivo_contado, diferencia, justificacion, hora_cierre FROM cajas.sesiones_caja WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND fecha=CURRENT_DATE ORDER BY hora_apertura DESC; Comprobar: caja CERRADA; hora_cierre no nula; justificación persistida conforme a la entrada. 7. Al terminar cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA';
Datos de entrada	{"perfil": "qa_cajero", "efectivoContado": "1989.99", "justificacion": "Descuadre sintetico QA"}
Resultado esperado	HTTP 200; diferencia=-10.01; resultado="FALTANTE"; caja CERRADA; hora_cierre no nula; justificación persistida conforme a la entrada.
Base / bloqueo	Criterios aprobados por el equipo QA el 06-10-2026

REQ-10 · Deposito en ventanilla

TC-REQ10-001 · Depósito: monto -0.01

Trazabilidad	REQ-10 → CP-REQ10-01 → TC-REQ10-001 Dato: DT-TC-REQ10-001
Técnica / prioridad	Valores límite · Alta Estado: Diseñado; no ejecutado
Precondiciones	PC-F: restaurar las cuentas implicadas a los valores de MockData_Sintetica.json; caja del cajero abierta con fondo Q2000.00; usuario qa_cajero con permisos efectivos. Ejecutar cada variante desde esa misma línea base.
Interfaz	POST http://localhost:15141/api/transacciones/deposito
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; 3. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Abrir caja con POST http://localhost:15141/api/cajas/abrir, Bearer de qa_cajero y cuerpo {"fondoInicial": 2000.00}; verificar HTTP 200. 5. En psql-QA registrar el instante de inicio: SELECT now(); 6. Enviar POST http://localhost:15141/api/transacciones/deposito con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"numeroCuenta": "AH000000000001", "monto": -0.01}. 7. Verificar en la respuesta: HTTP 400; message="El monto debe ser mayor a cero.". 8. En psql-QA ejecutar: SELECT c.numero_cuenta, c.saldo, c.saldo_disponible, c.monto_retirado_hoy, (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.created_at>=<inicio>) AS movs, (SELECT count(*) FROM contabilidad.asientos a JOIN transacciones.movimientos m ON m.id=a.movimiento_id WHERE m.cuenta_id=c.id AND a.created_at>=<inicio>) AS asientos FROM cuentas.cuentas c WHERE c.numero_cuenta IN ('AH000000000001','MO000000000002','AH000000000003') ORDER BY 1; Comprobar: delta de saldo, movimientos y asientos=0. 9. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_cajero", "numeroCuenta": "AH000000000001", "monto": "-0.01"}
Resultado esperado	HTTP 400; message="El monto debe ser mayor a cero."; delta de saldo, movimientos y asientos=0.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ10-002 · Depósito: monto 0.00

Trazabilidad	REQ-10 → CP-REQ10-01 → TC-REQ10-002 Dato: DT-TC-REQ10-002
Técnica / prioridad	Valores límite · Alta Estado: Diseñado; no ejecutado
Precondiciones	PC-F: restaurar las cuentas implicadas a los valores de MockData_Sintetica.json; caja del cajero abierta con fondo Q2000.00; usuario qa_cajero con permisos efectivos. Ejecutar cada variante desde esa misma línea base.
Interfaz	POST http://localhost:15141/api/transacciones/deposito
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; 3. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Abrir caja con POST http://localhost:15141/api/cajas/abrir, Bearer de qa_cajero y cuerpo {"fondoInicial": 2000.00}; verificar HTTP 200. 5. En psql-QA registrar el instante de inicio: SELECT now(); 6. Enviar POST http://localhost:15141/api/transacciones/deposito con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"numeroCuenta": "AH000000000001", "monto": 0.00}. 7. Verificar en la respuesta: HTTP 400; message="El monto debe ser mayor a cero.". 8. En psql-QA ejecutar: SELECT c.numero_cuenta, c.saldo, c.saldo_disponible, c.monto_retirado_hoy, (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.created_at>=<inicio>) AS movs, (SELECT count(*) FROM contabilidad.asientos a JOIN transacciones.movimientos m ON m.id=a.movimiento_id WHERE m.cuenta_id=c.id AND a.created_at>=<inicio>) AS asientos FROM cuentas.cuentas c WHERE c.numero_cuenta IN ('AH000000000001','MO000000000002','AH000000000003') ORDER BY 1; Comprobar: delta de saldo, movimientos y asientos=0. 9. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_cajero", "numeroCuenta": "AH000000000001", "monto": "0.00"}
Resultado esperado	HTTP 400; message="El monto debe ser mayor a cero."; delta de saldo, movimientos y asientos=0.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ10-003 · Depósito: monto 0.01

Trazabilidad	REQ-10 → CP-REQ10-01 → TC-REQ10-003 Dato: DT-TC-REQ10-003
Técnica / prioridad	Valores límite · Alta Estado: Diseñado; no ejecutado
Precondiciones	PC-F: restaurar las cuentas implicadas a los valores de MockData_Sintetica.json; caja del cajero abierta con fondo Q2000.00; usuario qa_cajero con permisos efectivos. Ejecutar cada variante desde esa misma línea base.
Interfaz	POST http://localhost:15141/api/transacciones/deposito
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; 3. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Abrir caja con POST http://localhost:15141/api/cajas/abrir, Bearer de qa_cajero y cuerpo {"fondoInicial": 2000.00}; verificar HTTP 200. 5. En psql-QA registrar el instante de inicio: SELECT now(); 6. Enviar POST http://localhost:15141/api/transacciones/deposito con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"numeroCuenta": "AH000000000001", "monto": 0.01}. 7. Verificar en la respuesta: HTTP 200; saldoNuevo=1000.01; monto=0.01; referencia no vacía; saldo aumenta exactamente 0.01. 8. En psql-QA ejecutar: SELECT c.numero_cuenta, c.saldo, c.saldo_disponible, c.monto_retirado_hoy, (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.created_at>=<inicio>) AS movs, (SELECT count(*) FROM contabilidad.asientos a JOIN transacciones.movimientos m ON m.id=a.movimiento_id WHERE m.cuenta_id=c.id AND a.created_at>=<inicio>) AS asientos FROM cuentas.cuentas c WHERE c.numero_cuenta IN ('AH000000000001','MO000000000002','AH000000000003') ORDER BY 1; Comprobar: un movimiento y dos asientos por 0.01. 9. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_cajero", "numeroCuenta": "AH000000000001", "monto": "0.01"}
Resultado esperado	HTTP 200; saldoNuevo=1000.01; monto=0.01; referencia no vacía; saldo aumenta exactamente 0.01; un movimiento y dos asientos por 0.01.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

REQ-11 · Retiro en ventanilla con validacion de saldo y limite diario

TC-REQ11-001 · Retiro en límite diario: 499.99

Trazabilidad	REQ-11 → CP-REQ11-01 → TC-REQ11-001 Dato: DT-TC-REQ11-001
Técnica / prioridad	Valores límite · Alta Estado: Diseñado; no ejecutado
Precondiciones	PC-F: restaurar las cuentas implicadas a los valores de MockData_Sintetica.json; caja del cajero abierta con fondo Q2000.00; usuario qa_cajero con permisos efectivos. Ejecutar cada variante desde esa misma línea base. Cuenta AH: saldo=1000, disponible=1000, mínimo=100, límite=500, retirado=0.
Interfaz	POST http://localhost:15141/api/transacciones/retiro
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; 3. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Abrir caja con POST http://localhost:15141/api/cajas/abrir, Bearer de qa_cajero y cuerpo {"fondoInicial": 2000.00}; verificar HTTP 200. 5. En psql-QA registrar el instante de inicio: SELECT now(); 6. Enviar POST http://localhost:15141/api/transacciones/retiro con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"numeroCuenta": "AH000000000001", "monto": 499.99}. 7. Verificar en la respuesta: HTTP 200; saldoNuevo=500.01. 8. En psql-QA ejecutar: SELECT c.numero_cuenta, c.saldo, c.saldo_disponible, c.monto_retirado_hoy, (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.created_at>=<inicio>) AS movs, (SELECT count(*) FROM contabilidad.asientos a JOIN transacciones.movimientos m ON m.id=a.movimiento_id WHERE m.cuenta_id=c.id AND a.created_at>=<inicio>) AS asientos FROM cuentas.cuentas c WHERE c.numero_cuenta IN ('AH000000000001','MO000000000002','AH000000000003') ORDER BY 1; Comprobar: monto_retirado_hoy=499.99; un movimiento RETIRO y partida doble. 9. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_cajero", "numeroCuenta": "AH000000000001", "monto": "499.99"}
Resultado esperado	HTTP 200; saldoNuevo=500.01; monto_retirado_hoy=499.99; un movimiento RETIRO y partida doble.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-04. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ11-002 · Retiro en límite diario: 500.00

Trazabilidad	REQ-11 → CP-REQ11-01 → TC-REQ11-002 Dato: DT-TC-REQ11-002
Técnica / prioridad	Valores límite · Alta Estado: Diseñado; no ejecutado
Precondiciones	PC-F: restaurar las cuentas implicadas a los valores de MockData_Sintetica.json; caja del cajero abierta con fondo Q2000.00; usuario qa_cajero con permisos efectivos. Ejecutar cada variante desde esa misma línea base. Cuenta AH: saldo=1000, disponible=1000, mínimo=100, límite=500, retirado=0.
Interfaz	POST http://localhost:15141/api/transacciones/retiro
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; 3. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Abrir caja con POST http://localhost:15141/api/cajas/abrir, Bearer de qa_cajero y cuerpo {"fondoInicial": 2000.00}; verificar HTTP 200. 5. En psql-QA registrar el instante de inicio: SELECT now(); 6. Enviar POST http://localhost:15141/api/transacciones/retiro con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"numeroCuenta": "AH000000000001", "monto": 500.00}. 7. Verificar en la respuesta: HTTP 200; saldoNuevo=500.0. 8. En psql-QA ejecutar: SELECT c.numero_cuenta, c.saldo, c.saldo_disponible, c.monto_retirado_hoy, (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.created_at>=<inicio>) AS movs, (SELECT count(*) FROM contabilidad.asientos a JOIN transacciones.movimientos m ON m.id=a.movimiento_id WHERE m.cuenta_id=c.id AND a.created_at>=<inicio>) AS asientos FROM cuentas.cuentas c WHERE c.numero_cuenta IN ('AH000000000001','MO000000000002','AH000000000003') ORDER BY 1; Comprobar: monto_retirado_hoy=500.00; un movimiento RETIRO y partida doble. 9. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_cajero", "numeroCuenta": "AH000000000001", "monto": "500.00"}
Resultado esperado	HTTP 200; saldoNuevo=500.0; monto_retirado_hoy=500.00; un movimiento RETIRO y partida doble.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-04. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ11-003 · Retiro en límite diario: 500.01

Trazabilidad	REQ-11 → CP-REQ11-01 → TC-REQ11-003 Dato: DT-TC-REQ11-003
Técnica / prioridad	Valores límite · Alta Estado: Diseñado; no ejecutado
Precondiciones	PC-F: restaurar las cuentas implicadas a los valores de MockData_Sintetica.json; caja del cajero abierta con fondo Q2000.00; usuario qa_cajero con permisos efectivos. Ejecutar cada variante desde esa misma línea base. Cuenta AH: saldo=1000, disponible=1000, mínimo=100, límite=500, retirado=0.
Interfaz	POST http://localhost:15141/api/transacciones/retiro
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; 3. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Abrir caja con POST http://localhost:15141/api/cajas/abrir, Bearer de qa_cajero y cuerpo {"fondoInicial": 2000.00}; verificar HTTP 200. 5. En psql-QA registrar el instante de inicio: SELECT now(); 6. Enviar POST http://localhost:15141/api/transacciones/retiro con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"numeroCuenta": "AH000000000001", "monto": 500.01}. 7. Verificar en la respuesta: HTTP 422. 8. En psql-QA ejecutar: SELECT c.numero_cuenta, c.saldo, c.saldo_disponible, c.monto_retirado_hoy, (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.created_at>=<inicio>) AS movs, (SELECT count(*) FROM contabilidad.asientos a JOIN transacciones.movimientos m ON m.id=a.movimiento_id WHERE m.cuenta_id=c.id AND a.created_at>=<inicio>) AS asientos FROM cuentas.cuentas c WHERE c.numero_cuenta IN ('AH000000000001','MO000000000002','AH000000000003') ORDER BY 1; Comprobar: sin débito, movimiento ni asiento; mensaje identifica límite diario 500 y retirado hoy 0. 9. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_cajero", "numeroCuenta": "AH000000000001", "monto": "500.01"}
Resultado esperado	HTTP 422; sin débito, movimiento ni asiento; mensaje identifica límite diario 500 y retirado hoy 0.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-04. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ11-004 · Saldo mínimo con retiro 899.99

Trazabilidad	REQ-11 → CP-REQ11-02 → TC-REQ11-004 Dato: DT-TC-REQ11-004
Técnica / prioridad	Valores límite · Alta Estado: Diseñado; no ejecutado
Precondiciones	PC-F: restaurar las cuentas implicadas a los valores de MockData_Sintetica.json; caja del cajero abierta con fondo Q2000.00; usuario qa_cajero con permisos efectivos. Ejecutar cada variante desde esa misma línea base. Anular límite diario para aislar mínimo; saldo=disponible=1000, mínimo=100.
Interfaz	POST http://localhost:15141/api/transacciones/retiro
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA ajustar la precondición: UPDATE cuentas.cuentas SET limite_retiro_diario=NULL WHERE numero_cuenta='AH000000000001'; 3. En psql-QA cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; 4. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 5. Abrir caja con POST http://localhost:15141/api/cajas/abrir, Bearer de qa_cajero y cuerpo {"fondoInicial": 2000.00}; verificar HTTP 200. 6. En psql-QA registrar el instante de inicio: SELECT now(); 7. Enviar POST http://localhost:15141/api/transacciones/retiro con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"numeroCuenta": "AH000000000001", "monto": 899.99}. 8. Verificar en la respuesta: HTTP 200. 9. En psql-QA ejecutar: SELECT c.numero_cuenta, c.saldo, c.saldo_disponible, c.monto_retirado_hoy, (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.created_at>=<inicio>) AS movs, (SELECT count(*) FROM contabilidad.asientos a JOIN transacciones.movimientos m ON m.id=a.movimiento_id WHERE m.cuenta_id=c.id AND a.created_at>=<inicio>) AS asientos FROM cuentas.cuentas c WHERE c.numero_cuenta IN ('AH000000000001','MO000000000002','AH000000000003') ORDER BY 1; Comprobar: saldo final=100.01; saldo final >=100. 10. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_cajero", "numeroCuenta": "AH000000000001", "monto": "899.99"}
Resultado esperado	HTTP 200; saldo final=100.01; saldo final >=100.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-04. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ11-005 · Saldo mínimo con retiro 900.00

Trazabilidad	REQ-11 → CP-REQ11-02 → TC-REQ11-005 Dato: DT-TC-REQ11-005
Técnica / prioridad	Valores límite · Alta Estado: Diseñado; no ejecutado
Precondiciones	PC-F: restaurar las cuentas implicadas a los valores de MockData_Sintetica.json; caja del cajero abierta con fondo Q2000.00; usuario qa_cajero con permisos efectivos. Ejecutar cada variante desde esa misma línea base. Anular límite diario para aislar mínimo; saldo=disponible=1000, mínimo=100.
Interfaz	POST http://localhost:15141/api/transacciones/retiro
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA ajustar la precondición: UPDATE cuentas.cuentas SET limite_retiro_diario=NULL WHERE numero_cuenta='AH000000000001'; 3. En psql-QA cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; 4. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 5. Abrir caja con POST http://localhost:15141/api/cajas/abrir, Bearer de qa_cajero y cuerpo {"fondoInicial": 2000.00}; verificar HTTP 200. 6. En psql-QA registrar el instante de inicio: SELECT now(); 7. Enviar POST http://localhost:15141/api/transacciones/retiro con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"numeroCuenta": "AH000000000001", "monto": 900.00}. 8. Verificar en la respuesta: HTTP 200. 9. En psql-QA ejecutar: SELECT c.numero_cuenta, c.saldo, c.saldo_disponible, c.monto_retirado_hoy, (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.created_at>=<inicio>) AS movs, (SELECT count(*) FROM contabilidad.asientos a JOIN transacciones.movimientos m ON m.id=a.movimiento_id WHERE m.cuenta_id=c.id AND a.created_at>=<inicio>) AS asientos FROM cuentas.cuentas c WHERE c.numero_cuenta IN ('AH000000000001','MO000000000002','AH000000000003') ORDER BY 1; Comprobar: saldo final=100.0; saldo final >=100. 10. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_cajero", "numeroCuenta": "AH000000000001", "monto": "900.00"}
Resultado esperado	HTTP 200; saldo final=100.0; saldo final >=100.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-04. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ11-006 · Saldo mínimo con retiro 900.01

Trazabilidad	REQ-11 → CP-REQ11-02 → TC-REQ11-006 Dato: DT-TC-REQ11-006
Técnica / prioridad	Valores límite · Alta Estado: Diseñado; no ejecutado
Precondiciones	PC-F: restaurar las cuentas implicadas a los valores de MockData_Sintetica.json; caja del cajero abierta con fondo Q2000.00; usuario qa_cajero con permisos efectivos. Ejecutar cada variante desde esa misma línea base. Anular límite diario para aislar mínimo; saldo=disponible=1000, mínimo=100.
Interfaz	POST http://localhost:15141/api/transacciones/retiro
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA ajustar la precondición: UPDATE cuentas.cuentas SET limite_retiro_diario=NULL WHERE numero_cuenta='AH000000000001'; 3. En psql-QA cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; 4. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 5. Abrir caja con POST http://localhost:15141/api/cajas/abrir, Bearer de qa_cajero y cuerpo {"fondoInicial": 2000.00}; verificar HTTP 200. 6. En psql-QA registrar el instante de inicio: SELECT now(); 7. Enviar POST http://localhost:15141/api/transacciones/retiro con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"numeroCuenta": "AH000000000001", "monto": 900.01}. 8. Verificar en la respuesta: HTTP 422; mensaje identifica saldo proyectado 99.99 por debajo del mínimo 100.00. 9. En psql-QA ejecutar: SELECT c.numero_cuenta, c.saldo, c.saldo_disponible, c.monto_retirado_hoy, (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.created_at>=<inicio>) AS movs, (SELECT count(*) FROM contabilidad.asientos a JOIN transacciones.movimientos m ON m.id=a.movimiento_id WHERE m.cuenta_id=c.id AND a.created_at>=<inicio>) AS asientos FROM cuentas.cuentas c WHERE c.numero_cuenta IN ('AH000000000001','MO000000000002','AH000000000003') ORDER BY 1; Comprobar: saldo permanece 1000.00; sin movimientos/asientos. 10. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_cajero", "numeroCuenta": "AH000000000001", "monto": "900.01"}
Resultado esperado	HTTP 422; saldo permanece 1000.00; sin movimientos/asientos; mensaje identifica saldo proyectado 99.99 por debajo del mínimo 100.00.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-04. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ11-007 · Saldo disponible insuficiente

Trazabilidad	REQ-11 → CP-REQ11-03 → TC-REQ11-007 Dato: DT-TC-REQ11-007
Técnica / prioridad	Tabla de decisión · Alta Estado: Diseñado; no ejecutado
Precondiciones	PC-F: restaurar las cuentas implicadas a los valores de MockData_Sintetica.json; caja del cajero abierta con fondo Q2000.00; usuario qa_cajero con permisos efectivos. Ejecutar cada variante desde esa misma línea base. Configurar saldo=1000, disponible=200, retenido=800; mínimo=100; límite=500.
Interfaz	POST http://localhost:15141/api/transacciones/retiro
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA ajustar la precondición: UPDATE cuentas.cuentas SET saldo_disponible=200.00, saldo_retenido=800.00 WHERE numero_cuenta='AH000000000001'; 3. En psql-QA cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; 4. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 5. Abrir caja con POST http://localhost:15141/api/cajas/abrir, Bearer de qa_cajero y cuerpo {"fondoInicial": 2000.00}; verificar HTTP 200. 6. En psql-QA registrar el instante de inicio: SELECT now(); 7. Enviar POST http://localhost:15141/api/transacciones/retiro con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"numeroCuenta": "AH000000000001", "monto": 200.01}. 8. Verificar en la respuesta: HTTP 422; mensaje identifica saldo disponible 200 y solicitado 200.01. 9. En psql-QA ejecutar: SELECT c.numero_cuenta, c.saldo, c.saldo_disponible, c.monto_retirado_hoy, (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.created_at>=<inicio>) AS movs, (SELECT count(*) FROM contabilidad.asientos a JOIN transacciones.movimientos m ON m.id=a.movimiento_id WHERE m.cuenta_id=c.id AND a.created_at>=<inicio>) AS asientos FROM cuentas.cuentas c WHERE c.numero_cuenta IN ('AH000000000001','MO000000000002','AH000000000003') ORDER BY 1; Comprobar: saldo permanece 1000; disponible permanece 200; cero registros financieros nuevos. 10. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_cajero", "numeroCuenta": "AH000000000001", "monto": "200.01"}
Resultado esperado	HTTP 422; saldo permanece 1000; disponible permanece 200; cero registros financieros nuevos; mensaje identifica saldo disponible 200 y solicitado 200.01.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-04. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

REQ-12 · Transferencia interna en ventanilla

TC-REQ12-001 · Transferencia interna válida

Trazabilidad	REQ-12 → CP-REQ12-01 → TC-REQ12-001 Dato: DT-TC-REQ12-001
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	PC-F: restaurar las cuentas implicadas a los valores de MockData_Sintetica.json; caja del cajero abierta con fondo Q2000.00; usuario qa_cajero con permisos efectivos. Ejecutar cada variante desde esa misma línea base.
Interfaz	POST http://localhost:15141/api/transacciones/transferencia
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; 3. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Abrir caja con POST http://localhost:15141/api/cajas/abrir, Bearer de qa_cajero y cuerpo {"fondoInicial": 2000.00}; verificar HTTP 200. 5. En psql-QA registrar el instante de inicio: SELECT now(); 6. Enviar POST http://localhost:15141/api/transacciones/transferencia con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"numeroCuentaOrigen": "AH000000000001", "numeroCuentaDestino": "MO000000000002", "monto": 100.00, "concepto": "E2 QA"}. 7. Verificar en la respuesta: HTTP 200; saldoOrigen=900.00, saldoDestino=600.00, monto=100. 8. En psql-QA ejecutar: SELECT c.numero_cuenta, c.saldo, c.saldo_disponible, c.monto_retirado_hoy, (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.created_at>=<inicio>) AS movs, (SELECT count(*) FROM contabilidad.asientos a JOIN transacciones.movimientos m ON m.id=a.movimiento_id WHERE m.cuenta_id=c.id AND a.created_at>=<inicio>) AS asientos FROM cuentas.cuentas c WHERE c.numero_cuenta IN ('AH000000000001','MO000000000002','AH000000000003') ORDER BY 1; Comprobar: suma de saldos conserva 1500; dos movimientos opuestos; partida doble equilibrada. 9. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_cajero", "numeroCuentaOrigen": "AH000000000001", "numeroCuentaDestino": "MO000000000002", "monto": "100.00", "concepto": "E2 QA"}
Resultado esperado	HTTP 200; saldoOrigen=900.00, saldoDestino=600.00, monto=100; suma de saldos conserva 1500; dos movimientos opuestos; partida doble equilibrada.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-04. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ12-002 · Origen igual a destino

Trazabilidad	REQ-12 → CP-REQ12-01 → TC-REQ12-002 Dato: DT-TC-REQ12-002
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	PC-F: restaurar las cuentas implicadas a los valores de MockData_Sintetica.json; caja del cajero abierta con fondo Q2000.00; usuario qa_cajero con permisos efectivos. Ejecutar cada variante desde esa misma línea base.
Interfaz	POST http://localhost:15141/api/transacciones/transferencia
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; 3. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Abrir caja con POST http://localhost:15141/api/cajas/abrir, Bearer de qa_cajero y cuerpo {"fondoInicial": 2000.00}; verificar HTTP 200. 5. En psql-QA registrar el instante de inicio: SELECT now(); 6. Enviar POST http://localhost:15141/api/transacciones/transferencia con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"numeroCuentaOrigen": "AH000000000001", "numeroCuentaDestino": "AH000000000001", "monto": 100.00, "concepto": "E2 QA"}. 7. Verificar en la respuesta: HTTP 400; message="La cuenta origen y destino no pueden ser la misma". 8. En psql-QA ejecutar: SELECT c.numero_cuenta, c.saldo, c.saldo_disponible, c.monto_retirado_hoy, (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.created_at>=<inicio>) AS movs, (SELECT count(*) FROM contabilidad.asientos a JOIN transacciones.movimientos m ON m.id=a.movimiento_id WHERE m.cuenta_id=c.id AND a.created_at>=<inicio>) AS asientos FROM cuentas.cuentas c WHERE c.numero_cuenta IN ('AH000000000001','MO000000000002','AH000000000003') ORDER BY 1; Comprobar: saldos intactos. 9. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_cajero", "numeroCuentaOrigen": "AH000000000001", "numeroCuentaDestino": "AH000000000001", "monto": "100.00", "concepto": "E2 QA"}
Resultado esperado	HTTP 400; message="La cuenta origen y destino no pueden ser la misma"; saldos intactos.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-04. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ12-003 · Límite diario uniforme entre ventanilla y portal

Trazabilidad	REQ-12 → CP-REQ12-02 → TC-REQ12-003 Dato: DT-TC-REQ12-003
Técnica / prioridad	Tabla de decisión · Alta Estado: Diseñado; no ejecutado
Precondiciones	PC-F: restaurar las cuentas implicadas a los valores de MockData_Sintetica.json; caja del cajero abierta con fondo Q2000.00; usuario qa_cajero con permisos efectivos. Ejecutar cada variante desde esa misma línea base. Repetir la operación desde portal con qa_cliente_a, restaurando los mismos saldos antes de cada canal.
Interfaz	POST http://localhost:15141/api/transacciones/transferencia
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; 3. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Abrir caja con POST http://localhost:15141/api/cajas/abrir, Bearer de qa_cajero y cuerpo {"fondoInicial": 2000.00}; verificar HTTP 200. 5. En psql-QA registrar el instante de inicio: SELECT now(); 6. Enviar POST http://localhost:15141/api/transacciones/transferencia con Authorization: Bearer <token> y cuerpo {"numeroCuentaOrigen":"AH000000000001","numeroCuentaDestino":"MO000000000002","monto":500.01,"concepto":"E2 QA"}. 7. Verificar en la respuesta: HTTP 422; message="La transferencia supera el limite diario de la cuenta origen.". 8. En psql-QA ejecutar: SELECT numero_cuenta, saldo, monto_retirado_hoy FROM cuentas.cuentas WHERE numero_cuenta IN ('AH000000000001','MO000000000002'); esperar 1000.00/0.00 y 500.00; sin movimientos desde <inicio>. 9. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 10. Autenticar qa_cliente_a con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cliente_a","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 11. Enviar POST http://localhost:15141/api/portal/transferencias con el mismo cuerpo y el token de qa_cliente_a; verificar el mismo HTTP 422, mensaje y ausencia de efectos con la consulta anterior. Un HTTP 200 por ventanilla se registra como defecto H-03.
Datos de entrada	{"perfil": "qa_cajero", "numeroCuentaOrigen": "AH000000000001", "numeroCuentaDestino": "MO000000000002", "monto": "500.01", "concepto": "E2 QA"}
Resultado esperado	Ventanilla y portal deben responder HTTP 422; message="La transferencia supera el limite diario de la cuenta origen."; saldos se conservan en 1000 y 500; acumulador permanece 0; sin movimientos/asientos nuevos. El 200 de ventanilla observado por análisis previo es defecto H-03.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-04. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

REQ-13 · Consulta paginada de historial de movimientos

TC-REQ13-001 · Tamaño de página 0

Trazabilidad	REQ-13 → CP-REQ13-01 → TC-REQ13-001 Dato: DT-TC-REQ13-001
Técnica / prioridad	Valores límite · Media Estado: Diseñado; no ejecutado
Precondiciones	qa_cajero; crear 120 movimientos exclusivamente sintéticos para la cuenta de consulta, con fechas distintas; el mock masivo trae 10 por cuenta y debe ampliarse con preparar_escenario.py.
Interfaz	GET http://localhost:15141/api/transacciones/historial/b5425a11-d429-58b4-906a-7820e0ad1e31?pagina=1&tamano=0
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py --paginacion (la cuenta AH000000000001 queda con 120 movimientos). 2. En psql-QA confirmar: SELECT count(*) FROM transacciones.movimientos WHERE cuenta_id='b5425a11-d429-58b4-906a-7820e0ad1e31'; esperar 120. 3. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Enviar GET http://localhost:15141/api/transacciones/historial/b5425a11-d429-58b4-906a-7820e0ad1e31?pagina=1&tamano=0 con Authorization: Bearer <token>. 5. Verificar en la respuesta: HTTP 200; arreglo con exactamente 20 registros de esa cuenta; fechas descendentes; tamaño efectivo=20. 6. En psql-QA comparar los IDs devueltos con: SELECT id FROM transacciones.movimientos WHERE cuenta_id='b5425a11-d429-58b4-906a-7820e0ad1e31' ORDER BY created_at DESC LIMIT 20 OFFSET 0; deben coincidir en orden.
Datos de entrada	{"perfil": "qa_cajero", "cuentaId": "b5425a11-d429-58b4-906a-7820e0ad1e31", "pagina": 1, "tamano": 0}
Resultado esperado	HTTP 200; arreglo con exactamente 20 registros de esa cuenta; fechas descendentes; tamaño efectivo=20.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ13-002 · Tamaño de página 1

Trazabilidad	REQ-13 → CP-REQ13-01 → TC-REQ13-002 Dato: DT-TC-REQ13-002
Técnica / prioridad	Valores límite · Media Estado: Diseñado; no ejecutado
Precondiciones	qa_cajero; crear 120 movimientos exclusivamente sintéticos para la cuenta de consulta, con fechas distintas; el mock masivo trae 10 por cuenta y debe ampliarse con preparar_escenario.py.
Interfaz	GET http://localhost:15141/api/transacciones/historial/b5425a11-d429-58b4-906a-7820e0ad1e31?pagina=1&tamano=1
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py --paginacion (la cuenta AH000000000001 queda con 120 movimientos). 2. En psql-QA confirmar: SELECT count(*) FROM transacciones.movimientos WHERE cuenta_id='b5425a11-d429-58b4-906a-7820e0ad1e31'; esperar 120. 3. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Enviar GET http://localhost:15141/api/transacciones/historial/b5425a11-d429-58b4-906a-7820e0ad1e31?pagina=1&tamano=1 con Authorization: Bearer <token>. 5. Verificar en la respuesta: HTTP 200; arreglo con exactamente 1 registros de esa cuenta; fechas descendentes; tamaño efectivo=1. 6. En psql-QA comparar los IDs devueltos con: SELECT id FROM transacciones.movimientos WHERE cuenta_id='b5425a11-d429-58b4-906a-7820e0ad1e31' ORDER BY created_at DESC LIMIT 1 OFFSET 0; deben coincidir en orden.
Datos de entrada	{"perfil": "qa_cajero", "cuentaId": "b5425a11-d429-58b4-906a-7820e0ad1e31", "pagina": 1, "tamano": 1}
Resultado esperado	HTTP 200; arreglo con exactamente 1 registros de esa cuenta; fechas descendentes; tamaño efectivo=1.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ13-003 · Tamaño de página 99

Trazabilidad	REQ-13 → CP-REQ13-01 → TC-REQ13-003 Dato: DT-TC-REQ13-003
Técnica / prioridad	Valores límite · Media Estado: Diseñado; no ejecutado
Precondiciones	qa_cajero; crear 120 movimientos exclusivamente sintéticos para la cuenta de consulta, con fechas distintas; el mock masivo trae 10 por cuenta y debe ampliarse con preparar_escenario.py.
Interfaz	GET http://localhost:15141/api/transacciones/historial/b5425a11-d429-58b4-906a-7820e0ad1e31?pagina=1&tamano=99
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py --paginacion (la cuenta AH000000000001 queda con 120 movimientos). 2. En psql-QA confirmar: SELECT count(*) FROM transacciones.movimientos WHERE cuenta_id='b5425a11-d429-58b4-906a-7820e0ad1e31'; esperar 120. 3. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Enviar GET http://localhost:15141/api/transacciones/historial/b5425a11-d429-58b4-906a-7820e0ad1e31?pagina=1&tamano=99 con Authorization: Bearer <token>. 5. Verificar en la respuesta: HTTP 200; arreglo con exactamente 99 registros de esa cuenta; fechas descendentes; tamaño efectivo=99. 6. En psql-QA comparar los IDs devueltos con: SELECT id FROM transacciones.movimientos WHERE cuenta_id='b5425a11-d429-58b4-906a-7820e0ad1e31' ORDER BY created_at DESC LIMIT 99 OFFSET 0; deben coincidir en orden.
Datos de entrada	{"perfil": "qa_cajero", "cuentaId": "b5425a11-d429-58b4-906a-7820e0ad1e31", "pagina": 1, "tamano": 99}
Resultado esperado	HTTP 200; arreglo con exactamente 99 registros de esa cuenta; fechas descendentes; tamaño efectivo=99.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ13-004 · Tamaño de página 100

Trazabilidad	REQ-13 → CP-REQ13-01 → TC-REQ13-004 Dato: DT-TC-REQ13-004
Técnica / prioridad	Valores límite · Media Estado: Diseñado; no ejecutado
Precondiciones	qa_cajero; crear 120 movimientos exclusivamente sintéticos para la cuenta de consulta, con fechas distintas; el mock masivo trae 10 por cuenta y debe ampliarse con preparar_escenario.py.
Interfaz	GET http://localhost:15141/api/transacciones/historial/b5425a11-d429-58b4-906a-7820e0ad1e31?pagina=1&tamano=100
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py --paginacion (la cuenta AH000000000001 queda con 120 movimientos). 2. En psql-QA confirmar: SELECT count(*) FROM transacciones.movimientos WHERE cuenta_id='b5425a11-d429-58b4-906a-7820e0ad1e31'; esperar 120. 3. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Enviar GET http://localhost:15141/api/transacciones/historial/b5425a11-d429-58b4-906a-7820e0ad1e31?pagina=1&tamano=100 con Authorization: Bearer <token>. 5. Verificar en la respuesta: HTTP 200; arreglo con exactamente 100 registros de esa cuenta; fechas descendentes; tamaño efectivo=100. 6. En psql-QA comparar los IDs devueltos con: SELECT id FROM transacciones.movimientos WHERE cuenta_id='b5425a11-d429-58b4-906a-7820e0ad1e31' ORDER BY created_at DESC LIMIT 100 OFFSET 0; deben coincidir en orden.
Datos de entrada	{"perfil": "qa_cajero", "cuentaId": "b5425a11-d429-58b4-906a-7820e0ad1e31", "pagina": 1, "tamano": 100}
Resultado esperado	HTTP 200; arreglo con exactamente 100 registros de esa cuenta; fechas descendentes; tamaño efectivo=100.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ13-005 · Tamaño de página 101

Trazabilidad	REQ-13 → CP-REQ13-01 → TC-REQ13-005 Dato: DT-TC-REQ13-005
Técnica / prioridad	Valores límite · Media Estado: Diseñado; no ejecutado
Precondiciones	qa_cajero; crear 120 movimientos exclusivamente sintéticos para la cuenta de consulta, con fechas distintas; el mock masivo trae 10 por cuenta y debe ampliarse con preparar_escenario.py.
Interfaz	GET http://localhost:15141/api/transacciones/historial/b5425a11-d429-58b4-906a-7820e0ad1e31?pagina=1&tamano=101
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py --paginacion (la cuenta AH000000000001 queda con 120 movimientos). 2. En psql-QA confirmar: SELECT count(*) FROM transacciones.movimientos WHERE cuenta_id='b5425a11-d429-58b4-906a-7820e0ad1e31'; esperar 120. 3. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Enviar GET http://localhost:15141/api/transacciones/historial/b5425a11-d429-58b4-906a-7820e0ad1e31?pagina=1&tamano=101 con Authorization: Bearer <token>. 5. Verificar en la respuesta: HTTP 200; arreglo con exactamente 100 registros de esa cuenta; fechas descendentes; tamaño efectivo=100. 6. En psql-QA comparar los IDs devueltos con: SELECT id FROM transacciones.movimientos WHERE cuenta_id='b5425a11-d429-58b4-906a-7820e0ad1e31' ORDER BY created_at DESC LIMIT 100 OFFSET 0; deben coincidir en orden.
Datos de entrada	{"perfil": "qa_cajero", "cuentaId": "b5425a11-d429-58b4-906a-7820e0ad1e31", "pagina": 1, "tamano": 101}
Resultado esperado	HTTP 200; arreglo con exactamente 100 registros de esa cuenta; fechas descendentes; tamaño efectivo=100.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ13-006 · Número de página 0

Trazabilidad	REQ-13 → CP-REQ13-02 → TC-REQ13-006 Dato: DT-TC-REQ13-006
Técnica / prioridad	Valores límite · Media Estado: Diseñado; no ejecutado
Precondiciones	Fixture ampliado a 120 movimientos por preparar_escenario.py; fechas únicas.
Interfaz	GET http://localhost:15141/api/transacciones/historial/b5425a11-d429-58b4-906a-7820e0ad1e31?pagina=0&tamano=20
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py --paginacion (la cuenta AH000000000001 queda con 120 movimientos). 2. En psql-QA confirmar: SELECT count(*) FROM transacciones.movimientos WHERE cuenta_id='b5425a11-d429-58b4-906a-7820e0ad1e31'; esperar 120. 3. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Enviar GET http://localhost:15141/api/transacciones/historial/b5425a11-d429-58b4-906a-7820e0ad1e31?pagina=0&tamano=20 con Authorization: Bearer <token>. 5. Verificar en la respuesta: HTTP 200; 20 registros; offset=0; mismos IDs que página 1. 6. En psql-QA comparar los IDs devueltos con: SELECT id FROM transacciones.movimientos WHERE cuenta_id='b5425a11-d429-58b4-906a-7820e0ad1e31' ORDER BY created_at DESC LIMIT 20 OFFSET 0; deben coincidir en orden.
Datos de entrada	{"perfil": "qa_cajero", "cuentaId": "b5425a11-d429-58b4-906a-7820e0ad1e31", "pagina": 0, "tamano": 20}
Resultado esperado	HTTP 200; 20 registros; offset=0; mismos IDs que página 1.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ13-007 · Número de página 1

Trazabilidad	REQ-13 → CP-REQ13-02 → TC-REQ13-007 Dato: DT-TC-REQ13-007
Técnica / prioridad	Valores límite · Media Estado: Diseñado; no ejecutado
Precondiciones	Fixture ampliado a 120 movimientos por preparar_escenario.py; fechas únicas.
Interfaz	GET http://localhost:15141/api/transacciones/historial/b5425a11-d429-58b4-906a-7820e0ad1e31?pagina=1&tamano=20
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py --paginacion (la cuenta AH000000000001 queda con 120 movimientos). 2. En psql-QA confirmar: SELECT count(*) FROM transacciones.movimientos WHERE cuenta_id='b5425a11-d429-58b4-906a-7820e0ad1e31'; esperar 120. 3. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Enviar GET http://localhost:15141/api/transacciones/historial/b5425a11-d429-58b4-906a-7820e0ad1e31?pagina=1&tamano=20 con Authorization: Bearer <token>. 5. Verificar en la respuesta: HTTP 200; 20 registros; offset=0; IDs corresponden al orden descendente del fixture. 6. En psql-QA comparar los IDs devueltos con: SELECT id FROM transacciones.movimientos WHERE cuenta_id='b5425a11-d429-58b4-906a-7820e0ad1e31' ORDER BY created_at DESC LIMIT 20 OFFSET 0; deben coincidir en orden.
Datos de entrada	{"perfil": "qa_cajero", "cuentaId": "b5425a11-d429-58b4-906a-7820e0ad1e31", "pagina": 1, "tamano": 20}
Resultado esperado	HTTP 200; 20 registros; offset=0; IDs corresponden al orden descendente del fixture.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ13-008 · Número de página 2

Trazabilidad	REQ-13 → CP-REQ13-02 → TC-REQ13-008 Dato: DT-TC-REQ13-008
Técnica / prioridad	Valores límite · Media Estado: Diseñado; no ejecutado
Precondiciones	Fixture ampliado a 120 movimientos por preparar_escenario.py; fechas únicas.
Interfaz	GET http://localhost:15141/api/transacciones/historial/b5425a11-d429-58b4-906a-7820e0ad1e31?pagina=2&tamano=20
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py --paginacion (la cuenta AH000000000001 queda con 120 movimientos). 2. En psql-QA confirmar: SELECT count(*) FROM transacciones.movimientos WHERE cuenta_id='b5425a11-d429-58b4-906a-7820e0ad1e31'; esperar 120. 3. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Enviar GET http://localhost:15141/api/transacciones/historial/b5425a11-d429-58b4-906a-7820e0ad1e31?pagina=2&tamano=20 con Authorization: Bearer <token>. 5. Verificar en la respuesta: HTTP 200; 20 registros; offset=20; IDs corresponden al orden descendente del fixture. 6. En psql-QA comparar los IDs devueltos con: SELECT id FROM transacciones.movimientos WHERE cuenta_id='b5425a11-d429-58b4-906a-7820e0ad1e31' ORDER BY created_at DESC LIMIT 20 OFFSET 20; deben coincidir en orden.
Datos de entrada	{"perfil": "qa_cajero", "cuentaId": "b5425a11-d429-58b4-906a-7820e0ad1e31", "pagina": 2, "tamano": 20}
Resultado esperado	HTTP 200; 20 registros; offset=20; IDs corresponden al orden descendente del fixture.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

REQ-14 · Apertura de cuenta AHORRO o MONETARIA

TC-REQ14-001 · Apertura tipo AHORRO

Trazabilidad	REQ-14 → CP-REQ14-01 → TC-REQ14-001 Dato: DT-TC-REQ14-001
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	PC-F: restaurar las cuentas implicadas a los valores de MockData_Sintetica.json; caja del cajero abierta con fondo Q2000.00; usuario qa_cajero con permisos efectivos. Ejecutar cada variante desde esa misma línea base. Cliente activo; tipos de cuenta activos del catálogo QA.
Interfaz	POST http://localhost:15141/api/cuentas
Pasos	1. En psql-QA confirmar el estado del cliente: SELECT estado FROM crm.clientes WHERE id='d80b392d-d16a-5f31-b7d6-3276492fa684'; 2. En psql-QA registrar el instante de inicio: SELECT now(); 3. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Enviar POST http://localhost:15141/api/cuentas con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"clienteId": "d80b392d-d16a-5f31-b7d6-3276492fa684", "tipoCuenta": "AHORRO"}. 5. Verificar en la respuesta: HTTP 200; id UUID y numeroCuenta no vacío; límite coincide con catálogo, no con importe supuesto. 6. En psql-QA ejecutar: SELECT numero_cuenta, tipo_cuenta, estado, saldo_minimo, limite_retiro_diario FROM cuentas.cuentas WHERE cliente_id='d80b392d-d16a-5f31-b7d6-3276492fa684' AND created_at>=<inicio>; y SELECT codigo, saldo_minimo, limite_retiro_diario FROM cuentas.tipos_cuenta WHERE codigo='AHORRO'; Comprobar: cuenta ACTIVA del cliente indicado; saldoMinimo coincide exactamente con cuentas.tipos_cuenta para AHORRO.
Datos de entrada	{"perfil": "qa_cajero", "clienteId": "d80b392d-d16a-5f31-b7d6-3276492fa684", "tipoCuenta": "AHORRO"}
Resultado esperado	HTTP 200; id UUID y numeroCuenta no vacío; cuenta ACTIVA del cliente indicado; saldoMinimo coincide exactamente con cuentas.tipos_cuenta para AHORRO; límite coincide con catálogo, no con importe supuesto.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ14-002 · Apertura tipo MONETARIA

Trazabilidad	REQ-14 → CP-REQ14-01 → TC-REQ14-002 Dato: DT-TC-REQ14-002
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	PC-F: restaurar las cuentas implicadas a los valores de MockData_Sintetica.json; caja del cajero abierta con fondo Q2000.00; usuario qa_cajero con permisos efectivos. Ejecutar cada variante desde esa misma línea base. Cliente activo; tipos de cuenta activos del catálogo QA.
Interfaz	POST http://localhost:15141/api/cuentas
Pasos	1. En psql-QA confirmar el estado del cliente: SELECT estado FROM crm.clientes WHERE id='d80b392d-d16a-5f31-b7d6-3276492fa684'; 2. En psql-QA registrar el instante de inicio: SELECT now(); 3. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Enviar POST http://localhost:15141/api/cuentas con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"clienteId": "d80b392d-d16a-5f31-b7d6-3276492fa684", "tipoCuenta": "MONETARIA"}. 5. Verificar en la respuesta: HTTP 200; id UUID y numeroCuenta no vacío; límite coincide con catálogo, no con importe supuesto. 6. En psql-QA ejecutar: SELECT numero_cuenta, tipo_cuenta, estado, saldo_minimo, limite_retiro_diario FROM cuentas.cuentas WHERE cliente_id='d80b392d-d16a-5f31-b7d6-3276492fa684' AND created_at>=<inicio>; y SELECT codigo, saldo_minimo, limite_retiro_diario FROM cuentas.tipos_cuenta WHERE codigo='MONETARIA'; Comprobar: cuenta ACTIVA del cliente indicado; saldoMinimo coincide exactamente con cuentas.tipos_cuenta para MONETARIA.
Datos de entrada	{"perfil": "qa_cajero", "clienteId": "d80b392d-d16a-5f31-b7d6-3276492fa684", "tipoCuenta": "MONETARIA"}
Resultado esperado	HTTP 200; id UUID y numeroCuenta no vacío; cuenta ACTIVA del cliente indicado; saldoMinimo coincide exactamente con cuentas.tipos_cuenta para MONETARIA; límite coincide con catálogo, no con importe supuesto.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ14-003 · Cliente inactivo no puede abrir cuenta

Trazabilidad	REQ-14 → CP-REQ14-02 → TC-REQ14-003 Dato: DT-TC-REQ14-003
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	PC-F: restaurar las cuentas implicadas a los valores de MockData_Sintetica.json; caja del cajero abierta con fondo Q2000.00; usuario qa_cajero con permisos efectivos. Ejecutar cada variante desde esa misma línea base.
Interfaz	POST http://localhost:15141/api/cuentas
Pasos	1. En psql-QA confirmar el estado del cliente: SELECT estado FROM crm.clientes WHERE id='1a6c4002-0bcc-58d0-a474-b1524f76d24d'; 2. En psql-QA registrar el instante de inicio: SELECT now(); 3. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Enviar POST http://localhost:15141/api/cuentas con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"clienteId": "1a6c4002-0bcc-58d0-a474-b1524f76d24d", "tipoCuenta": "AHORRO"}. 5. Verificar en la respuesta: HTTP 422; message="El cliente no esta activo. Estado: INACTIVO". 6. En psql-QA ejecutar: SELECT numero_cuenta, tipo_cuenta, estado, saldo_minimo, limite_retiro_diario FROM cuentas.cuentas WHERE cliente_id='1a6c4002-0bcc-58d0-a474-b1524f76d24d' AND created_at>=<inicio>; y SELECT codigo, saldo_minimo, limite_retiro_diario FROM cuentas.tipos_cuenta WHERE codigo='AHORRO'; Comprobar: ninguna cuenta nueva.
Datos de entrada	{"perfil": "qa_cajero", "clienteId": "1a6c4002-0bcc-58d0-a474-b1524f76d24d", "tipoCuenta": "AHORRO"}
Resultado esperado	HTTP 422; message="El cliente no esta activo. Estado: INACTIVO"; ninguna cuenta nueva.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

REQ-15 · Consulta de cuentas y movimientos propios en el portal

TC-REQ15-001 · Portal lista solo cuentas propias

Trazabilidad	REQ-15 → CP-REQ15-01 → TC-REQ15-001 Dato: DT-TC-REQ15-001
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	qa_cliente_a vinculado a cliente 1; cliente 2 con cuentas separadas; no crear cuentas adicionales para este caso.
Interfaz	GET http://localhost:15141/api/portal/cuentas
Pasos	1. Autenticar qa_cliente_a con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cliente_a","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 2. Enviar GET http://localhost:15141/api/portal/cuentas con Authorization: Bearer <token>. 3. Verificar en la respuesta: HTTP 200; exactamente las dos cuentas del cliente 1 de la línea base; ningún cliente_id diferente ni cuenta AH000000000003. 4. En psql-QA comparar con: SELECT numero_cuenta FROM cuentas.cuentas WHERE cliente_id=(SELECT cliente_id FROM cuentas.cuentas WHERE numero_cuenta='AH000000000001') ORDER BY 1; la respuesta debe contener exactamente esas cuentas.
Datos de entrada	{"perfil": "qa_cliente_a"}
Resultado esperado	HTTP 200; exactamente las dos cuentas del cliente 1 de la línea base; ningún cliente_id diferente ni cuenta AH000000000003.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ15-002 · Consulta de movimientos ajenos

Trazabilidad	REQ-15 → CP-REQ15-02 → TC-REQ15-002 Dato: DT-TC-REQ15-002
Técnica / prioridad	Tabla de decisión · Alta Estado: Diseñado; no ejecutado
Precondiciones	Entorno QA E2 activo; datos sintéticos cargados; autenticación y permisos del perfil indicado en la entrada.
Interfaz	GET http://localhost:15141/api/portal/movimientos?cuentaId=917c75bc-51d9-5a1f-b927-b0a7bb6b0ead
Pasos	1. Autenticar qa_cliente_a con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cliente_a","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 2. Enviar GET http://localhost:15141/api/portal/movimientos?cuentaId=917c75bc-51d9-5a1f-b927-b0a7bb6b0ead con Authorization: Bearer <token>. 3. Verificar en la respuesta: HTTP 200; arreglo vacío. 4. En psql-QA confirmar que la cuenta es ajena: SELECT cliente_id FROM cuentas.cuentas WHERE id='917c75bc-51d9-5a1f-b927-b0a7bb6b0ead'; debe diferir del cliente de qa_cliente_a.
Datos de entrada	{"perfil": "qa_cliente_a", "cuentaId": "917c75bc-51d9-5a1f-b927-b0a7bb6b0ead"}
Resultado esperado	HTTP 200; arreglo vacío; no retorna ningún movimiento del cliente 2 (el contrato de consulta filtra propiedad, no devuelve 403).
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ15-003 · Resumen del portal limitado al cliente autenticado

Trazabilidad	REQ-15 → CP-REQ15-01 → TC-REQ15-003 Dato: DT-TC-REQ15-003
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	qa_cliente_a vinculado al cliente 0001; cuentas del cliente restauradas (1000.00 y 500.00).
Interfaz	GET http://localhost:15141/api/portal/resumen
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO0000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. Autenticar qa_cliente_a con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cliente_a","password":"E2Qa2026*"}; guardar token. 3. Enviar GET http://localhost:15141/api/portal/resumen con Authorization: Bearer <token>. 4. Verificar HTTP 200, cliente.id, nombreCompleto, los dos numeroCuenta exactos, saldoDisponibleTotal=1500.00 y cuentasActivas=2. 5. En psql-QA comparar con: SELECT numero_cuenta, saldo_disponible FROM cuentas.cuentas WHERE cliente_id='d80b392d-d16a-5f31-b7d6-3276492fa684' ORDER BY numero_cuenta; la respuesta debe contener exactamente esas filas.
Datos de entrada	{"perfil": "qa_cliente_a"}
Resultado esperado	HTTP 200; cliente.id="d80b392d-d16a-5f31-b7d6-3276492fa684"; cliente.nombreCompleto="Persona Sintetica QA 0001"; cuentas contiene exactamente AH000000000001 y MO0000000000002; saldoDisponibleTotal=1500.00; cuentasActivas=2; no aparece AH000000000003 ni MO0000000000004.
Base / bloqueo	Criterios aprobados por el equipo QA el 06-10-2026

TC-REQ15-004 · Portal sin JWT

Trazabilidad	REQ-15 → CP-REQ15-03 → TC-REQ15-004 Dato: DT-TC-REQ15-004
Técnica / prioridad	Tabla de decisión · Alta Estado: Diseñado; no ejecutado
Precondiciones	API interna QA activa; no enviar Authorization ni cookies.
Interfaz	GET http://localhost:15141/api/portal/resumen
Pasos	1. Enviar GET http://localhost:15141/api/portal/resumen sin encabezado Authorization. 2. Verificar HTTP 401. 3. Verificar que el cuerpo no contiene las claves cliente, cuentas ni saldoDisponibleTotal.
Datos de entrada	{"Authorization": "ausente"}
Resultado esperado	HTTP 401; cuerpo vacío, sin datos de cliente ni de cuentas.
Base / bloqueo	Criterios aprobados por el equipo QA el 06-10-2026

TC-REQ15-005 · JWT interno sin permiso de portal

Trazabilidad	REQ-15 → CP-REQ15-03 → TC-REQ15-005 Dato: DT-TC-REQ15-005
Técnica / prioridad	Tabla de decisión · Alta Estado: Diseñado; no ejecutado
Precondiciones	qa_cajero activo con rol CAJERO, sin permiso portal.mi_banco.ver.
Interfaz	GET http://localhost:15141/api/portal/resumen
Pasos	1. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; guardar token. 2. En psql-QA confirmar que el permiso no está asignado: SELECT count(*) FROM seguridad.permisos p WHERE p.codigo='portal.mi_banco.ver' AND p.id IN (SELECT rp.permiso_id FROM seguridad.roles_permisos rp JOIN seguridad.usuarios_roles ur ON ur.rol_id=rp.rol_id JOIN seguridad.usuarios u ON u.id=ur.usuario_id WHERE u.username='qa_cajero'); esperar 0. 3. Enviar GET http://localhost:15141/api/portal/resumen con Authorization: Bearer <token>. 4. Verificar HTTP 403 y ausencia de las claves cliente y cuentas.
Datos de entrada	{"perfil": "qa_cajero"}
Resultado esperado	HTTP 403; cuerpo sin datos de cliente ni de cuentas. qa_cajero no posee portal.mi_banco.ver.
Base / bloqueo	Criterios aprobados por el equipo QA el 06-10-2026

TC-REQ15-006 · Cliente B obtiene solo sus cuentas

Trazabilidad	REQ-15 → CP-REQ15-01 → TC-REQ15-006 Dato: DT-TC-REQ15-006
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	qa_cliente_b vinculado al cliente 0002; AH000000000003 restaurada a 1000.00; MO000000000004 con saldo de línea base 500.00.
Interfaz	GET http://localhost:15141/api/portal/cuentas
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. Autenticar qa_cliente_b con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cliente_b","password":"E2Qa2026*"}; guardar token. 3. Enviar GET http://localhost:15141/api/portal/cuentas con Authorization: Bearer <token>. 4. Verificar HTTP 200, longitud 2, numeroCuenta, saldoDisponible y estado de cada elemento; verificar que no aparece AH000000000001 ni MO000000000002.
Datos de entrada	{"perfil": "qa_cliente_b"}
Resultado esperado	HTTP 200; arreglo con exactamente 2 elementos: AH000000000003 (saldoDisponible=1000.00, ACTIVA) y MO000000000004 (saldoDisponible=500.00, ACTIVA); ninguna cuenta del cliente 0001.
Base / bloqueo	Criterios aprobados por el equipo QA el 06-10-2026

REQ-16 · Transferencia desde el portal del cliente

TC-REQ16-001 · Origen pertenece a otro cliente

Trazabilidad	REQ-16 → CP-REQ16-01 → TC-REQ16-001 Dato: DT-TC-REQ16-001
Técnica / prioridad	Tabla de decisión · Alta Estado: Diseñado; no ejecutado
Precondiciones	Token de cliente 1; cuenta origen del cliente 2 activa; saldo suficiente.
Interfaz	POST http://localhost:15141/api/portal/transferencias
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA registrar el instante de inicio: SELECT now(); 3. Autenticar qa_cliente_a con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cliente_a","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Enviar POST http://localhost:15141/api/portal/transferencias con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"numeroCuentaOrigen": "AH000000000003", "numeroCuentaDestino": "MO000000000002", "monto": 10.00}. 5. Verificar en la respuesta: HTTP 403; message="La cuenta origen no pertenece al cliente autenticado.". 6. En psql-QA ejecutar: SELECT c.numero_cuenta, c.saldo, c.saldo_disponible, c.monto_retirado_hoy, (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.created_at>=<inicio>) AS movs, (SELECT count(*) FROM contabilidad.asientos a JOIN transacciones.movimientos m ON m.id=a.movimiento_id WHERE m.cuenta_id=c.id AND a.created_at>=<inicio>) AS asientos FROM cuentas.cuentas c WHERE c.numero_cuenta IN ('AH000000000001','MO000000000002','AH000000000003') ORDER BY 1; Comprobar: todos los saldos sin cambio. 7. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_cliente_a", "numeroCuentaOrigen": "AH000000000003", "numeroCuentaDestino": "MO000000000002", "monto": "10.00"}
Resultado esperado	HTTP 403; message="La cuenta origen no pertenece al cliente autenticado."; todos los saldos sin cambio.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-04. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ16-002 · Transferencia portal válida

Trazabilidad	REQ-16 → CP-REQ16-02 → TC-REQ16-002 Dato: DT-TC-REQ16-002
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	PC-F: restaurar las cuentas implicadas a los valores de MockData_Sintetica.json; caja del cajero abierta con fondo Q2000.00; usuario qa_cajero con permisos efectivos. Ejecutar cada variante desde esa misma línea base.
Interfaz	POST http://localhost:15141/api/portal/transferencias
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA registrar el instante de inicio: SELECT now(); 3. Autenticar qa_cliente_a con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cliente_a","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Enviar POST http://localhost:15141/api/portal/transferencias con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"numeroCuentaOrigen": "AH000000000001", "numeroCuentaDestino": "MO000000000002", "monto": 100.00, "concepto": "E2 QA"}. 5. Verificar en la respuesta: HTTP 200; saldoOrigen=900, saldoDestino=600. 6. En psql-QA ejecutar: SELECT c.numero_cuenta, c.saldo, c.saldo_disponible, c.monto_retirado_hoy, (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.created_at>=<inicio>) AS movs, (SELECT count(*) FROM contabilidad.asientos a JOIN transacciones.movimientos m ON m.id=a.movimiento_id WHERE m.cuenta_id=c.id AND a.created_at>=<inicio>) AS asientos FROM cuentas.cuentas c WHERE c.numero_cuenta IN ('AH000000000001','MO000000000002','AH000000000003') ORDER BY 1; Comprobar: monto_retirado_hoy origen aumenta 100; dos movimientos y contabilidad equilibrada. 7. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_cliente_a", "numeroCuentaOrigen": "AH000000000001", "numeroCuentaDestino": "MO000000000002", "monto": "100.00", "concepto": "E2 QA"}
Resultado esperado	HTTP 200; saldoOrigen=900, saldoDestino=600; monto_retirado_hoy origen aumenta 100; dos movimientos y contabilidad equilibrada.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-04. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ16-003 · Portal excede límite diario

Trazabilidad	REQ-16 → CP-REQ16-02 → TC-REQ16-003 Dato: DT-TC-REQ16-003
Técnica / prioridad	Valores límite · Alta Estado: Diseñado; no ejecutado
Precondiciones	PC-F: restaurar las cuentas implicadas a los valores de MockData_Sintetica.json; caja del cajero abierta con fondo Q2000.00; usuario qa_cajero con permisos efectivos. Ejecutar cada variante desde esa misma línea base.
Interfaz	POST http://localhost:15141/api/portal/transferencias
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA registrar el instante de inicio: SELECT now(); 3. Autenticar qa_cliente_a con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cliente_a","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Enviar POST http://localhost:15141/api/portal/transferencias con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"numeroCuentaOrigen": "AH000000000001", "numeroCuentaDestino": "MO000000000002", "monto": 500.01, "concepto": "E2 QA"}. 5. Verificar en la respuesta: HTTP 422; message="La transferencia supera el limite diario de la cuenta origen.". 6. En psql-QA ejecutar: SELECT c.numero_cuenta, c.saldo, c.saldo_disponible, c.monto_retirado_hoy, (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.created_at>=<inicio>) AS movs, (SELECT count(*) FROM contabilidad.asientos a JOIN transacciones.movimientos m ON m.id=a.movimiento_id WHERE m.cuenta_id=c.id AND a.created_at>=<inicio>) AS asientos FROM cuentas.cuentas c WHERE c.numero_cuenta IN ('AH000000000001','MO000000000002','AH000000000003') ORDER BY 1; Comprobar: saldos y acumulador intactos. 7. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_cliente_a", "numeroCuentaOrigen": "AH000000000001", "numeroCuentaDestino": "MO000000000002", "monto": "500.01", "concepto": "E2 QA"}
Resultado esperado	HTTP 422; message="La transferencia supera el limite diario de la cuenta origen."; saldos y acumulador intactos.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-04. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

REQ-17 · Administracion de configuracion de bancos ACH

TC-REQ17-001 · Configuración ACH sin autenticación

Trazabilidad	REQ-17 → CP-REQ17-01 → TC-REQ17-001 Dato: DT-TC-REQ17-001
Técnica / prioridad	Tabla de decisión · Alta Estado: Diseñado; no ejecutado
Precondiciones	Entorno QA E2 activo; datos sintéticos cargados; autenticación y permisos del perfil indicado en la entrada.
Interfaz	GET http://localhost:15141/api/ach/bancos
Pasos	1. Enviar GET http://localhost:15141/api/ach/bancos sin encabezado Authorization. 2. Verificar en la respuesta: HTTP 401; no datos de integración. Se aplica criterio REQ-06 también a REQ-17; si HTTP 200, registrar defecto de autorización H-05.
Datos de entrada	{"Authorization": "ausente"}
Resultado esperado	HTTP 401; no datos de integración. Se aplica criterio REQ-06 también a REQ-17; si HTTP 200, registrar defecto de autorización H-05.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-05. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ17-002 · Contraseña ACH oculta y cifrada en persistencia

Trazabilidad	REQ-17 → CP-REQ17-02 → TC-REQ17-002 Dato: DT-TC-REQ17-002
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	Configurar un banco exclusivamente sintético con secreto SoloMock9* mediante creación autorizada; token qa_admin. La política D-05 exige cifrado autenticado y gestión de clave externa, aunque el código actual no lo implemente.
Interfaz	GET http://localhost:15141/api/ach/bancos
Pasos	1. Autenticar qa_admin con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_admin","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 2. Enviar POST http://localhost:15141/api/ach/bancos con Authorization: Bearer <token> y cuerpo {"nombre":"Banco QA Mock","codigo":"QA-MOCK","descripcion":"Fixture QA","activo":true,"apiBaseUrl":"http://mock-bank:8081","apiEndpoint":"/success/transferencias","httpMethod":"POST","authLoginEndpoint":null,"authEmail":"qa@example.invalid","authPassword":"SoloMock9*","headers":{},"estructuraJsonFormulario":{},"estructuraJsonPayload":{},"lookupCuentaEndpoint":null,"lookupCuentasDisponiblesEndpoint":null}; guardar el id. 3. Enviar GET http://localhost:15141/api/ach/bancos y GET http://localhost:15141/api/ach/bancos/<id> con el mismo token; verificar HTTP 200 y que ninguna respuesta contiene authPassword, auth_password ni el texto SoloMock9*. 4. En Mongo QA ejecutar: docker compose -f docker-compose.qa.yml exec -T mongo mongosh --quiet --eval "db.getSiblingDB('bancoerp_e2_qa').ach_banks_configurations.findOne({codigo:'QA-MOCK'},{auth_password:1})"; el valor no debe ser SoloMock9* ni su base64. 5. Revisar docker-compose.qa.yml y appsettings: la clave de cifrado debe provenir de una variable de entorno externa a la base y al repositorio. Si se almacena texto plano, registrar defecto H-05.
Datos de entrada	{"perfil": "qa_admin", "banco": "QA-MOCK", "authPassword": "SoloMock9*"}
Resultado esperado	Con permiso catalogos.configuracion.editar, GET /api/ach/bancos HTTP 200; no authPassword/auth_password ni valor SoloMock9* en ninguna respuesta. En Mongo el valor almacenado no coincide con el texto de entrada ni con base64 reversible sin clave; el descifrado controlado usa clave externa a base/repositorio y reproduce SoloMock9*. Si el código conserva texto plano, registrar defecto H-05.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-05. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ17-003 · Usuario autenticado sin permiso administrativo

Trazabilidad	REQ-17 → CP-REQ17-01 → TC-REQ17-003 Dato: DT-TC-REQ17-003
Técnica / prioridad	Tabla de decisión · Alta Estado: Diseñado; no ejecutado
Precondiciones	JWT qa_cliente_a válido, sin permiso catalogos.configuracion.editar; catálogo QA cargado.
Interfaz	GET http://localhost:15141/api/ach/bancos
Pasos	1. Autenticar qa_cliente_a con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cliente_a","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 2. Enviar GET http://localhost:15141/api/ach/bancos con Authorization: Bearer <token>. 3. Verificar en la respuesta: HTTP 403; ninguna configuración ni credencial de bancos en el cuerpo.
Datos de entrada	{"perfil": "qa_cliente_a"}
Resultado esperado	HTTP 403; ninguna configuración ni credencial de bancos en el cuerpo.
Base / bloqueo	Criterios aprobados por el equipo QA el 06-10-2026

REQ-18 · Envio de transferencia ACH saliente

TC-REQ18-001 · Timeout ACH a los 15 segundos, sin reintento automático

Trazabilidad	REQ-18 → CP-REQ18-01 → TC-REQ18-001 Dato: DT-TC-REQ18-001
Técnica / prioridad	Tabla de decisión · Alta Estado: Diseñado; no ejecutado
Precondiciones	Banco simulado mock-bank:8081 activo y configuraciones cargadas por configurar_banco_mock.py; qa_cliente_a dueño de cuenta AH000000000001; saldo=disponible=1000, mínimo=100, límite=500 y retirado=0. Restaurar antes de cada caso.
Interfaz	POST http://localhost:15141/api/ach/transferencias/enviar
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. Ejecutar python3 scripts/configurar_banco_mock.py; enviar POST http://localhost:18081/reset y registrar GET http://localhost:18081/stats (intentos=0). 3. Autenticar qa_cliente_a con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cliente_a","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. En psql-QA registrar el instante de inicio: SELECT now(); 5. Enviar POST http://localhost:15141/api/ach/transferencias/enviar con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"bancoAchId": "e2000000000000000000004", "cuentaOrigenId": "b5425a11-d429-58b4-906a-7820e0ad1e31", "monto": 100.00, "descripcion": "QA E2 timeout", "datosFormulario": {"numeroCuentaDestino": "EXT-QA-0001"}}; medir el tiempo con reloj monotónico. 6. Verificar en la respuesta: A los 15 s (tolerancia de observación 1 s), HTTP 504; message="Tiempo de espera agotado; requiere conciliacion."; mock registra una sola petición por código; no reintento durante 30 s adicionales. 7. En psql-QA ejecutar: SELECT estado, mensaje_error, codigo_confirmacion FROM ach.transferencias WHERE fecha_creacion>=<inicio>; y SELECT saldo FROM cuentas.cuentas WHERE numero_cuenta='AH000000000001'; Consultar GET http://localhost:18081/stats después de esperar 30 s adicionales. 8. Comprobar: transferencia persistida estado ERROR y mensaje_error contiene "requiere conciliacion"; saldo=1000 y ningún ACH_OUT. 9. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_cliente_a", "bancoAchId": "e2000000000000000000004", "cuentaOrigenId": "b5425a11-d429-58b4-906a-7820e0ad1e31", "monto": "100.00", "descripcion": "QA E2 timeout", "datosFormulario": {"numeroCuentaDestino": "EXT-QA-0001"}}
Resultado esperado	A los 15 s (tolerancia de observación 1 s), HTTP 504; message="Tiempo de espera agotado; requiere conciliacion."; transferencia persistida estado ERROR y mensaje_error contiene "requiere conciliacion"; saldo=1000 y ningún ACH_OUT; mock registra una sola petición por código; no reintento durante 30 s adicionales.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-08. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ18-002 · Código ya procesado no produce otro débito

Trazabilidad	REQ-18 → CP-REQ18-02 → TC-REQ18-002 Dato: DT-TC-REQ18-002
Técnica / prioridad	Transición de estados · Alta Estado: Diseñado; no ejecutado
Precondiciones	Banco simulado mock-bank:8081 activo y configuraciones cargadas por configurar_banco_mock.py; qa_cliente_a dueño de cuenta AH000000000001; saldo=disponible=1000, mínimo=100, límite=500 y retirado=0. Restaurar antes de cada caso. El receptor simulado /duplicate rechaza el código recibido como ya procesado.
Interfaz	POST http://localhost:15141/api/ach/transferencias/enviar
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. Ejecutar python3 scripts/configurar_banco_mock.py; enviar POST http://localhost:18081/reset y registrar GET http://localhost:18081/stats (intentos=0). 3. Autenticar qa_cliente_a con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cliente_a","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. En psql-QA registrar el instante de inicio: SELECT now(); 5. Enviar POST http://localhost:15141/api/ach/transferencias/enviar con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"bancoAchId": "e2000000000000000000005", "cuentaOrigenId": "b5425a11-d429-58b4-906a-7820e0ad1e31", "monto": 100.00, "descripcion": "QA E2 duplicate", "datosFormulario": {"numeroCuentaDestino": "EXT-QA-0001"}}; medir el tiempo con reloj monotónico. 6. Verificar en la respuesta: HTTP 409; message="La transferencia ya fue procesada."; receptor retorna 409 por el mismo código y no incrementa acreditaciones. No generar un código nuevo como mecanismo de reintento de una operación incierta. 7. En psql-QA ejecutar: SELECT estado, mensaje_error, codigo_confirmacion FROM ach.transferencias WHERE fecha_creacion>=<inicio>; y SELECT saldo FROM cuentas.cuentas WHERE numero_cuenta='AH000000000001'; Consultar GET http://localhost:18081/stats. 8. Comprobar: saldo local permanece 1000; ningún nuevo ACH_OUT. 9. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_cliente_a", "bancoAchId": "e2000000000000000000005", "cuentaOrigenId": "b5425a11-d429-58b4-906a-7820e0ad1e31", "monto": "100.00", "descripcion": "QA E2 duplicate", "datosFormulario": {"numeroCuentaDestino": "EXT-QA-0001"}}
Resultado esperado	HTTP 409; message="La transferencia ya fue procesada."; saldo local permanece 1000; ningún nuevo ACH_OUT; receptor retorna 409 por el mismo código y no incrementa acreditaciones. No generar un código nuevo como mecanismo de reintento de una operación incierta.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-08. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ18-003 · HTTP 500 del banco externo

Trazabilidad	REQ-18 → CP-REQ18-01 → TC-REQ18-003 Dato: DT-TC-REQ18-003
Técnica / prioridad	Partición de equivalencia + Tabla de decisión · Alta Estado: Diseñado; no ejecutado
Precondiciones	Banco simulado mock-bank:8081 activo y configuraciones cargadas por configurar_banco_mock.py; qa_cliente_a dueño de cuenta AH000000000001; saldo=disponible=1000, mínimo=100, límite=500 y retirado=0. Restaurar antes de cada caso.
Interfaz	POST http://localhost:15141/api/ach/transferencias/enviar
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. Ejecutar python3 scripts/configurar_banco_mock.py y POST http://localhost:18081/reset; registrar GET http://localhost:18081/stats. 3. En psql-QA registrar el instante de inicio: SELECT now(); 4. Autenticar qa_cliente_a con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cliente_a","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 5. Enviar POST http://localhost:15141/api/ach/transferencias/enviar con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"bancoAchId": "e2000000000000000000002", "cuentaOrigenId": "b5425a11-d429-58b4-906a-7820e0ad1e31", "monto": 100.00, "descripcion": "QA E2 http500", "datosFormulario": {"numeroCuentaDestino": "EXT-QA-0001"}}. 6. Verificar en la respuesta: HTTP 502; message="Banco externo no disponible."; una petición al mock; sin reintento automático. 7. En psql-QA ejecutar: SELECT estado, mensaje_error, codigo_confirmacion FROM ach.transferencias WHERE fecha_creacion>=<inicio>; SELECT saldo, monto_retirado_hoy FROM cuentas.cuentas WHERE numero_cuenta='AH000000000001'; y GET http://localhost:18081/stats. Comprobar: estado persistido ERROR; saldo=1000, acumulador=0 y ningún ACH_OUT. 8. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_cliente_a", "bancoAchId": "e2000000000000000000002", "cuentaOrigenId": "b5425a11-d429-58b4-906a-7820e0ad1e31", "monto": "100.00", "descripcion": "QA E2 http500", "datosFormulario": {"numeroCuentaDestino": "EXT-QA-0001"}}
Resultado esperado	HTTP 502; message="Banco externo no disponible."; estado persistido ERROR; saldo=1000, acumulador=0 y ningún ACH_OUT; una petición al mock; sin reintento automático.
Base / bloqueo	D-08 aprobada; código actual debe contrastarse y puede fallar este oráculo.

TC-REQ18-004 · JSON inválido del banco externo

Trazabilidad	REQ-18 → CP-REQ18-01 → TC-REQ18-004 Dato: DT-TC-REQ18-004
Técnica / prioridad	Partición de equivalencia + Tabla de decisión · Alta Estado: Diseñado; no ejecutado
Precondiciones	Banco simulado mock-bank:8081 activo y configuraciones cargadas por configurar_banco_mock.py; qa_cliente_a dueño de cuenta AH000000000001; saldo=disponible=1000, mínimo=100, límite=500 y retirado=0. Restaurar antes de cada caso.
Interfaz	POST http://localhost:15141/api/ach/transferencias/enviar
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. Ejecutar python3 scripts/configurar_banco_mock.py y POST http://localhost:18081/reset; registrar GET http://localhost:18081/stats. 3. En psql-QA registrar el instante de inicio: SELECT now(); 4. Autenticar qa_cliente_a con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cliente_a","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 5. Enviar POST http://localhost:15141/api/ach/transferencias/enviar con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"bancoAchId": "e2000000000000000000003", "cuentaOrigenId": "b5425a11-d429-58b4-906a-7820e0ad1e31", "monto": 100.00, "descripcion": "QA E2 invalid", "datosFormulario": {"numeroCuentaDestino": "EXT-QA-0001"}}. 6. Verificar en la respuesta: HTTP 502; message="Respuesta invalida del banco externo."; una petición al mock; sin reintento automático. 7. En psql-QA ejecutar: SELECT estado, mensaje_error, codigo_confirmacion FROM ach.transferencias WHERE fecha_creacion>=<inicio>; SELECT saldo, monto_retirado_hoy FROM cuentas.cuentas WHERE numero_cuenta='AH000000000001'; y GET http://localhost:18081/stats. Comprobar: estado persistido ERROR; saldo=1000, acumulador=0 y ningún ACH_OUT. 8. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_cliente_a", "bancoAchId": "e2000000000000000000003", "cuentaOrigenId": "b5425a11-d429-58b4-906a-7820e0ad1e31", "monto": "100.00", "descripcion": "QA E2 invalid", "datosFormulario": {"numeroCuentaDestino": "EXT-QA-0001"}}
Resultado esperado	HTTP 502; message="Respuesta invalida del banco externo."; estado persistido ERROR; saldo=1000, acumulador=0 y ningún ACH_OUT; una petición al mock; sin reintento automático.
Base / bloqueo	D-08 aprobada; código actual debe contrastarse y puede fallar este oráculo.

TC-REQ18-005 · Confirmación positiva válida y débito único

Trazabilidad	REQ-18 → CP-REQ18-02 → TC-REQ18-005 Dato: DT-TC-REQ18-005
Técnica / prioridad	Transición de estados · Alta Estado: Diseñado; no ejecutado
Precondiciones	Banco simulado mock-bank:8081 activo y configuraciones cargadas por configurar_banco_mock.py; qa_cliente_a dueño de cuenta AH000000000001; saldo=disponible=1000, mínimo=100, límite=500 y retirado=0. Restaurar antes de cada caso.
Interfaz	POST http://localhost:15141/api/ach/transferencias/enviar
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. Ejecutar python3 scripts/configurar_banco_mock.py y POST http://localhost:18081/reset; registrar GET http://localhost:18081/stats. 3. En psql-QA registrar el instante de inicio: SELECT now(); 4. Autenticar qa_cliente_a con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cliente_a","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 5. Enviar POST http://localhost:15141/api/ach/transferencias/enviar con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"bancoAchId": "e2000000000000000000001", "cuentaOrigenId": "b5425a11-d429-58b4-906a-7820e0ad1e31", "monto": 100.00, "descripcion": "QA E2 success", "datosFormulario": {"numeroCuentaDestino": "EXT-QA-0001"}}. 6. Verificar en la respuesta: HTTP 200; estadoNombre=ENVIADA; conservar request/response/código para conciliación. 7. En psql-QA ejecutar: SELECT estado, mensaje_error, codigo_confirmacion FROM ach.transferencias WHERE fecha_creacion>=<inicio>; SELECT saldo, monto_retirado_hoy FROM cuentas.cuentas WHERE numero_cuenta='AH000000000001'; y GET http://localhost:18081/stats. Comprobar: saldo local=900, acumulador=100; un único ACH_OUT de 100; receptor registra una acreditación de 100 para el código generado. 8. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_cliente_a", "bancoAchId": "e2000000000000000000001", "cuentaOrigenId": "b5425a11-d429-58b4-906a-7820e0ad1e31", "monto": "100.00", "descripcion": "QA E2 success", "datosFormulario": {"numeroCuentaDestino": "EXT-QA-0001"}}
Resultado esperado	HTTP 200; estadoNombre=ENVIADA; saldo local=900, acumulador=100; un único ACH_OUT de 100; receptor registra una acreditación de 100 para el código generado; conservar request/response/código para conciliación.
Base / bloqueo	D-08 aprobada; fixture de receptor success con ledger sintético.

REQ-19 · Recepcion idempotente de ACH entrante

TC-REQ19-001 · ACH entrante válido

Trazabilidad	REQ-19 → CP-REQ19-01 → TC-REQ19-001 Dato: DT-TC-REQ19-001
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	qa_externo BANCO_EXTERNO con tres permisos; cuenta destino restaurada a 1000; código inexistente.
Interfaz	POST http://localhost:15254/api/external/ach/transferencias
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO0000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA confirmar código nuevo: SELECT count(*) FROM ach.transferencias_entrantes WHERE codigo_confirmacion LIKE 'E2-TC-REQ19-001%'; esperar 0. 3. En psql-QA registrar el instante de inicio: SELECT now(); 4. Autenticar qa_externo con POST http://localhost:15254/api/external/auth/login y cuerpo {"email":"qa_externo@example.invalid","password":"E2Qa2026*"}; verificar HTTP 200 y guardar accessToken como <token>. 5. Enviar POST http://localhost:15254/api/external/ach/transferencias con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"codigo_confirmacion": "E2-TC-REQ19-001", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "100.00", "moneda": "Q", "tipo": "entrante", "referencia": "E2 sintético"} (en Postman se agrega el sufijo - {{runId}} al código). 6. Verificar en la respuesta: HTTP 201; status=true; message="Transferencia recibida correctamente."; data.estado=APROBADA; monto=100. 7. En psql-QA ejecutar: SELECT c.saldo, (SELECT count(*) FROM ach.transferencias_entrantes t WHERE t.codigo_confirmacion LIKE 'E2-TC-REQ19-001%'), (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.tipo='ACH_IN' AND m.created_at>=<inicio>) FROM cuentas.cuentas c WHERE c.numero_cuenta='AH000000000001'; Comprobar: saldo destino=1100; una transferencia, un movimiento ACH_IN y dos asientos de 100. 8. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_externo", "codigo_confirmacion": "E2-TC-REQ19-001", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "100.00", "moneda": "Q", "tipo": "entrante", "referencia": "E2 sintético"}
Resultado esperado	HTTP 201; status=true; message="Transferencia recibida correctamente."; data.estado=APROBADA; monto=100; saldo destino=1100; una transferencia, un movimiento ACH_IN y dos asientos de 100.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ19-002 · Reenvío secuencial duplicado

Trazabilidad	REQ-19 → CP-REQ19-02 → TC-REQ19-002 Dato: DT-TC-REQ19-002
Técnica / prioridad	Transición de estados · Alta Estado: Diseñado; no ejecutado
Precondiciones	Cuenta restaurada; código inexistente al inicio.
Interfaz	POST http://localhost:15254/api/external/ach/transferencias
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. Autenticar qa_externo con POST http://localhost:15254/api/external/auth/login y cuerpo {"email":"qa_externo@example.invalid","password":"E2Qa2026*"}; verificar HTTP 200 y guardar accessToken como <token>. 3. En psql-QA registrar el instante de inicio: SELECT now(); 4. Enviar POST http://localhost:15254/api/external/ach/transferencias con Authorization: Bearer <token> y cuerpo {"codigo_confirmacion": "E2-TC-REQ19-002", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "100.00", "moneda": "Q", "tipo": "entrante", "referencia": "E2 sintético"} (en Postman con sufijo -{{runId}}); verificar HTTP 201. 5. Enviar exactamente la misma petición una segunda vez. 6. Verificar en la segunda respuesta: HTTP 409; message="La transferencia ya fue procesada."; errors.codigo_confirmacion=["El codigo de confirmacion ya existe."]. 7. En psql-QA ejecutar: SELECT saldo FROM cuentas.cuentas WHERE numero_cuenta='AH000000000001'; y SELECT count(*), sum(monto) FROM ach.transferencias_entrantes WHERE codigo_confirmacion LIKE 'E2-TC-REQ19-002%'; y SELECT count(*) FROM transacciones.movimientos m JOIN cuentas.cuentas c ON c.id=m.cuenta_id WHERE c.numero_cuenta='AH000000000001' AND m.tipo='ACH_IN' AND m.created_at>=<inicio>; Esperar saldo=1100.00, una transferencia de 100.00 y un ACH_IN. 8. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_externo", "codigo_confirmacion": "E2-TC-REQ19-002", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "100.00", "moneda": "Q", "tipo": "entrante", "referencia": "E2 sintético"}
Resultado esperado	Primer envío 201; segundo 409; message="La transferencia ya fue procesada."; errors.codigo_confirmacion=["El codigo de confirmacion ya existe."]; saldo final=1100 y una sola transferencia.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ19-003 · Cinco solicitudes concurrentes con mismo código

Trazabilidad	REQ-19 → CP-REQ19-02 → TC-REQ19-003 Dato: DT-TC-REQ19-003
Técnica / prioridad	Transición de estados · Alta Estado: Diseñado; no ejecutado
Precondiciones	Cuenta restaurada; código inexistente; reloj de peticiones registrado.
Interfaz	POST http://localhost:15254/api/external/ach/transferencias
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. Autenticar qa_externo con POST http://localhost:15254/api/external/auth/login y cuerpo {"email":"qa_externo@example.invalid","password":"E2Qa2026*"}; verificar HTTP 200 y guardar accessToken como <token>. 3. En psql-QA registrar el instante de inicio: SELECT now(); 4. Lanzar 5 POST simultáneos a http://localhost:15254/api/external/ach/transferencias con Authorization: Bearer <token> y el cuerpo {"codigo_confirmacion": "E2-TC-REQ19-003", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "100.00", "moneda": "Q", "tipo": "entrante", "referencia": "E2 sintético"}, por ejemplo con Promise.all en Node.js o 5 instancias de curl en segundo plano. 5. Registrar los 5 códigos HTTP; verificar exactamente un 201 y cuatro 409. 6. En psql-QA ejecutar: SELECT saldo FROM cuentas.cuentas WHERE numero_cuenta='AH000000000001'; y SELECT count(*), sum(monto) FROM ach.transferencias_entrantes WHERE codigo_confirmacion LIKE 'E2-TC-REQ19-003%'; y SELECT count(*) FROM transacciones.movimientos m JOIN cuentas.cuentas c ON c.id=m.cuenta_id WHERE c.numero_cuenta='AH000000000001' AND m.tipo='ACH_IN' AND m.created_at>=<inicio>; Esperar saldo=1100.00, una transferencia, un ACH_IN y asientos DB=CR=100.00. 7. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_externo", "codigo_confirmacion": "E2-TC-REQ19-003", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "100.00", "moneda": "Q", "tipo": "entrante", "referencia": "E2 sintético", "concurrencia": 5}
Resultado esperado	Multiconjunto HTTP={201:1,409:4}; saldo destino=1100; exactamente una transferencia y un ACH_IN para el código; suma DB=CR=100.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ19-004 · Validación de monto ACH 0.00

Trazabilidad	REQ-19 → CP-REQ19-03 → TC-REQ19-004 Dato: DT-TC-REQ19-004
Técnica / prioridad	Valores límite · Alta Estado: Diseñado; no ejecutado
Precondiciones	Token externo válido; destino restaurado; código nuevo por caso.
Interfaz	POST http://localhost:15254/api/external/ach/transferencias
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA confirmar código nuevo: SELECT count(*) FROM ach.transferencias_entrantes WHERE codigo_confirmacion LIKE 'E2-TC-REQ19-004%'; esperar 0. 3. En psql-QA registrar el instante de inicio: SELECT now(); 4. Autenticar qa_externo con POST http://localhost:15254/api/external/auth/login y cuerpo {"email":"qa_externo@example.invalid","password":"E2Qa2026*"}; verificar HTTP 200 y guardar accessToken como <token>. 5. Enviar POST http://localhost:15254/api/external/ach/transferencias con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"codigo_confirmacion": "E2-TC-REQ19-004", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "0.00", "moneda": "Q", "tipo": "entrante", "referencia": "E2 sintético"} (en Postman se agrega el sufijo -{{runId}} al código). 6. Verificar en la respuesta: HTTP 400; status=false; message="Solicitud invalida."; errors.monto=["El monto debe ser mayor a cero."]. 7. En psql-QA ejecutar: SELECT c.saldo, (SELECT count(*) FROM ach.transferencias_entrantes t WHERE t.codigo_confirmacion LIKE 'E2-TC-REQ19-004%'), (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.tipo='ACH_IN' AND m.created_at>=<inicio>) FROM cuentas.cuentas c WHERE c.numero_cuenta='AH000000000001'; Comprobar: saldo intacto; sin transferencia ni ACH_IN. 8. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_externo", "codigo_confirmacion": "E2-TC-REQ19-004", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "0.00", "moneda": "Q", "tipo": "entrante", "referencia": "E2 sintético"}
Resultado esperado	HTTP 400; status=false; message="Solicitud invalida."; errors.monto=["El monto debe ser mayor a cero."]; saldo intacto; sin transferencia ni ACH_IN.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ19-005 · Validación de monto ACH -0.01

Trazabilidad	REQ-19 → CP-REQ19-03 → TC-REQ19-005 Dato: DT-TC-REQ19-005
Técnica / prioridad	Valores límite · Alta Estado: Diseñado; no ejecutado
Precondiciones	Token externo válido; destino restaurado; código nuevo por caso.
Interfaz	POST http://localhost:15254/api/external/ach/transferencias
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA confirmar código nuevo: SELECT count(*) FROM ach.transferencias_entrantes WHERE codigo_confirmacion LIKE 'E2-TC-REQ19-005%'; esperar 0. 3. En psql-QA registrar el instante de inicio: SELECT now(); 4. Autenticar qa_externo con POST http://localhost:15254/api/external/auth/login y cuerpo {"email":"qa_externo@example.invalid","password":"E2Qa2026*"}; verificar HTTP 200 y guardar accessToken como <token>. 5. Enviar POST http://localhost:15254/api/external/ach/transferencias con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"codigo_confirmacion": "E2-TC-REQ19-005", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "-0.01", "moneda": "Q", "tipo": "entrante", "referencia": "E2 sintético"} (en Postman se agrega el sufijo -{{runId}} al código). 6. Verificar en la respuesta: HTTP 400; status=false; message="Solicitud invalida."; errors.monto=["El monto debe ser mayor a cero."]. 7. En psql-QA ejecutar: SELECT c.saldo, (SELECT count(*) FROM ach.transferencias_entrantes t WHERE t.codigo_confirmacion LIKE 'E2-TC-REQ19-005%'), (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.tipo='ACH_IN' AND m.created_at>=<inicio>) FROM cuentas.cuentas c WHERE c.numero_cuenta='AH000000000001'; Comprobar: saldo intacto; sin transferencia ni ACH_IN. 8. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_externo", "codigo_confirmacion": "E2-TC-REQ19-005", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "-0.01", "moneda": "Q", "tipo": "entrante", "referencia": "E2 sintético"}
Resultado esperado	HTTP 400; status=false; message="Solicitud invalida."; errors.monto=["El monto debe ser mayor a cero."]; saldo intacto; sin transferencia ni ACH_IN.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ19-006 · Validación de monto ACH 0.001

Trazabilidad	REQ-19 → CP-REQ19-03 → TC-REQ19-006 Dato: DT-TC-REQ19-006
Técnica / prioridad	Valores límite · Alta Estado: Diseñado; no ejecutado
Precondiciones	Token externo válido; destino restaurado; código nuevo por caso.
Interfaz	POST http://localhost:15254/api/external/ach/transferencias
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA confirmar código nuevo: SELECT count(*) FROM ach.transferencias_entrantes WHERE codigo_confirmacion LIKE 'E2-TC-REQ19-006%'; esperar 0. 3. En psql-QA registrar el instante de inicio: SELECT now(); 4. Autenticar qa_externo con POST http://localhost:15254/api/external/auth/login y cuerpo {"email":"qa_externo@example.invalid","password":"E2Qa2026*"}; verificar HTTP 200 y guardar accessToken como <token>. 5. Enviar POST http://localhost:15254/api/external/ach/transferencias con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"codigo_confirmacion": "E2-TC-REQ19-006", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "0.001", "moneda": "Q", "tipo": "entrante", "referencia": "E2 sintético"} (en Postman se agrega el sufijo -{{runId}} al código). 6. Verificar en la respuesta: HTTP 400; status=false; message="Solicitud invalida."; errors.monto=["El monto admite como maximo 2 decimales."]. 7. En psql-QA ejecutar: SELECT c.saldo, (SELECT count(*) FROM ach.transferencias_entrantes t WHERE t.codigo_confirmacion LIKE 'E2-TC-REQ19-006%'), (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.tipo='ACH_IN' AND m.created_at>=<inicio>) FROM cuentas.cuentas c WHERE c.numero_cuenta='AH000000000001'; Comprobar: saldo intacto; sin transferencia ni ACH_IN. 8. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_externo", "codigo_confirmacion": "E2-TC-REQ19-006", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "0.001", "moneda": "Q", "tipo": "entrante", "referencia": "E2 sintético"}
Resultado esperado	HTTP 400; status=false; message="Solicitud invalida."; errors.monto=["El monto admite como maximo 2 decimales."]; saldo intacto; sin transferencia ni ACH_IN.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ19-007 · Validación de monto ACH 10000000000000000.00

Trazabilidad	REQ-19 → CP-REQ19-03 → TC-REQ19-007 Dato: DT-TC-REQ19-007
Técnica / prioridad	Valores límite · Alta Estado: Diseñado; no ejecutado
Precondiciones	Token externo válido; destino restaurado; código nuevo por caso.
Interfaz	POST http://localhost:15254/api/external/ach/transferencias
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA confirmar código nuevo: SELECT count(*) FROM ach.transferencias_entrantes WHERE codigo_confirmacion LIKE 'E2-TC-REQ19-007%'; esperar 0. 3. En psql-QA registrar el instante de inicio: SELECT now(); 4. Autenticar qa_externo con POST http://localhost:15254/api/external/auth/login y cuerpo {"email":"qa_externo@example.invalid","password":"E2Qa2026*"}; verificar HTTP 200 y guardar accessToken como <token>. 5. Enviar POST http://localhost:15254/api/external/ach/transferencias con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"codigo_confirmacion": "E2-TC-REQ19-007", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "10000000000000000.00", "moneda": "Q", "tipo": "entrante", "referencia": "E2 sintético"} (en Postman se agrega el sufijo -{{runId}} al código). 6. Verificar en la respuesta: HTTP 400; status=false; message="Solicitud invalida."; errors.monto=["El monto excede el maximo permitido."]. 7. En psql-QA ejecutar: SELECT c.saldo, (SELECT count(*) FROM ach.transferencias_entrantes t WHERE t.codigo_confirmacion LIKE 'E2-TC-REQ19-007%'), (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.tipo='ACH_IN' AND m.created_at>=<inicio>) FROM cuentas.cuentas c WHERE c.numero_cuenta='AH000000000001'; Comprobar: saldo intacto; sin transferencia ni ACH_IN. 8. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_externo", "codigo_confirmacion": "E2-TC-REQ19-007", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "10000000000000000.00", "moneda": "Q", "tipo": "entrante", "referencia": "E2 sintético"}
Resultado esperado	HTTP 400; status=false; message="Solicitud invalida."; errors.monto=["El monto excede el maximo permitido."]; saldo intacto; sin transferencia ni ACH_IN.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ19-008 · Moneda fuera de partición

Trazabilidad	REQ-19 → CP-REQ19-03 → TC-REQ19-008 Dato: DT-TC-REQ19-008
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	Entorno QA E2 activo; datos sintéticos cargados; autenticación y permisos del perfil indicado en la entrada.
Interfaz	POST http://localhost:15254/api/external/ach/transferencias
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA confirmar código nuevo: SELECT count(*) FROM ach.transferencias_entrantes WHERE codigo_confirmacion LIKE 'E2-TC-REQ19-008%'; esperar 0. 3. En psql-QA registrar el instante de inicio: SELECT now(); 4. Autenticar qa_externo con POST http://localhost:15254/api/external/auth/login y cuerpo {"email":"qa_externo@example.invalid","password":"E2Qa2026*"}; verificar HTTP 200 y guardar accessToken como <token>. 5. Enviar POST http://localhost:15254/api/external/ach/transferencias con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"codigo_confirmacion": "E2-TC-REQ19-008", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "100.00", "moneda": "USD", "tipo": "entrante", "referencia": "E2 sintético"} (en Postman se agrega el sufijo -{{runId}} al código). 6. Verificar en la respuesta: HTTP 400; errors.moneda=["La moneda soportada para ACH entrante es GTQ/Q."]. 7. En psql-QA ejecutar: SELECT c.saldo, (SELECT count(*) FROM ach.transferencias_entrantes t WHERE t.codigo_confirmacion LIKE 'E2-TC-REQ19-008%'), (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.tipo='ACH_IN' AND m.created_at>=<inicio>) FROM cuentas.cuentas c WHERE c.numero_cuenta='AH000000000001'; Comprobar: sin acreditación. 8. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_externo", "codigo_confirmacion": "E2-TC-REQ19-008", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "100.00", "moneda": "USD", "tipo": "entrante", "referencia": "E2 sintético"}
Resultado esperado	HTTP 400; errors.moneda=["La moneda soportada para ACH entrante es GTQ/Q."]; sin acreditación.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ19-009 · Combinación Pairwise PW-01

Trazabilidad	REQ-19 → CP-REQ19-04 → TC-REQ19-009 Dato: DT-TC-REQ19-009
Técnica / prioridad	Pairwise · Alta Estado: Diseñado; no ejecutado
Precondiciones	Restaurar destino a 1000 por fila; token externo válido; código de la fila inexistente.
Interfaz	POST http://localhost:15254/api/external/ach/transferencias
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA confirmar código nuevo: SELECT count(*) FROM ach.transferencias_entrantes WHERE codigo_confirmacion LIKE 'E2-PW-01%'; esperar 0. 3. En psql-QA registrar el instante de inicio: SELECT now(); 4. Autenticar qa_externo con POST http://localhost:15254/api/external/auth/login y cuerpo {"email":"qa_externo@example.invalid","password":"E2Qa2026*"}; verificar HTTP 200 y guardar accessToken como <token>. 5. Enviar POST http://localhost:15254/api/external/ach/transferencias con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"codigo_confirmacion": "E2-PW-01", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "0.01", "moneda": "Q"} (en Postman se agrega el sufijo -{{runId}} al código). 6. Verificar en la respuesta: HTTP 201; status=true; estado=APROBADA. 7. En psql-QA ejecutar: SELECT c.saldo, (SELECT count(*) FROM ach.transferencias_entrantes t WHERE t.codigo_confirmacion LIKE 'E2-PW-01%'), (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.tipo='ACH_IN' AND m.created_at>=<inicio>) FROM cuentas.cuentas c WHERE c.numero_cuenta='AH000000000001'; Comprobar: delta de saldo=0.01; una transferencia y un ACH_IN; partida doble por ese monto. 8. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_externo", "codigo_confirmacion": "E2-PW-01", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "0.01", "moneda": "Q"}
Resultado esperado	HTTP 201; status=true; estado=APROBADA; delta de saldo=0.01; una transferencia y un ACH_IN; partida doble por ese monto.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ19-010 · Combinación Pairwise PW-02

Trazabilidad	REQ-19 → CP-REQ19-04 → TC-REQ19-010 Dato: DT-TC-REQ19-010
Técnica / prioridad	Pairwise · Alta Estado: Diseñado; no ejecutado
Precondiciones	Restaurar destino a 1000 por fila; token externo válido; código de la fila inexistente.
Interfaz	POST http://localhost:15254/api/external/ach/transferencias
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA confirmar código nuevo: SELECT count(*) FROM ach.transferencias_entrantes WHERE codigo_confirmacion LIKE 'E2-PW-02%'; esperar 0. 3. En psql-QA registrar el instante de inicio: SELECT now(); 4. Autenticar qa_externo con POST http://localhost:15254/api/external/auth/login y cuerpo {"email":"qa_externo@example.invalid","password":"E2Qa2026*"}; verificar HTTP 200 y guardar accessToken como <token>. 5. Enviar POST http://localhost:15254/api/external/ach/transferencias con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"codigo_confirmacion": "E2-PW-02", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "100.00", "moneda": "Q", "tipo": "entrante", "referencia": "E2 combinatorio"} (en Postman se agrega el sufijo -{{runId}} al código). 6. Verificar en la respuesta: HTTP 201; status=true; estado=APROBADA. 7. En psql-QA ejecutar: SELECT c.saldo, (SELECT count(*) FROM ach.transferencias_entrantes t WHERE t.codigo_confirmacion LIKE 'E2-PW-02%'), (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.tipo='ACH_IN' AND m.created_at>=<inicio>) FROM cuentas.cuentas c WHERE c.numero_cuenta='AH000000000001'; Comprobar: delta de saldo=100.00; una transferencia y un ACH_IN; partida doble por ese monto. 8. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_externo", "codigo_confirmacion": "E2-PW-02", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "100.00", "moneda": "Q", "tipo": "entrante", "referencia": "E2 combinatorio"}
Resultado esperado	HTTP 201; status=true; estado=APROBADA; delta de saldo=100.00; una transferencia y un ACH_IN; partida doble por ese monto.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ19-011 · Combinación Pairwise PW-03

Trazabilidad	REQ-19 → CP-REQ19-04 → TC-REQ19-011 Dato: DT-TC-REQ19-011
Técnica / prioridad	Pairwise · Alta Estado: Diseñado; no ejecutado
Precondiciones	Restaurar destino a 1000 por fila; token externo válido; código de la fila inexistente.
Interfaz	POST http://localhost:15254/api/external/ach/transferencias
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA confirmar código nuevo: SELECT count(*) FROM ach.transferencias_entrantes WHERE codigo_confirmacion LIKE 'E2-PW-03%'; esperar 0. 3. En psql-QA registrar el instante de inicio: SELECT now(); 4. Autenticar qa_externo con POST http://localhost:15254/api/external/auth/login y cuerpo {"email":"qa_externo@example.invalid","password":"E2Qa2026*"}; verificar HTTP 200 y guardar accessToken como <token>. 5. Enviar POST http://localhost:15254/api/external/ach/transferencias con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"codigo_confirmacion": "E2-PW-03", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "100.00", "moneda": "GTQ"} (en Postman se agrega el sufijo -{{runId}} al código). 6. Verificar en la respuesta: HTTP 201; status=true; estado=APROBADA. 7. En psql-QA ejecutar: SELECT c.saldo, (SELECT count(*) FROM ach.transferencias_entrantes t WHERE t.codigo_confirmacion LIKE 'E2-PW-03%'), (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.tipo='ACH_IN' AND m.created_at>=<inicio>) FROM cuentas.cuentas c WHERE c.numero_cuenta='AH000000000001'; Comprobar: delta de saldo=100.00; una transferencia y un ACH_IN; partida doble por ese monto. 8. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_externo", "codigo_confirmacion": "E2-PW-03", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "100.00", "moneda": "GTQ"}
Resultado esperado	HTTP 201; status=true; estado=APROBADA; delta de saldo=100.00; una transferencia y un ACH_IN; partida doble por ese monto.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ19-012 · Combinación Pairwise PW-04

Trazabilidad	REQ-19 → CP-REQ19-04 → TC-REQ19-012 Dato: DT-TC-REQ19-012
Técnica / prioridad	Pairwise · Alta Estado: Diseñado; no ejecutado
Precondiciones	Restaurar destino a 1000 por fila; token externo válido; código de la fila inexistente.
Interfaz	POST http://localhost:15254/api/external/ach/transferencias
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA confirmar código nuevo: SELECT count(*) FROM ach.transferencias_entrantes WHERE codigo_confirmacion LIKE 'E2-PW-04%'; esperar 0. 3. En psql-QA registrar el instante de inicio: SELECT now(); 4. Autenticar qa_externo con POST http://localhost:15254/api/external/auth/login y cuerpo {"email":"qa_externo@example.invalid","password":"E2Qa2026*"}; verificar HTTP 200 y guardar accessToken como <token>. 5. Enviar POST http://localhost:15254/api/external/ach/transferencias con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"codigo_confirmacion": "E2-PW-04", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "0.01", "moneda": "GTQ", "tipo": "entrante", "referencia": "E2 combinatorio"} (en Postman se agrega el sufijo -{{runId}} al código). 6. Verificar en la respuesta: HTTP 201; status=true; estado=APROBADA. 7. En psql-QA ejecutar: SELECT c.saldo, (SELECT count(*) FROM ach.transferencias_entrantes t WHERE t.codigo_confirmacion LIKE 'E2-PW-04%'), (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.tipo='ACH_IN' AND m.created_at>=<inicio>) FROM cuentas.cuentas c WHERE c.numero_cuenta='AH000000000001'; Comprobar: delta de saldo=0.01; una transferencia y un ACH_IN; partida doble por ese monto. 8. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_externo", "codigo_confirmacion": "E2-PW-04", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "0.01", "moneda": "GTQ", "tipo": "entrante", "referencia": "E2 combinatorio"}
Resultado esperado	HTTP 201; status=true; estado=APROBADA; delta de saldo=0.01; una transferencia y un ACH_IN; partida doble por ese monto.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ19-013 · Combinación Pairwise PW-05

Trazabilidad	REQ-19 → CP-REQ19-04 → TC-REQ19-013 Dato: DT-TC-REQ19-013
Técnica / prioridad	Pairwise · Alta Estado: Diseñado; no ejecutado
Precondiciones	Restaurar destino a 1000 por fila; token externo válido; código de la fila inexistente.
Interfaz	POST http://localhost:15254/api/external/ach/transferencias
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA confirmar código nuevo: SELECT count(*) FROM ach.transferencias_entrantes WHERE codigo_confirmacion LIKE 'E2-PW-05%'; esperar 0. 3. En psql-QA registrar el instante de inicio: SELECT now(); 4. Autenticar qa_externo con POST http://localhost:15254/api/external/auth/login y cuerpo {"email":"qa_externo@example.invalid","password":"E2Qa2026*"}; verificar HTTP 200 y guardar accessToken como <token>. 5. Enviar POST http://localhost:15254/api/external/ach/transferencias con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"codigo_confirmacion": "E2-PW-05", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "0.01", "moneda": "Q", "referencia": "E2 combinatorio"} (en Postman se agrega el sufijo -{{runId}} al código). 6. Verificar en la respuesta: HTTP 201; status=true; estado=APROBADA. 7. En psql-QA ejecutar: SELECT c.saldo, (SELECT count(*) FROM ach.transferencias_entrantes t WHERE t.codigo_confirmacion LIKE 'E2-PW-05%'), (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.tipo='ACH_IN' AND m.created_at>=<inicio>) FROM cuentas.cuentas c WHERE c.numero_cuenta='AH000000000001'; Comprobar: delta de saldo=0.01; una transferencia y un ACH_IN; partida doble por ese monto. 8. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_externo", "codigo_confirmacion": "E2-PW-05", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "0.01", "moneda": "Q", "referencia": "E2 combinatorio"}
Resultado esperado	HTTP 201; status=true; estado=APROBADA; delta de saldo=0.01; una transferencia y un ACH_IN; partida doble por ese monto.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ19-014 · Combinación Pairwise PW-06

Trazabilidad	REQ-19 → CP-REQ19-04 → TC-REQ19-014 Dato: DT-TC-REQ19-014
Técnica / prioridad	Pairwise · Alta Estado: Diseñado; no ejecutado
Precondiciones	Restaurar destino a 1000 por fila; token externo válido; código de la fila inexistente.
Interfaz	POST http://localhost:15254/api/external/ach/transferencias
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO0000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA confirmar código nuevo: SELECT count(*) FROM ach.transferencias_entrantes WHERE codigo_confirmacion LIKE 'E2-PW-06%'; esperar 0. 3. En psql-QA registrar el instante de inicio: SELECT now(); 4. Autenticar qa_externo con POST http://localhost:15254/api/external/auth/login y cuerpo {"email":"qa_externo@example.invalid","password":"E2Qa2026*"}; verificar HTTP 200 y guardar accessToken como <token>. 5. Enviar POST http://localhost:15254/api/external/ach/transferencias con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"codigo_confirmacion": "E2-PW-06", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "0.01", "moneda": "Q", "tipo": "entrante"} (en Postman se agrega el sufijo -{{runId}} al código). 6. Verificar en la respuesta: HTTP 201; status=true; estado=APROBADA. 7. En psql-QA ejecutar: SELECT c.saldo, (SELECT count(*) FROM ach.transferencias_entrantes t WHERE t.codigo_confirmacion LIKE 'E2-PW-06%'), (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.tipo='ACH_IN' AND m.created_at>=<inicio>) FROM cuentas.cuentas c WHERE c.numero_cuenta='AH000000000001'; Comprobar: delta de saldo=0.01; una transferencia y un ACH_IN; partida doble por ese monto. 8. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_externo", "codigo_confirmacion": "E2-PW-06", "banco_externo": "QA-MOCK", "cuenta_externa": "EXT-QA-0001", "numeroCuentaDestino": "AH000000000001", "monto": "0.01", "moneda": "Q", "tipo": "entrante"}
Resultado esperado	HTTP 201; status=true; estado=APROBADA; delta de saldo=0.01; una transferencia y un ACH_IN; partida doble por ese monto.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

REQ-20 · Autenticacion de banco externo con rol BANCO_EXTERNO

TC-REQ20-001 · Login externo perfil externo

Trazabilidad	REQ-20 → CP-REQ20-01 → TC-REQ20-001 Dato: DT-TC-REQ20-001
Técnica / prioridad	Tabla de decisión · Alta Estado: Diseñado; no ejecutado
Precondiciones	Usuario sintético con rol/estado descrito en MockData; sin token.
Interfaz	POST http://localhost:15254/api/external/auth/login
Pasos	1. En psql-QA confirmar el perfil: SELECT u.estado, r.codigo FROM seguridad.usuarios u LEFT JOIN seguridad.usuarios_roles ur ON ur.usuario_id=u.id LEFT JOIN seguridad.roles r ON r.id=ur.rol_id WHERE u.email='qa_externo@example.invalid'; 2. En psql-QA registrar el instante de inicio: SELECT now(); 3. Enviar POST http://localhost:15254/api/external/auth/login con Content-Type: application/json, sin encabezado Authorization y cuerpo {"email": "qa_externo@example.invalid", "password": "E2Qa2026*"}. 4. Verificar en la respuesta: HTTP 200; accessToken no vacío, tokenType=Bearer, expiresAt UTC futuro. 5. En psql-QA ejecutar: SELECT count(*) FROM seguridad.sesiones s JOIN seguridad.usuarios u ON u.id=s.usuario_id WHERE u.email='qa_externo@example.invalid' AND s.estado AND s.fecha_inicio>=<inicio>; Comprobar: sesión activa del banco.
Datos de entrada	{"email": "qa_externo@example.invalid", "password": "E2Qa2026*"}
Resultado esperado	HTTP 200; accessToken no vacío, tokenType=Bearer, expiresAt UTC futuro; sesión activa del banco.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ20-002 · Login externo perfil cajero

Trazabilidad	REQ-20 → CP-REQ20-01 → TC-REQ20-002 Dato: DT-TC-REQ20-002
Técnica / prioridad	Tabla de decisión · Alta Estado: Diseñado; no ejecutado
Precondiciones	Usuario sintético con rol/estado descrito en MockData; sin token.
Interfaz	POST http://localhost:15254/api/external/auth/login
Pasos	1. En psql-QA confirmar el perfil: SELECT u.estado, r.codigo FROM seguridad.usuarios u LEFT JOIN seguridad.usuarios_roles ur ON ur.usuario_id=u.id LEFT JOIN seguridad.roles r ON r.id=ur.rol_id WHERE u.email='qa_cajero@example.invalid'; 2. En psql-QA registrar el instante de inicio: SELECT now(); 3. Enviar POST http://localhost:15254/api/external/auth/login con Content-Type: application/json, sin encabezado Authorization y cuerpo {"email": "qa_cajero@example.invalid", "password": "E2Qa2026*"}. 4. Verificar en la respuesta: HTTP 403; no accessToken. 5. En psql-QA ejecutar: SELECT count(*) FROM seguridad.sesiones s JOIN seguridad.usuarios u ON u.id=s.usuario_id WHERE u.email='qa_cajero@example.invalid' AND s.estado AND s.fecha_inicio>=<inicio>; Comprobar: no sesión externa nueva.
Datos de entrada	{"email": "qa_cajero@example.invalid", "password": "E2Qa2026*"}
Resultado esperado	HTTP 403; no accessToken; no sesión externa nueva.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ20-003 · Login externo perfil inactivo

Trazabilidad	REQ-20 → CP-REQ20-01 → TC-REQ20-003 Dato: DT-TC-REQ20-003
Técnica / prioridad	Tabla de decisión · Alta Estado: Diseñado; no ejecutado
Precondiciones	Usuario sintético con rol/estado descrito en MockData; sin token.
Interfaz	POST http://localhost:15254/api/external/auth/login
Pasos	1. En psql-QA confirmar el perfil: SELECT u.estado, r.codigo FROM seguridad.usuarios u LEFT JOIN seguridad.usuarios_roles ur ON ur.usuario_id=u.id LEFT JOIN seguridad.roles r ON r.id=ur.rol_id WHERE u.email='qa_inactivo@example.invalid'; 2. En psql-QA registrar el instante de inicio: SELECT now(); 3. Enviar POST http://localhost:15254/api/external/auth/login con Content-Type: application/json, sin encabezado Authorization y cuerpo {"email": "qa_inactivo@example.invalid", "password": "E2Qa2026*"}. 4. Verificar en la respuesta: HTTP 403; no accessToken. 5. En psql-QA ejecutar: SELECT count(*) FROM seguridad.sesiones s JOIN seguridad.usuarios u ON u.id=s.usuario_id WHERE u.email='qa_inactivo@example.invalid' AND s.estado AND s.fecha_inicio>=<inicio>; Comprobar: no sesión externa nueva.
Datos de entrada	{"email": "qa_inactivo@example.invalid", "password": "E2Qa2026*"}
Resultado esperado	HTTP 403; no accessToken; no sesión externa nueva.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ20-004 · Consulta ACH sin JWT

Trazabilidad	REQ-20 → CP-REQ20-02 → TC-REQ20-004 Dato: DT-TC-REQ20-004
Técnica / prioridad	Tabla de decisión · Alta Estado: Diseñado; no ejecutado
Precondiciones	Entorno QA E2 activo; datos sintéticos cargados; autenticación y permisos del perfil indicado en la entrada.
Interfaz	GET http://localhost:15254/api/external/ach/cuentas/search/AH000000000001
Pasos	1. Enviar GET http://localhost:15254/api/external/ach/cuentas/search/AH000000000001 sin encabezado Authorization. 2. Verificar en la respuesta: HTTP 401; ninguna información de cuenta.
Datos de entrada	{"Authorization": "ausente", "numeroCuenta": "AH000000000001"}
Resultado esperado	HTTP 401; ninguna información de cuenta.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

REQ-21 · Generacion automatica de asientos contables

TC-REQ21-001 · Partida doble de depósito exitoso

Trazabilidad	REQ-21 → CP-REQ21-01 → TC-REQ21-001 Dato: DT-TC-REQ21-001
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	PC-F: restaurar las cuentas implicadas a los valores de MockData_Sintetica.json; caja del cajero abierta con fondo Q2000.00; usuario qa_cajero con permisos efectivos. Ejecutar cada variante desde esa misma línea base. Plan contable 1101 y cuenta de pasivo del tipo activos; verificar solo movimiento nuevo, excluyendo histórico mock.
Interfaz	POST http://localhost:15141/api/transacciones/deposito
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; 3. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 4. Abrir caja con POST http://localhost:15141/api/cajas/abrir, Bearer de qa_cajero y cuerpo {"fondoInicial": 2000.00}; verificar HTTP 200. 5. En psql-QA registrar el instante de inicio: SELECT now(); 6. Enviar POST http://localhost:15141/api/transacciones/deposito con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"numeroCuenta": "AH000000000001", "monto": 100.00}. 7. Verificar en la respuesta: HTTP 200; SUM(DB)-SUM(CR)=0. 8. En psql-QA ejecutar: SELECT c.numero_cuenta, c.saldo, c.saldo_disponible, c.monto_retirado_hoy, (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.created_at>=<inicio>) AS movs, (SELECT count(*) FROM contabilidad.asientos a JOIN transacciones.movimientos m ON m.id=a.movimiento_id WHERE m.cuenta_id=c.id AND a.created_at>=<inicio>) AS asientos FROM cuentas.cuentas c WHERE c.numero_cuenta IN ('AH000000000001','MO000000000002','AH000000000003') ORDER BY 1; Comprobar: un movimiento nuevo; exactamente dos asientos ligados a ese movimiento: DB 100 y CR 100; saldo final 1100. 9. Restaurar la línea base con python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_cajero", "numeroCuenta": "AH000000000001", "monto": "100.00"}
Resultado esperado	HTTP 200; un movimiento nuevo; exactamente dos asientos ligados a ese movimiento: DB 100 y CR 100; SUM(DB)-SUM(CR)=0; saldo final 1100.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ21-002 · Rollback por falta de cuenta contable

Trazabilidad	REQ-21 → CP-REQ21-02 → TC-REQ21-002 Dato: DT-TC-REQ21-002
Técnica / prioridad	Tabla de decisión · Alta Estado: Diseñado; no ejecutado
Precondiciones	PC-F: restaurar las cuentas implicadas a los valores de MockData_Sintetica.json; caja del cajero abierta con fondo Q2000.00; usuario qa_cajero con permisos efectivos. Ejecutar cada variante desde esa misma línea base. Desactivar 1101 temporalmente solo en QA; reactivar al terminar.
Interfaz	POST http://localhost:15141/api/transacciones/deposito
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA ajustar la precondición: UPDATE contabilidad.plan_cuentas SET activo=false WHERE codigo='1101'; 3. En psql-QA cerrar cajas abiertas de qa_cajero: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; 4. Autenticar qa_cajero con POST http://localhost:15141/api/auth/login y cuerpo {"username":"qa_cajero","password":"E2Qa2026*"}; verificar HTTP 200 y guardar token como <token>. 5. Abrir caja con POST http://localhost:15141/api/cajas/abrir, Bearer de qa_cajero y cuerpo {"fondoInicial": 2000.00}; verificar HTTP 200. 6. En psql-QA registrar el instante de inicio: SELECT now(); 7. Enviar POST http://localhost:15141/api/transacciones/deposito con Content-Type: application/json, Authorization: Bearer <token> y cuerpo {"numeroCuenta": "AH000000000001", "monto": 100.00}. 8. Verificar en la respuesta: HTTP 422; saldo/disponible quedan 1000; excepción identifica falta de cuenta contable activa 1101. 9. En psql-QA ejecutar: SELECT c.numero_cuenta, c.saldo, c.saldo_disponible, c.monto_retirado_hoy, (SELECT count(*) FROM transacciones.movimientos m WHERE m.cuenta_id=c.id AND m.created_at>=<inicio>) AS movs, (SELECT count(*) FROM contabilidad.asientos a JOIN transacciones.movimientos m ON m.id=a.movimiento_id WHERE m.cuenta_id=c.id AND a.created_at>=<inicio>) AS asientos FROM cuentas.cuentas c WHERE c.numero_cuenta IN ('AH000000000001','MO000000000002','AH000000000003') ORDER BY 1; Comprobar: conteo de movimientos, asientos y auditoría EXITOSO de la operación no aumenta. 10. Restaurar: UPDATE contabilidad.plan_cuentas SET activo=true WHERE codigo='1101'; y ejecutar python3 scripts/preparar_escenario.py.
Datos de entrada	{"perfil": "qa_cajero", "numeroCuenta": "AH000000000001", "monto": "100.00", "maestro_inactivo": "1101"}
Resultado esperado	HTTP 422; saldo/disponible quedan 1000; conteo de movimientos, asientos y auditoría EXITOSO de la operación no aumenta; excepción identifica falta de cuenta contable activa 1101.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ21-003 · Partida doble de retiro en cuenta AHORRO

Trazabilidad	REQ-21 → CP-REQ21-01 → TC-REQ21-003 Dato: DT-TC-REQ21-003
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	PC-F: cuentas restauradas; caja de qa_cajero ABIERTA con fondo Q2000.00; plan contable 1101 y 2101 activo.
Interfaz	POST http://localhost:15141/api/transacciones/retiro
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA cerrar cajas previas: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; autenticar qa_cajero (POST http://localhost:15141/api/auth/login con {"username":"qa_cajero","password":"E2Qa2026*"}) y abrir caja con POST http://localhost:15141/api/cajas/abrir y cuerpo {"fondoInicial": 2000.00} (HTTP 200). 3. Enviar POST http://localhost:15141/api/transacciones/retiro con Authorization: Bearer <token>, Content-Type: application/json y cuerpo {"numeroCuenta": "AH000000000001", "monto": 100.00}; guardar la referencia devuelta. 4. Verificar HTTP 200, saldoNuevo=900.00 y referencia no vacía. 5. En psql-QA ejecutar: SELECT pc.codigo, a.tipo_asiento, a.monto, a.sp_origen FROM contabilidad.asientos a JOIN contabilidad.plan_cuentas pc ON pc.id=a.cuenta_contable_id JOIN transacciones.movimientos m ON m.id=a.movimiento_id WHERE m.referencia='<referencia>' ORDER BY a.tipo_asiento; Esperar dos filas: 1101/CR/100.00/sp_retiro y 2101/DB/100.00/sp_retiro. 6. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO000000000002: saldo 500.00; AH000000000003: saldo 1000.00).
Datos de entrada	{"perfil": "qa_cajero", "numeroCuenta": "AH000000000001", "monto": "100.00"}
Resultado esperado	HTTP 200; saldoNuevo=900.00; referencia no vacía; un movimiento RETIRO de 100.00; exactamente dos asientos con sp_origen="sp_retiro": DB 100.00 en 2101 (Depositos de ahorro) y CR 100.00 en 1101 (Caja general); SUM(DB)=SUM(CR)=100.00.
Base / bloqueo	Criterios aprobados por el equipo QA el 06-10-2026

TC-REQ21-004 · Partida doble de depósito en cuenta MONETARIA

Trazabilidad	REQ-21 → CP-REQ21-01 → TC-REQ21-004 Dato: DT-TC-REQ21-004
Técnica / prioridad	Partición de equivalencia · Alta Estado: Diseñado; no ejecutado
Precondiciones	PC-F: MO0000000000002 restaurada a 500.00; caja de qa_cajero ABIERTA con fondo Q2000.00; plan contable 1101 y 2102 activo.
Interfaz	POST http://localhost:15141/api/transacciones/deposito
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO0000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA cerrar cajas previas: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; autenticar qa_cajero (POST http://localhost:15141/api/auth/login con {"username":"qa_cajero","password":"E2Qa2026*"}) y abrir caja con POST http://localhost:15141/api/cajas/abrir y cuerpo {"fondoInicial": 2000.00} (HTTP 200). 3. Enviar POST http://localhost:15141/api/transacciones/deposito con Authorization: Bearer <token>, Content-Type: application/json y cuerpo {"numeroCuenta": "MO0000000000002", "monto": 50.00}; guardar la referencia. 4. Verificar HTTP 200 y saldoNuevo=550.00. 5. En psql-QA ejecutar: SELECT pc.codigo, a.tipo_asiento, a.monto, a.sp_origen FROM contabilidad.asientos a JOIN contabilidad.plan_cuentas pc ON pc.id=a.cuenta_contable_id JOIN transacciones.movimientos m ON m.id=a.movimiento_id WHERE m.referencia='<referencia>' ORDER BY a.tipo_asiento; Esperar dos filas: 1101/DB/50.00/sp_deposito y 2102/CR/50.00/sp_deposito. 6. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO0000000000002: saldo 500.00; AH000000000003: saldo 1000.00).
Datos de entrada	{"perfil": "qa_cajero", "numeroCuenta": "MO0000000000002", "monto": "50.00"}
Resultado esperado	HTTP 200; saldoNuevo=550.00; un movimiento DEPOSITO de 50.00; exactamente dos asientos con sp_origen="sp_deposito": DB 50.00 en 1101 (Caja general) y CR 50.00 en 2102 (Depositos monetarios); ningún asiento en 2101.
Base / bloqueo	Criterios aprobados por el equipo QA el 06-10-2026

TC-REQ21-005 · Asientos de transferencia interna entre AHORRO y MONETARIA

Trazabilidad	REQ-21 → CP-REQ21-02 → TC-REQ21-005 Dato: DT-TC-REQ21-005
Técnica / prioridad	Tabla de decisión · Alta Estado: Diseñado; no ejecutado
Precondiciones	PC-F: cuentas restauradas; caja de qa_cajero ABIERTA con Q2000.00.
Interfaz	POST http://localhost:15141/api/transacciones/transferencia
Pasos	1. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO0000000000002: saldo 500.00; AH000000000003: saldo 1000.00). 2. En psql-QA cerrar cajas previas: UPDATE cajas.sesiones_caja SET estado='CERRADA', hora_cierre=now() WHERE cajero_id=(SELECT e.id FROM rrhh.empleados e JOIN seguridad.usuarios u ON u.id=e.usuario_id WHERE u.username='qa_cajero') AND estado='ABIERTA'; autenticar qa_cajero (POST http://localhost:15141/api/auth/login con {"username":"qa_cajero","password":"E2Qa2026*"}) y abrir caja con POST http://localhost:15141/api/cajas/abrir y cuerpo {"fondoInicial": 2000.00} (HTTP 200). 3. Enviar POST http://localhost:15141/api/transacciones/transferencia con Authorization: Bearer <token> y cuerpo {"numeroCuentaOrigen": "AH000000000001", "numeroCuentaDestino": "MO0000000000002", "monto": 100.00, "concepto": "E2 QA asientos"}; guardar la referencia. 4. Verificar HTTP 200, saldoOrigen=900.00 y saldoDestino=600.00. 5. En psql-QA ejecutar: SELECT pc.codigo, a.tipo_asiento, sum(a.monto) FROM contabilidad.asientos a JOIN contabilidad.plan_cuentas pc ON pc.id=a.cuenta_contable_id JOIN transacciones.movimientos m ON m.id=a.movimiento_id WHERE m.referencia='<referencia>' OR m.referencia_par='<referencia>' GROUP BY 1,2; esperar 2101/DB/100.00 y 2102/CR/100.00. 6. Ejecutar python3 scripts/preparar_escenario.py (AH000000000001: saldo=disponible=1000.00, mínimo 100.00, límite 500.00, retirado 0; MO0000000000002: saldo 500.00; AH000000000003: saldo 1000.00).
Datos de entrada	{"perfil": "qa_cajero", "numeroCuentaOrigen": "AH000000000001", "numeroCuentaDestino": "MO0000000000002", "monto": "100.00", "concepto": "E2 QA asientos"}
Resultado esperado	HTTP 200; saldoOrigen=900.00; saldoDestino=600.00; los movimientos de la transferencia tienen asientos balanceados: DB 100.00 en 2101 (pasivo de la cuenta AHORRO origen) y CR 100.00 en 2102 (pasivo de la cuenta MONETARIA destino); SUM(DB)=SUM(CR)=100.00. Cero asientos constituye defecto contra REQ-21.
Base / bloqueo	REQ-21 de E1 (partida doble de cada transacción). Análisis de fuente: sp_transferir_fondos no inserta en contabilidad.asientos; si la consulta devuelve 0 filas se registra defecto.

REQ-22 · Restriccion de origenes mediante CORS

TC-REQ22-001 · Preflight desde http://localhost:13000

Trazabilidad	REQ-22 → CP-REQ22-01 → TC-REQ22-001 Dato: DT-TC-REQ22-001
Técnica / prioridad	Partición de equivalencia · Media Estado: Diseñado; no ejecutado
Precondiciones	Entorno QA E2 activo; datos sintéticos cargados; autenticación y permisos del perfil indicado en la entrada.
Interfaz	OPTIONS http://localhost:15141/api/clientes
Pasos	1. Enviar OPTIONS http://localhost:15141/api/clientes con encabezados Origin: http://localhost:13000 y Access-Control-Request-Method: GET (sin Authorization). 2. Registrar el código HTTP y los encabezados Access-Control-*. 3. Verificar HTTP 204 y Access-Control-Allow-Origin=http://localhost:13000.
Datos de entrada	{"Origin": "http://localhost:13000", "Access-Control-Request-Method": "GET"}
Resultado esperado	HTTP 204; Access-Control-Allow-Origin=http://localhost:13000.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ22-002 · Preflight desde http://origen-no-permitido.invalid

Trazabilidad	REQ-22 → CP-REQ22-01 → TC-REQ22-002 Dato: DT-TC-REQ22-002
Técnica / prioridad	Partición de equivalencia · Media Estado: Diseñado; no ejecutado
Precondiciones	Entorno QA E2 activo; datos sintéticos cargados; autenticación y permisos del perfil indicado en la entrada.
Interfaz	OPTIONS http://localhost:15141/api/clientes
Pasos	1. Enviar OPTIONS http://localhost:15141/api/clientes con encabezados Origin: http://origen-no-permitido.invalid y Access-Control-Request-Method: GET (sin Authorization). 2. Registrar el código HTTP y los encabezados Access-Control-*. 3. Verificar HTTP 204 y que no existe Access-Control-Allow-Origin; el navegador bloquea la lectura (CORS no exige 403 en el servidor).
Datos de entrada	{"Origin": "http://origen-no-permitido.invalid", "Access-Control-Request-Method": "GET"}
Resultado esperado	HTTP 204; ausencia de Access-Control-Allow-Origin; el navegador bloquea lectura. CORS no exige 403 del servidor.
Base / bloqueo	Contrato técnico vigente; cotejar con Entrega 1

TC-REQ22-003 · Preflight POST con encabezados desde origen permitido

Trazabilidad	REQ-22 → CP-REQ22-01 → TC-REQ22-003 Dato: DT-TC-REQ22-003
Técnica / prioridad	Partición de equivalencia · Media Estado: Diseñado; no ejecutado
Precondiciones	CORS_ALLOWED_ORIGINS del entorno QA = http://localhost:13000.
Interfaz	OPTIONS http://localhost:15141/api/transacciones/deposito
Pasos	1. Enviar OPTIONS http://localhost:15141/api/transacciones/deposito con encabezados Origin: http://localhost:13000, Access-Control-Request-Method: POST y Access-Control-Request-Headers: content-type,authorization (sin Authorization). 2. Registrar el código HTTP y todos los encabezados Access-Control-* de la respuesta. 3. Verificar HTTP 204, Access-Control-Allow-Origin=http://localhost:13000, Access-Control-Allow-Methods=POST y Access-Control-Allow-Headers=content-type,authorization.
Datos de entrada	{"Origin": "http://localhost:13000", "Access-Control-Request-Method": "POST", "Access-Control-Request-Headers": "content-type,authorization"}
Resultado esperado	HTTP 204; Access-Control-Allow-Origin=http://localhost:13000; Access-Control-Allow-Methods=POST; Access-Control-Allow-Headers=content-type,authorization.
Base / bloqueo	Criterios aprobados por el equipo QA el 06-10-2026

TC-REQ22-004 · Origen con puerto vecino 13001

Trazabilidad	REQ-22 → CP-REQ22-02 → TC-REQ22-004 Dato: DT-TC-REQ22-004
Técnica / prioridad	Valores límite · Media Estado: Diseñado; no ejecutado
Precondiciones	CORS_ALLOWED_ORIGINS del entorno QA = http://localhost:13000; el puerto 13001 es el vecino inmediato del origen permitido.
Interfaz	OPTIONS http://localhost:15141/api/transacciones/deposito
Pasos	1. Enviar OPTIONS http://localhost:15141/api/transacciones/deposito con encabezados Origin: http://localhost:13001, Access-Control-Request-Method: POST y Access-Control-Request-Headers: content-type,authorization (sin Authorization). 2. Registrar el código HTTP y todos los encabezados Access-Control-* de la respuesta. 3. Verificar HTTP 204 y que la respuesta NO contiene Access-Control-Allow-Origin, Access-Control-Allow-Methods ni Access-Control-Allow-Headers.
Datos de entrada	{"Origin": "http://localhost:13001", "Access-Control-Request-Method": "POST", "Access-Control-Request-Headers": "content-type,authorization"}
Resultado esperado	HTTP 204; sin Access-Control-Allow-Origin, Access-Control-Allow-Methods ni Access-Control-Allow-Headers; el navegador bloquea la petición real.
Base / bloqueo	Criterios aprobados por el equipo QA el 06-10-2026

TC-REQ22-005 · Origen que contiene el permitido como prefijo

Trazabilidad	REQ-22 → CP-REQ22-02 → TC-REQ22-005 Dato: DT-TC-REQ22-005
Técnica / prioridad	Partición de equivalencia · Media Estado: Diseñado; no ejecutado
Precondiciones	CORS_ALLOWED_ORIGINS del entorno QA = http://localhost:13000; dominio .invalid reservado, sin resolución DNS.
Interfaz	OPTIONS http://localhost:15141/api/transacciones/deposito
Pasos	1. Enviar OPTIONS http://localhost:15141/api/transacciones/deposito con encabezados Origin: http://localhost:13000.evil.invalid, Access-Control-Request-Method: POST y Access-Control-Request-Headers: content-type,authorization (sin Authorization). 2. Registrar el código HTTP y todos los encabezados Access-Control-* de la respuesta. 3. Verificar HTTP 204 y que la respuesta NO contiene Access-Control-Allow-Origin, Access-Control-Allow-Methods ni Access-Control-Allow-Headers.
Datos de entrada	{"Origin": "http://localhost:13000.evil.invalid", "Access-Control-Request-Method": "POST", "Access-Control-Request-Headers": "content-type,authorization"}
Resultado esperado	HTTP 204; sin Access-Control-Allow-Origin, Access-Control-Allow-Methods ni Access-Control-Allow-Headers; la comparación de origen es exacta, no por prefijo.
Base / bloqueo	Criterios aprobados por el equipo QA el 06-10-2026

REQ-23 · Despliegue reproducible con contenedores e integracion continua

TC-REQ23-001 · Arranque aislado reproducible

Trazabilidad	REQ-23 → CP-REQ23-01 → TC-REQ23-001 Dato: DT-TC-REQ23-001
Técnica / prioridad	Transición de estados · Media Estado: Diseñado; no ejecutado
Precondiciones	Entorno QA E2 activo; datos sintéticos cargados; autenticación y permisos del perfil indicado en la entrada.
Pasos	1. Desde la raíz del repositorio ejecutar en orden: python3 ENTREGA_02_DISENO_E_IMPLEMENTACION_QA/03_Evidencias_y_MockData/scripts/preparar.py, disenar.py y coleccion.py. 2. Ejecutar docker compose -f ENTREGA_02_DISENO_E_IMPLEMENTACION_QA/03_Evidencias_y_MockData/docker-compose.qa.yml up -d y esperar a que postgres reporte (healthy) en docker compose ps. 3. Ejecutar python3 .../scripts/configurar_banco_mock.py y python3 .../scripts/readiness.py. 4. Verificar en Readiness_Resultados.json: dictamen APTO CON RESTRICCIONES, RD-01 con 6 servicios running, RD-02 a RD-04 HTTP 200 y RD-05 con usuarios=8, clientes=1000, cuentas=2000, movimientos=10000.
Datos de entrada	{"compose": "docker-compose.qa.yml", "proyecto": "bancoerp-e2-qa", "puertos": [13000, 15141, 15254, 15433, 17017]}
Resultado esperado	Cinco servicios activos; postgres healthy; frontend y Swagger HTTP 200; pg/mongo accesibles; conteos iniciales 8/1000/2000/10000; sin conexiones remotas configuradas.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-06. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ23-002 · Compuerta local bloquea vulnerabilidad Alta

Trazabilidad	REQ-23 → CP-REQ23-02 → TC-REQ23-002 Dato: DT-TC-REQ23-002
Técnica / prioridad	Tabla de decisión · Media Estado: Diseñado; no ejecutado
Precondiciones	Fixture de vulnerabilidad completamente sintética incluido; Python disponible. La verificación local no afirma una corrida CI remota.
Pasos	1. Desde 03_Evidencias_y_MockData ejecutar: python3 scripts/compuerta_vulnerabilidades.py Vulnerabilidades_QA_Alta.json; echo "exit=$?". 2. Verificar en la salida JSON: decision="RECHAZADO", altas=1, criticas=0; y exit=1. 3. Revisar .github/workflows/ci.yml: si no invoca la compuerta, registrar defecto H-09 (la liberación no queda bloqueada en CI).
Datos de entrada	{"fixture": "Vulnerabilidades_QA_Alta.json", "comando": "python3 scripts/compuerta_vulnerabilidades.py Vulnerabilidades_QA_Alta.json"}
Resultado esperado	Proceso termina con exit code=1 y JSON decision=RECHAZADO; altas=1 y criticas=0. En el pipeline la etapa de liberación debe quedar bloqueada; el workflow actual carece de integración de esta compuerta, lo que se registra como defecto H-09.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-06. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ23-003 · Compuerta de severidad CRITICA

Trazabilidad	REQ-23 → CP-REQ23-02 → TC-REQ23-003 Dato: DT-TC-REQ23-003
Técnica / prioridad	Tabla de decisión · Media Estado: Diseñado; no ejecutado
Precondiciones	Script local compuerta_vulnerabilidades.py y fixture incluido; no se ejecuta pipeline remoto.
Pasos	1. Desde 03_Evidencias_y_MockData ejecutar: python3 scripts/compuerta_vulnerabilidades.py Vulnerabilidades_QA_Critica.json; echo "exit=$?". 2. Verificar en la salida JSON: decision="RECHAZADO", criticas=1; y exit=1. 3. Guardar la salida como evidencia local.
Datos de entrada	{"fixture": "Vulnerabilidades_QA_Critica.json", "severidad": "CRITICA"}
Resultado esperado	Exit code=1; JSON decision=RECHAZADO; un hallazgo sintético CRITICA; liberación bloqueada.
Base / bloqueo	Criterios aprobados por el equipo QA el 06-10-2026

TC-REQ23-004 · Compuerta de severidad MEDIA

Trazabilidad	REQ-23 → CP-REQ23-02 → TC-REQ23-004 Dato: DT-TC-REQ23-004
Técnica / prioridad	Tabla de decisión · Media Estado: Diseñado; no ejecutado
Precondiciones	Script local compuerta_vulnerabilidades.py y fixture incluido; no se ejecuta pipeline remoto.
Pasos	1. Desde 03_Evidencias_y_MockData ejecutar: python3 scripts/compuerta_vulnerabilidades.py Vulnerabilidades_QA_Media.json; echo "exit=$?". 2. Verificar en la salida JSON: decision="APROBADO_CON_SEGUIMIENTO", medias=1; y exit=0. 3. Guardar la salida como evidencia local.
Datos de entrada	{"fixture": "Vulnerabilidades_QA_Media.json", "severidad": "MEDIA"}
Resultado esperado	Exit code=0; JSON decision=APROBADO_CON_SEGUIMIENTO; un hallazgo sintético MEDIA; no bloquea liberación; MEDIA se conserva en seguimiento.
Base / bloqueo	Criterios aprobados por el equipo QA el 06-10-2026

TC-REQ23-005 · Compuerta de severidad BAJA

Trazabilidad	REQ-23 → CP-REQ23-02 → TC-REQ23-005 Dato: DT-TC-REQ23-005
Técnica / prioridad	Tabla de decisión · Media Estado: Diseñado; no ejecutado
Precondiciones	Script local compuerta_vulnerabilidades.py y fixture incluido; no se ejecuta pipeline remoto.
Pasos	1. Desde 03_Evidencias_y_MockData ejecutar: python3 scripts/compuerta_vulnerabilidades.py Vulnerabilidades_QA_Baja.json; echo "exit=$?". 2. Verificar en la salida JSON: decision="APROBADO", bajas=1; y exit=0. 3. Guardar la salida como evidencia local.
Datos de entrada	{"fixture": "Vulnerabilidades_QA_Baja.json", "severidad": "BAJA"}
Resultado esperado	Exit code=0; JSON decision=APROBADO; un hallazgo sintético BAJA; no bloquea liberación; MEDIA se conserva en seguimiento.
Base / bloqueo	Criterios aprobados por el equipo QA el 06-10-2026

REQ-24 · Interfaz usable y responsive con modo claro y oscuro

TC-REQ24-001 · Interfaz 1366px tema claro

Trazabilidad	REQ-24 → CP-REQ24-01 → TC-REQ24-001 Dato: DT-TC-REQ24-001
Técnica / prioridad	Partición de equivalencia · Baja Estado: Diseñado; no ejecutado
Precondiciones	Entorno QA E2 activo; datos sintéticos cargados; autenticación y permisos del perfil indicado en la entrada.
Interfaz	GET http://localhost:13000/login
Pasos	1. Abrir Google Chrome en http://localhost:13000/login; en DevTools > Toggle device toolbar fijar el viewport en 1366x768. 2. Seleccionar el tema claro con el botón de tema (title "Cambiar a modo claro"); verificar en la consola localStorage.getItem("nexus-theme-preference") = "light". 3. En la consola ejecutar document.documentElement.scrollWidth <= window.innerWidth; esperar true. Verificar que los campos de usuario, contraseña y el botón de ingreso están completos dentro del viewport. 4. Iniciar sesión con qa_cajero / E2Qa2026*; en el dashboard abrir el menú principal (en 390 px, el menú móvil) y navegar a Clientes y Cuentas. 5. En cada pantalla repetir la comprobación de scrollWidth, verificar que ningún control se superpone y que el tema seleccionado se mantiene; guardar captura de login y dashboard.
Datos de entrada	{"viewport": "1366x768", "tema": "claro", "ruta": "/login"}
Resultado esperado	HTTP 200; login visible; controles dentro del viewport; sin superposición ni scroll horizontal (scrollWidth<=innerWidth). Tras login qa_cajero, dashboard y navegación principal cumplen las mismas condiciones con tema claro.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-07. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ24-002 · Interfaz 390px tema oscuro

Trazabilidad	REQ-24 → CP-REQ24-01 → TC-REQ24-002 Dato: DT-TC-REQ24-002
Técnica / prioridad	Partición de equivalencia · Baja Estado: Diseñado; no ejecutado
Precondiciones	Entorno QA E2 activo; datos sintéticos cargados; autenticación y permisos del perfil indicado en la entrada.
Interfaz	GET http://localhost:13000/login
Pasos	1. Abrir Google Chrome en http://localhost:13000/login; en DevTools > Toggle device toolbar fijar el viewport en 390x844. 2. Seleccionar el tema oscuro con el botón de tema (title "Cambiar a modo oscuro"); verificar en la consola localStorage.getItem("nexus-theme-preference") = "dark". 3. En la consola ejecutar document.documentElement.scrollWidth <= window.innerWidth; esperar true. Verificar que los campos de usuario, contraseña y el botón de ingreso están completos dentro del viewport. 4. Iniciar sesión con qa_cajero / E2Qa2026*; en el dashboard abrir el menú principal (en 390 px, el menú móvil) y navegar a Clientes y Cuentas. 5. En cada pantalla repetir la comprobación de scrollWidth, verificar que ningún control se superpone y que el tema seleccionado se mantiene; guardar captura de login y dashboard.
Datos de entrada	{"viewport": "390x844", "tema": "oscuro", "ruta": "/login"}
Resultado esperado	HTTP 200; login visible; controles dentro del viewport; sin superposición ni scroll horizontal (scrollWidth<=innerWidth). Tras login qa_cajero, dashboard y navegación principal cumplen las mismas condiciones con tema oscuro.
Base / bloqueo	Criterio vigente aprobado por el equipo QA (Líder QA) el 06-10-2026: D-07. Comparar contra software actual; una diferencia es defecto, no bloqueo de diseño.

TC-REQ24-003 · Login y navegación 1366x768 tema oscuro

Trazabilidad	REQ-24 → CP-REQ24-01 → TC-REQ24-003 Dato: DT-TC-REQ24-003
Técnica / prioridad	Tabla de decisión · Baja Estado: Diseñado; no ejecutado
Precondiciones	Entorno E2 QA activo, usuario y fixture sintéticos restaurados; registrar conteos previos.
Interfaz	GET http://localhost:13000/login
Pasos	1. Abrir Google Chrome en http://localhost:13000/login; en DevTools > Toggle device toolbar fijar el viewport en 1366x768. 2. Seleccionar el tema oscuro con el botón de tema (title "Cambiar a modo oscuro"); verificar en la consola localStorage.getItem("nexus-theme-preference") = "dark". 3. En la consola ejecutar document.documentElement.scrollWidth <= window.innerWidth; esperar true. Verificar que los campos de usuario, contraseña y el botón de ingreso están completos dentro del viewport. 4. Iniciar sesión con qa_cajero / E2Qa2026*; en el dashboard abrir el menú principal (en 390 px, el menú móvil) y navegar a Clientes y Cuentas. 5. En cada pantalla repetir la comprobación de scrollWidth, verificar que ningún control se superpone y que el tema seleccionado se mantiene; guardar captura de login y dashboard.
Datos de entrada	{"viewport": "1366x768", "tema": "oscuro", "perfil": "qa_cajero", "ruta": "/login"}
Resultado esperado	HTTP 200; login y dashboard visibles; navegación principal operable; controles dentro del viewport, sin superposiciones; scrollWidth<=innerWidth en ambas pantallas; tema seleccionado persiste al navegar.
Base / bloqueo	Criterios aprobados por el equipo QA el 06-10-2026

TC-REQ24-004 · Login y navegación 390x844 tema claro

Trazabilidad	REQ-24 → CP-REQ24-01 → TC-REQ24-004 Dato: DT-TC-REQ24-004
Técnica / prioridad	Tabla de decisión · Baja Estado: Diseñado; no ejecutado
Precondiciones	Entorno E2 QA activo, usuario y fixture sintéticos restaurados; registrar conteos previos.
Interfaz	GET http://localhost:13000/login
Pasos	1. Abrir Google Chrome en http://localhost:13000/login; en DevTools > Toggle device toolbar fijar el viewport en 390x844. 2. Seleccionar el tema claro con el botón de tema (title "Cambiar a modo claro"); verificar en la consola localStorage.getItem("nexus-theme-preference") = "light". 3. En la consola ejecutar document.documentElement.scrollWidth <= window.innerWidth; esperar true. Verificar que los campos de usuario, contraseña y el botón de ingreso están completos dentro del viewport. 4. Iniciar sesión con qa_cajero / E2Qa2026*; en el dashboard abrir el menú principal (en 390 px, el menú móvil) y navegar a Clientes y Cuentas. 5. En cada pantalla repetir la comprobación de scrollWidth, verificar que ningún control se superpone y que el tema seleccionado se mantiene; guardar captura de login y dashboard.
Datos de entrada	{"viewport": "390x844", "tema": "claro", "perfil": "qa_cajero", "ruta": "/login"}
Resultado esperado	HTTP 200; login y dashboard visibles; navegación principal operable; controles dentro del viewport, sin superposiciones; scrollWidth<=innerWidth en ambas pantallas; tema seleccionado persiste al navegar.
Base / bloqueo	Criterios aprobados por el equipo QA el 06-10-2026

4. Test Data Matrix y Mock Data

4.1 Estructura, volumen y origen

Toda la información de negocio se genera desde cero mediante UUIDv5 con semilla 8d8c1be6-7b83-4e00-b7a1-6cae2f1a2c02. Los nombres declaran “Persona Sintetica QA”, los correos usan example.invalid y los documentos son identificadores de prueba: no certifican identidades ni se usan en trámites externos. No se importaron bases ni registros de producción. Los scripts de esquema se derivaron del repositorio excluyendo el seed de usuario externo de la migración 013.

Entidad / archivo	Cantidad	Campos y restricciones
Usuarios JSON/SQL	8	UUID, username QA único, email reservado, rol, estado y contraseña exclusiva QA; hash bcrypt generado por pgcrypto al importar.
Clientes JSON/SQL	1,000	UUID, tipo NATURAL, nombre ficticio, documento sintético único, fecha fija, estado y enlace usuario para clientes A/B.
Cuentas JSON/CSV/SQL	2,000	Dos por cliente: AHORRO y MONETARIA; FK cliente; saldo/disponible exactos; mínimo y límite; 1 bloqueada y 1 cerrada.
Movimientos JSON/SQL	10,000	10 históricos por cliente sobre su cuenta AH; referencia única; fechas ordenables; no se afirma que constituyan una contabilidad inicial auditada.
Data Matrix Excel	139 filas	Una por caso; ID_Dato único, caso/REQ, categoría, estructura exacta, fixture y oráculo. Mismos datos visibles en cada ficha de §3.7.
Paginación ampliada	110 adicionales bajo demanda	preparar_escenario.py --paginacion lleva la cuenta AH000000000001 de 10 a 120 registros iniciales para probar tamaños 99/100/101. No se ejecuta antes del conteo de readiness.

4.2 Fixtures principales

Conjunto	Clase	Datos exactos / preparación	Uso
DT-LOGIN	Válido	qa_cajero / E2Qa2026*	REQ-01, 05, 08, 10–14, 21
DT-BLOQUEO	Inválido / límite	qa_bloqueo; contador 3/4; error Erronea9*; N aprobado=5; bloqueo de 30 min	REQ-02, 03, 04, 07
DT-ORIGEN	Válido / límite	AH000000000001: saldo 1000, disponible 1000, mínimo 100, límite 500, retirado 0	REQ-10, 11, 12, 16, 19, 21
DT-DESTINO	Válido	MO000000000002: saldo 500; cliente 1	REQ-12, 16
DT-AJENO	Inválido / acceso	AH000000000003: cliente 2; usuario qa_cliente_a corresponde al cliente 1	REQ-15, 16
DT-CLIENTE-INACTIVO	Inválido	Cliente 1000, estado INACTIVO; UUID en JSON	REQ-14
DT-EXTERNO	Válido	qa_externo@example.invalid / E2Qa2026*; BANCO_EXTERNO con tres permisos	REQ-19, 20
DT-PASSWORD	Límites / clases	7, 8, 9 caracteres; sin mayúscula, minúscula o dígito; historial reciente	REQ-03
DT-PAGINACIÓN	Límites	120 movimientos; pagina=0/1/2; tamano=0/1/99/100/101	REQ-13
DT-CORS	Válido / inválido / límite	http://localhost:13000; http://localhost:13001; http://localhost:13000.evil.invalid; http://origen-no-permitido.invalid	REQ-22
DT-REFRESH	Inválido	"" (vacío); E2-QA-TOKEN-INEXISTENTE-0001; R1 tras logout	REQ-05

Los DT-TC-… son los identificadores canónicos por caso. Los conjuntos DT-LOGIN y similares agrupan fixtures para facilitar lectura; no reemplazan la fila individual de Data_Matrix. Cada UUID puede resolverse por el número de cuenta o por la clave lógica del usuario en MockData_Sintetica.json.

4.3 Consistencia, limpieza y manejo de datos

El generador fija IDs, referencias, correos y volúmenes. Las FK cliente→cuenta y cuenta→movimiento se verifican; los montos se almacenan como cadenas exactas en JSON/CSV y como numeric en PostgreSQL. La carga SQL rechaza cualquier base distinta de bancoerp_e2_qa y usa ON CONFLICT para no duplicar IDs. Es idempotente para cargar registros faltantes; no restablece saldos ni estados modificados por pruebas. Para ello se usa preparar_escenario.py y ajustes específicos de cada precondición.

No se reutiliza un código de confirmación entre casos. Postman agrega runId al código; los reenvíos del mismo caso conservan exactamente el mismo valor. Cambiar runId entre corridas evita falsos 409, pero no evita la necesidad de restaurar saldos. Los usuarios destinados a bloqueo y recuperación están separados del cajero y del banco externo.

Las contraseñas publicadas son exclusivas de este entorno sintético. Los JWT y refresh tokens producidos durante la preparación se enmascaran. La red, base y volúmenes QA son propios; no se configuran destinos de producción. No existe una política de firewall de egreso y no se afirma aislamiento de red absoluto.

5. Informe técnico de alistamiento del entorno

5.1 Resultado, fecha y límites del dictamen

Fecha de comprobación: 2026-10-06T18:42:29-06:00 (UTC−6). Responsable del dictamen: Líder QA. Dictamen: APTO CON RESTRICCIONES. Los 15 controles de preparación resultaron APTO; además, las 12 comprobaciones del mock y la compuerta local resultaron OK. El dictamen habilita preparar/ejecutar escenarios con oráculo y fixtures listos; no certifica seguridad integral ni la aceptación del producto.

Restricciones de aceptación del software: no se ejecutó la suite funcional completa, pipeline remoto, carga ni TLS. El banco simulado, el contrato saliente y el inventario de eventos ya están preparados. Las reglas aprobadas pueden diferir del código actual; esa diferencia constituye defecto y no información pendiente. La consulta anónima al catálogo ACH sigue respondiendo 200 (OBS-01, vinculada a H-05), por lo que la aceptación integral de REQ-06/17 debe bloquearse y su corrección priorizarse.

Resumen de alistamiento según el Formato 2 de la guía. Los responsables corresponden a los roles QA definidos en la matriz RACI de la Entrega 1; la evidencia está en 03_Evidencias_y_MockData/Capturas_Readiness_Entorno salvo que se indique otra carpeta.

Componente	Configuración / URL	Estado de verificación	Evidencia adjunta	Responsable
Servidor Web/App (Frontend)	http://localhost:13000/login	APTO · HTTP 200	01_Frontend_QA.png; RD-02	Tester Funcional
Internal API	http://localhost:15141/swagger	APTO · Swagger HTTP 200; login 200; sin token 401	02_Swagger_Interna_QA.png; OpenAPI_Interna.json; RD-03/08/11	Tester Funcional
Public API	http://localhost:15254/swagger	APTO · Swagger HTTP 200; login externo 200; cuenta ACTIVA	03_Swagger_Publica_QA.png; OpenAPI_Publica.json; RD-04/09/10	Tester de Seguridad e Integración
Base de Datos	PostgreSQL 16 · 127.0.0.1:15433 / bancoerp_e2_qa	APTO · Base poblada con Mock Data: 8 usuarios, 1000 clientes, 2000 cuentas, 10000 movimientos	Readiness_Resultados.json; 03_Evidencias_y_MockData/MockData_Sintetica.sql	Analista de Pruebas
Base documental	MongoDB 8 · 127.0.0.1:17017 / bancoerp_e2_qa	APTO · ping ok; 5 bancos simulados	Readiness_Log.txt; scripts/seed_banco_mock.js	Analista de Pruebas
Aislamiento de producción	Red, volúmenes y base exclusivos; puertos 127.0.0.1	APTO · host postgres interno, base QA, ASPNETCORE_ENVIRONMENT=QA, código montado solo lectura	Readiness_Resultados.json (RD-06); docker-compose.qa.yml	Tester de Seguridad e Integración
CORS	Origen permitido http://localhost:13000	APTO · permitido con Allow-Origin; no permitido sin cabecera	Readiness_Resultados.json (RD-12/13)	Tester de Seguridad e Integración
Banco externo simulado	http://localhost:18081/health	APTO · 12/12 controles del mock y compuerta OK	05_Mock_y_Compuerta.png; Mock_y_Compuerta_Resultados.json	Tester de Seguridad e Integración
Colección API	Postman v2.1 · 139 peticiones	APTO · importable; huella SHA-256 registrada	03_Evidencias_y_MockData/Postman_Collection_QA.json y Postman_Environment_QA.json	Tester Funcional
Orquestación	docker compose · proyecto bancoerp-e2-qa	APTO · 6 servicios running; postgres healthy	Readiness_Log.txt; 04_Readiness_Observado.png	Líder QA

5.2 Arquitectura QA y aislamiento comprobado

Componente	URL / destino	Aislamiento
Frontend React	http://localhost:13000/login	Publicación 127.0.0.1:13000; proxy hacia internal-api QA.
Internal API	http://localhost:15141/swagger	ASPNETCORE_ENVIRONMENT=QA; BD postgres / bancoerp_e2_qa.

Componente	URL / destino	Aislamiento
Public API	http://localhost:15254/swagger	Issuer/audience BancoERP-E2-QA; usuario externo sintético.
PostgreSQL 16	127.0.0.1:15433 → postgres:5432	Base bancoerp_e2_qa; volumen bancoerp-e2-qa_e2-pg; datos desde generación propia.
MongoDB 8	127.0.0.1:17017 → mongo:27017	BD bancoerp_e2_qa; volumen propio; sin configuraciones de bancos reales.
Banco externo simulado	http://localhost:18081/health	Seis servicios en total; mock-bank:8081 en red QA; cinco escenarios locales sin conexiones externas.
Contenedores / red	Proyecto bancoerp-e2-qa	Red bancoerp-e2-qa_default; APIs con montaje de código solo lectura; servicios de desarrollo existentes permanecen aparte.

La comprobación RD-06 inspecciona las variables de conexión efectivas sin publicar contraseñas; confirma host interno postgres, base QA y ambiente QA para ambas APIs. El alcance del aislamiento probado es de configuración, persistencia y publicación local. La conectividad saliente a producción no se ensayó ni quedó bloqueada por firewall.

5.3 Checklist observado

ID	Verificación	Resultado / evidencia	Responsable
RD-01	Seis servicios QA activos	APTO · Detalle completo en Readiness_Resultados.json. Evidencia: Readiness_Log.txt; 04_Readiness_Observado.png	Líder QA
RD-02	Accesibilidad http://localhost:13000/login	APTO · {"url": "http://localhost:13000/login", "HTTP": 200, "content_type": "text/html", "bytes": 612} Evidencia: 01_Frontend_QA.png	Tester Funcional
RD-03	Accesibilidad http://localhost:15141/swagger/v1/swagger.json	APTO · {"url": "http://localhost:15141/swagger/v1/swagger.json", "HTTP": 200, "content_type": "application/json;charset=utf-8", "bytes": 81379, "rutas": 72} Evidencia: 02_Swagger_Interna_QA.png; OpenAPI_Interna.json	Tester Funcional
RD-04	Accesibilidad http://localhost:15254/swagger/v1/swagger.json	APTO · {"url": "http://localhost:15254/swagger/v1/swagger.json", "HTTP": 200, "content_type": "application/json;charset=utf-8", "bytes": 14329, "rutas": 4} Evidencia: 03_Swagger_Publica_QA.png; OpenAPI_Publica.json	Tester de Seguridad e Integración
RD-05	Base QA y datos masivos cargados	APTO · {"database": "bancoerp_e2_qa", "usuarios": 8, "clientes": 1000, "cuentas": 2000, "movimientos": 10000, "plan_activo": 2} Evidencia: Readiness_Resultados.json (conteos); MockData_Sintetica.sql	Analista de Pruebas
RD-06	Aislamiento: destinos internos, red y montajes	APTO · Detalle completo en Readiness_Resultados.json. Evidencia: Readiness_Resultados.json (RD-06); docker-compose.qa.yml	Tester de Seguridad e Integración
RD-07	MongoDB QA accesible	APTO · {"database": "bancoerp_e2_qa", "host": "mongo", "uso": "Configuración ACH; cinco bancos simulados locales, sin bancos reales"} Evidencia: Readiness_Log.txt	Analista de Pruebas
RD-08	Login de preparación interno	APTO · {"HTTP": 200, "token_presente": true, "token": "[ENMASCARADO]", "usuario": "qa_cajero"} Evidencia: Readiness_Resultados.json (token enmascarado)	Tester Funcional
RD-09	Login de preparación externo	APTO · {"HTTP": 200, "token_presente": true, "token": "[ENMASCARADO]", "usuario": "qa_externo@example.invalid"} Evidencia: Readiness_Resultados.json (token enmascarado)	Tester de Seguridad e Integración
RD-10	Cuenta destino sintética consultable	APTO · Detalle completo en Readiness_Resultados.json. Evidencia: Readiness_Resultados.json	Analista de Pruebas
RD-11	Control de autenticación representativo	APTO · {"HTTP": 401, "ruta": "/api/clientes", "token": "ausente"} Evidencia: Readiness_Resultados.json	Tester de Seguridad e Integración
RD-12	CORS http://localhost:13000	APTO · {"HTTP": 204, "Origin": "http://localhost:13000", "allow_origin": "http://localhost:13000"} Evidencia: Readiness_Resultados.json	Tester de Seguridad e Integración
RD-13	CORS http://origen-no-permitido.invalid	APTO · {"HTTP": 204, "Origin": "http://origen-no-permitido.invalid", "allow_origin": null} Evidencia: Readiness_Resultados.json	Tester de Seguridad e Integración

ID	Verificación	Resultado / evidencia	Responsable
RD-15	Banco simulado QA accesible	APTO · {"HTTP": 200, "response": {"status": "READY", "synthetic": true, "scenarios": ["success", "http500", "invalid", "timeout", "duplicate"], "timeoutDelaySeconds": 20}} Evidencia: 05_Mock_y_Compuerta.png; Mock_y_Compuerta_Resultados.json	Tester de Seguridad e Integración
RD-14	Colección Postman disponible	APTO · {"archivo": "Postman_Collection_QA.json", "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json", "sha256": "c00ec04250517e83ccbd332dad0681bc914f8c953faa0730df8c951a63a4c334"} Evidencia: SHA-256 en Readiness_Resultados.json	Tester Funcional

5.4 Banco simulado, compuerta y diferencias de implementación

Banco simulado: success confirma una vez y guarda ledger por código; duplicate responde 409; http500 devuelve 500; invalid devuelve HTTP 200 con JSON mal formado; timeout demora 20 s para exceder el contrato de 15 s. /cuentas/search valida una cuenta sintética; /stats permite comprobar intentos y acreditaciones; /reset limpia solo el ledger del fixture. Configuraciones Mongo locales se generan con configurar_banco_mock.py.

mock_smoke.py comprobó ocho propiedades del fixture y cuatro decisiones de severidad de la compuerta local: 12/12 OK. El test de timeout del fixture usa un cliente de 0.5 s para demostrar demora; no afirma que BancoERP ya implemente su timeout de 15 s. compuerta_vulnerabilidades.py rechaza ALTA/CRITICA, conserva MEDIA en seguimiento y acepta BAJA. El workflow CI existente aún requiere integrar esta regla.

Control	Resultado real
MOCK-01	OK · {"status": "READY", "synthetic": true, "scenarios": ["success", "http500", "invalid", "timeout", "duplicate"], "timeoutDelaySeconds": 20}
MOCK-02	OK · {"status": true, "message": "Cuenta sintetica encontrada.", "data": {"numeroCuenta": "EXT-QA-0001", "estado": true, "moneda": "GTQ", "tipoCuenta": "AHORRO", "titular": {"nombre": "Titular QA Sintetico"}}}
MOCK-03	OK · {"HTTP": 201, "response": {"status": true, "message": "Transferencia sintetica aprobada.", "data": {"estado": "APROBADA", "codigo_confirmacion": "E2-SMOKE-0001", "monto": "100.00"}}}
MOCK-04	OK · {"HTTP": 409, "response": {"status": false, "message": "La transferencia ya fue procesada.", "codigo_confirmacion": "E2-SMOKE-0001"}}
MOCK-05	OK · {"HTTP": 500, "response": {"status": false, "message": "Fallo sintetico de banco externo."}}
MOCK-06	OK · {"HTTP": 200, "cuerpo_JSON_invalido": true}
MOCK-07	OK · {"timeout_cliente_fixture_segundos": 0.5, "demora_mock_segundos": 20, "observado": 0.501, "nota": "Verifica demora del fixture, no el timeout de 15 s del producto."}
MOCK-08	OK · {"processed": ["E2-SMOKE-0001"], "attempts": {"E2-SMOKE-0001": 2, "E2-SMOKE-0002": 1, "E2-SMOKE-0003": 1, "E2-SMOKE-0004": 1}}
GATE-ALTA	OK · {"exit_code": 1, "decision": "RECHAZADO", "criticas": 0, "altas": 1, "medias": 0, "bajas": 0, "seguimiento": []}
GATE-CRITICA	OK · {"exit_code": 1, "decision": "RECHAZADO", "criticas": 1, "altas": 0, "medias": 0, "bajas": 0, "seguimiento": []}
GATE-MEDIA	OK · {"exit_code": 0, "decision": "APROBADO_CON_SEGUIMIENTO", "criticas": 0, "altas": 0, "medias": 1, "bajas": 0, "seguimiento": [{"id": "QA-HALLAZGO-MEDIA", "severidad": "MEDIA", "componente": "componente-sintetico"}]}
GATE-BAJA	OK · {"exit_code": 0, "decision": "APROBADO", "criticas": 0, "altas": 0, "medias": 0, "bajas": 1, "seguimiento": []}

Diferencias verificadas por análisis de fuente, no por ejecución de la nueva suite: sp_transferir_fondos carece de límite diario (H-03); los controladores de catálogo ACH carecen de autorización y el DTO incluye AuthPassword (H-05); EnviarAsync no fija timeout de 15 s y devuelve estados de error mediante un resultado HTTP 200; el débito precede a la persistencia del JSON de respuesta. El workflow no tiene compuerta de vulnerabilidades (H-09). Deben corregirse y verificarse en una fase posterior; cerrar definiciones no las corrige.

5.5 Evidencias visuales

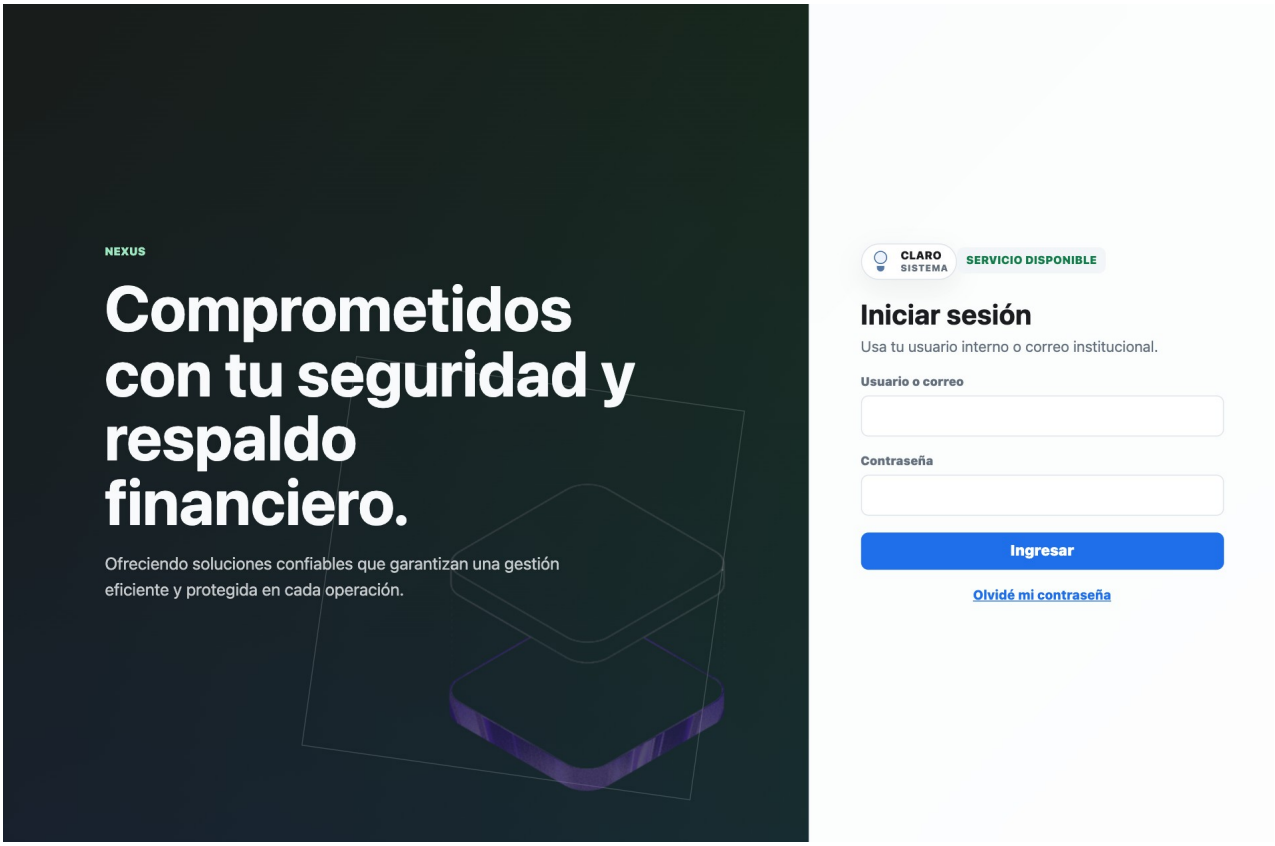


Figura 1. Pantalla de login del frontend QA en localhost:13000.

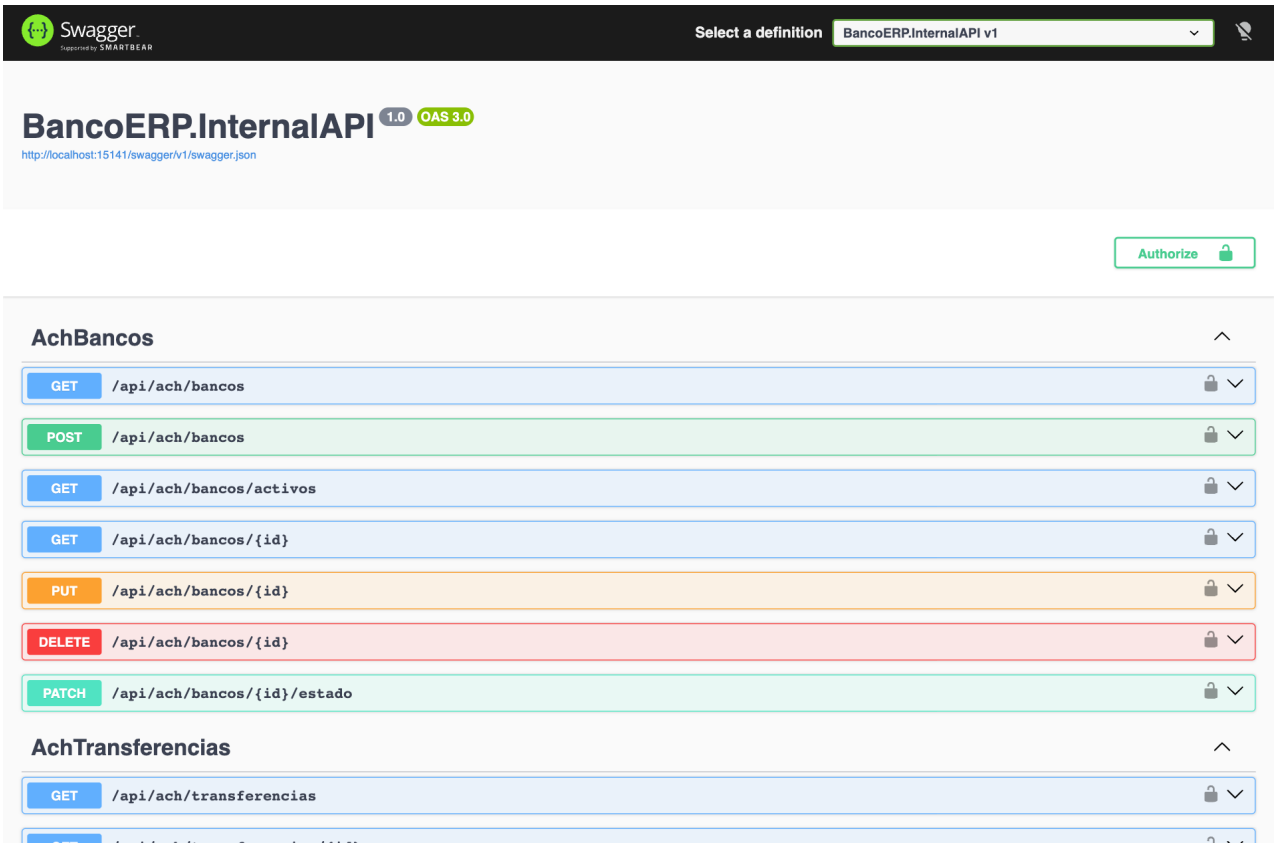


Figura 2. Swagger real de Internal API QA en localhost:15141.

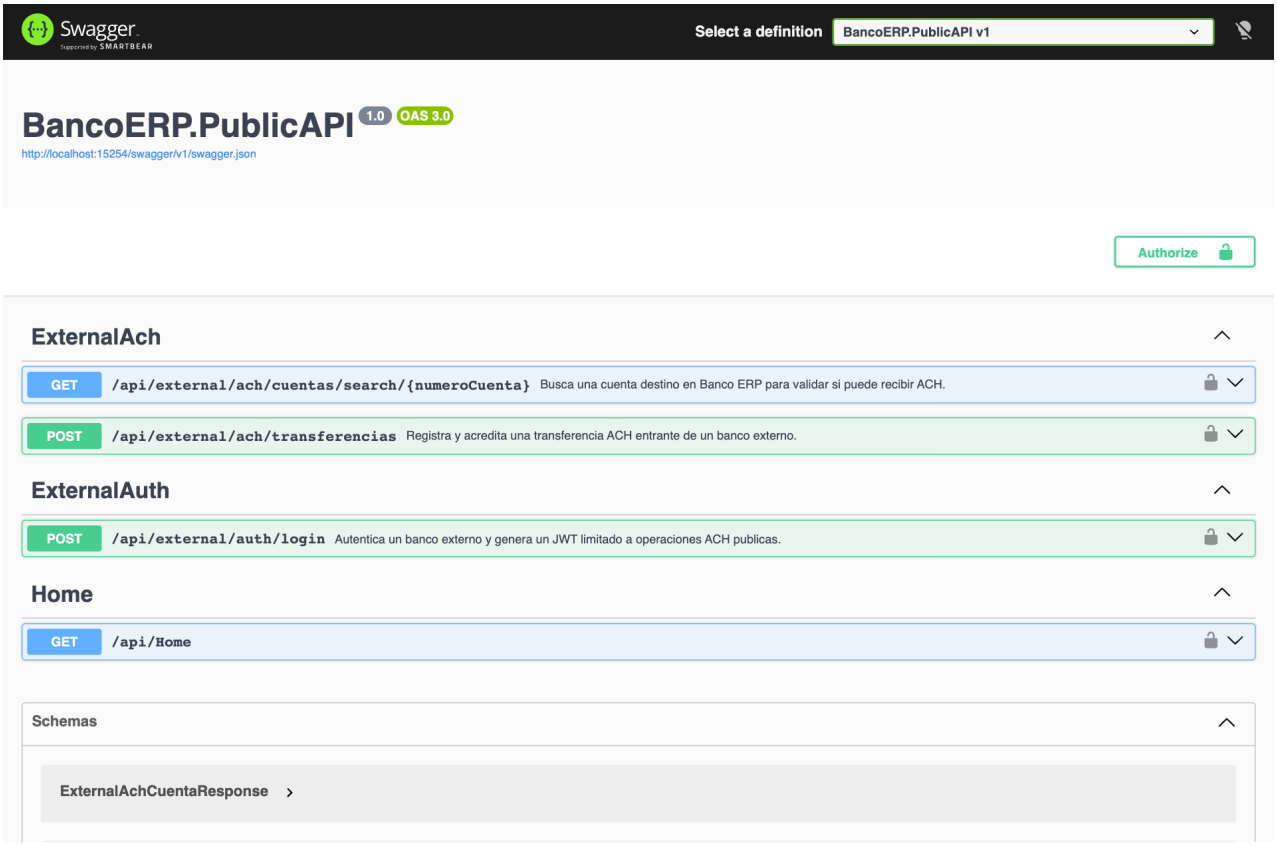
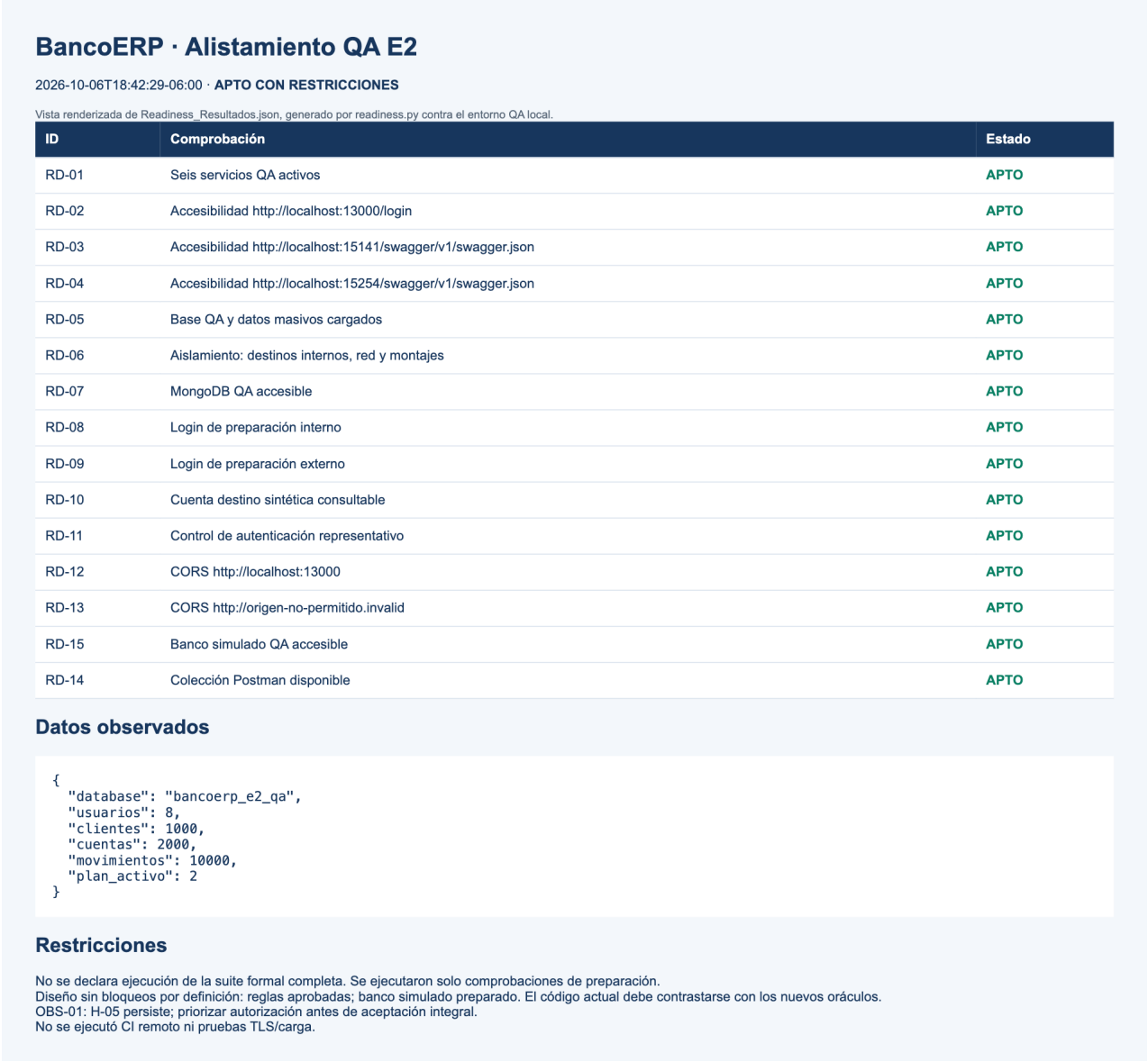


Figura 3. Swagger real de Public API QA en localhost:15254.



BancoERP · Alistamiento QA E2

2026-10-06T18:42:29-06:00 · **APTO CON RESTRICCIONES**

Vista renderizada de Readiness_Resultados.json, generado por readiness.py contra el entorno QA local.

ID	Comprobación	Estado
RD-01	Seis servicios QA activos	APTO
RD-02	Accesibilidad http://localhost:13000/login	APTO
RD-03	Accesibilidad http://localhost:15141/swagger/v1/swagger.json	APTO
RD-04	Accesibilidad http://localhost:15254/swagger/v1/swagger.json	APTO
RD-05	Base QA y datos masivos cargados	APTO
RD-06	Aislamiento: destinos internos, red y montajes	APTO
RD-07	MongoDB QA accesible	APTO
RD-08	Login de preparación interno	APTO
RD-09	Login de preparación externo	APTO
RD-10	Cuenta destino sintética consultable	APTO
RD-11	Control de autenticación representativo	APTO
RD-12	CORS http://localhost:13000	APTO
RD-13	CORS http://origen-no-permitido.invalid	APTO
RD-15	Banco simulado QA accesible	APTO
RD-14	Colección Postman disponible	APTO

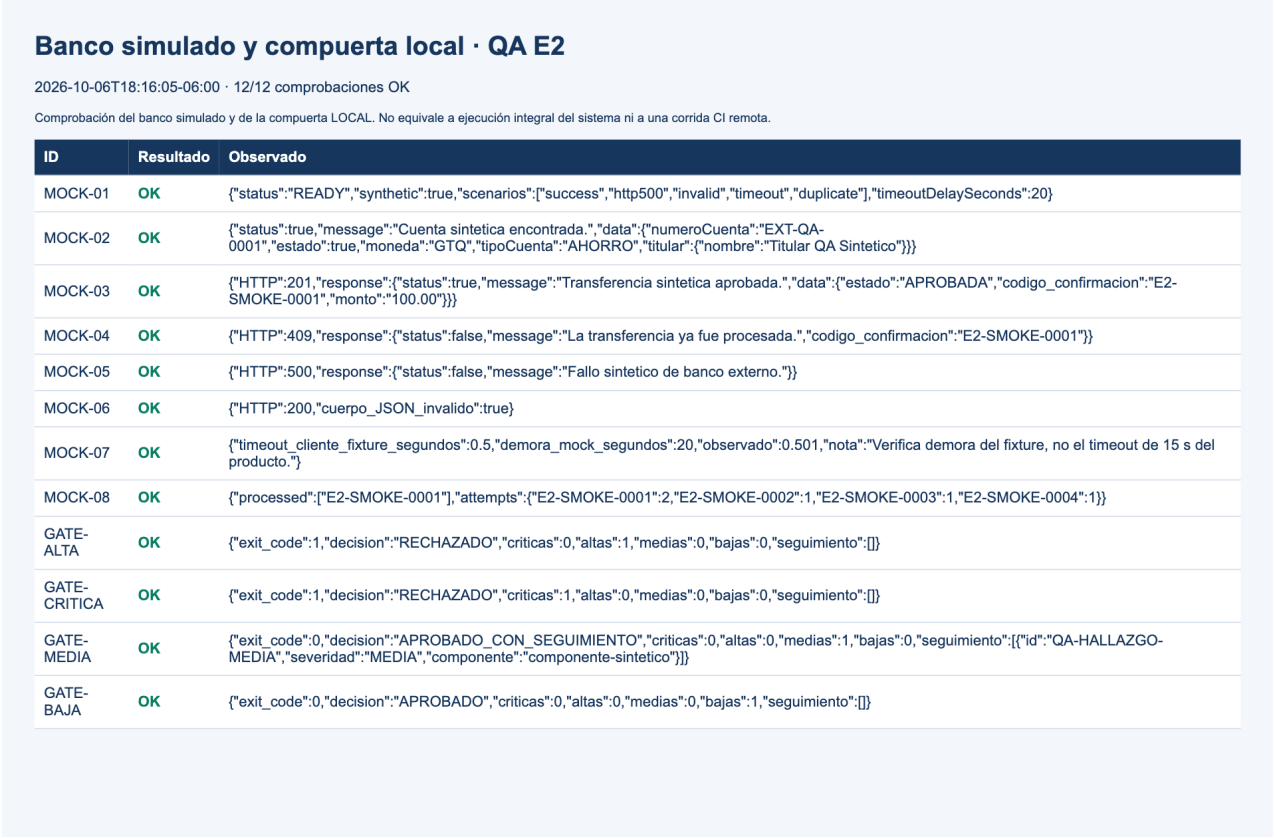
Datos observados

```
{
  "database": "bancoerp_e2_qa",
  "usuarios": 8,
  "clientes": 1000,
  "cuentas": 2000,
  "movimientos": 10000,
  "plan_activo": 2
}
```

Restricciones

No se declara ejecución de la suite formal completa. Se ejecutaron solo comprobaciones de preparación.
Diseño sin bloqueos por definición: reglas aprobadas; banco simulado preparado. El código actual debe contrastarse con los nuevos oráculos.
OBS-01: H-05 persiste; priorizar autorización antes de aceptación integral.
No se ejecutó CI remoto ni pruebas TLS/carga.

Figura 4. Vista renderizada del JSON real de preparación; complementa el log y las respuestas guardadas.



Banco simulado y compuerta local · QA E2

2026-10-06T18:16:05-06:00 · 12/12 comprobaciones OK

Comprobación del banco simulado y de la compuerta LOCAL. No equivale a ejecución integral del sistema ni a una corrida CI remota.

ID	Resultado	Observado
MOCK-01	OK	{"status":"READY","synthetic":true,"scenarios":["success","http500","invalid","timeout","duplicate"],"timeoutDelaySeconds":20}
MOCK-02	OK	{"status":true,"message":"Cuenta sintetica encontrada.","data":{"numeroCuenta":"EXT-QA-0001","estado":true,"moneda":"GTQ","tipoCuenta":"AHORRO","titular":{"nombre":"Titular QA Sintetico"}}}
MOCK-03	OK	{"HTTP":201,"response":{"status":true,"message":"Transferencia sintetica aprobada.","data":{"estado":"APROBADA","codigo_confirmacion":"E2-SMOKE-0001","monto":"100.00"}}}
MOCK-04	OK	{"HTTP":409,"response":{"status":false,"message":"La transferencia ya fue procesada.","codigo_confirmacion":"E2-SMOKE-0001"}}
MOCK-05	OK	{"HTTP":500,"response":{"status":false,"message":"Fallo sintetico de banco externo."}}
MOCK-06	OK	{"HTTP":200,"cuerpo_JSON_invalido":true}
MOCK-07	OK	{"timeout_cliente_fixture_segundos":0.5,"demora_mock_segundos":20,"observado":0.501,"nota":"Verifica demora del fixture, no el timeout de 15 s del producto."}
MOCK-08	OK	{"processed":["E2-SMOKE-0001"],"attempts":{"E2-SMOKE-0001":2,"E2-SMOKE-0002":1,"E2-SMOKE-0003":1,"E2-SMOKE-0004":1}}
GATE-ALTA	OK	{"exit_code":1,"decision":"RECHAZADO","criticas":0,"altas":1,"medias":0,"bajas":0,"seguimiento":[]}
GATE-CRITICA	OK	{"exit_code":1,"decision":"RECHAZADO","criticas":1,"altas":0,"medias":0,"bajas":0,"seguimiento":[]}
GATE-MEDIA	OK	{"exit_code":0,"decision":"APROBADO_CON_SEGUIMIENTO","criticas":0,"altas":0,"medias":1,"bajas":0,"seguimiento":[{"id":"QA-HALLAZGO-MEDIA","severidad":"MEDIA","componente":"componente-sintetico"}]}
GATE-BAJA	OK	{"exit_code":0,"decision":"APROBADO","criticas":0,"altas":0,"medias":0,"bajas":1,"seguimiento":[]}

Figura 5. Resultados reales de los 12 controles del banco simulado y la compuerta local; no representan la suite funcional completa.

5.6 Preparación reproducible y colección API

Desde la raíz del repositorio, ejecutar en este orden. Requiere Docker activo; imágenes locales bancoerp-dev-internal-api, bancoerp-dev-public-api y bancoerp-dev-frontend. Si no están presentes, construirlas con los Dockerfile.dev correspondientes. Los archivos de esquema están incluidos; no se necesita conexión a la base remota.

```
python3 ENTREGA_02_DISENO_E_IMPLEMENTACION_QA/03_Evidencias_y_MockData/scripts/preparar.py
python3 ENTREGA_02_DISENO_E_IMPLEMENTACION_QA/03_Evidencias_y_MockData/scripts/disenar.py
python3 ENTREGA_02_DISENO_E_IMPLEMENTACION_QA/03_Evidencias_y_MockData/scripts/coleccion.py
docker compose -f ENTREGA_02_DISENO_E_IMPLEMENTACION_QA/03_Evidencias_y_MockData/docker-compose.qa.yml up -d
python3 ENTREGA_02_DISENO_E_IMPLEMENTACION_QA/03_Evidencias_y_MockData/scripts/configurar_banco_mock.py
python3 ENTREGA_02_DISENO_E_IMPLEMENTACION_QA/03_Evidencias_y_MockData/scripts/mock_smoke.py
python3 ENTREGA_02_DISENO_E_IMPLEMENTACION_QA/03_Evidencias_y_MockData/scripts/readiness.py
```

En una base nueva, init_all.sql crea esquema, catálogos, permisos y los 13,008 registros sintéticos. No se inicializa de nuevo un volumen existente. El código se monta desde ../../Backend y ../../Frontend en modo lectura. Al terminar puede usarse docker compose -f .../docker-compose.qa.yml stop; no se requiere borrar volúmenes.

Importar Postman_Collection_QA.json y Postman_Environment_QA.json. La colección tiene 139 peticiones: 10 de preparación y 129 plantillas de casos HTTP. Los casos sin interfaz HTTP se ejecutan con sus scripts/inspecciones de la ficha. Antes de una corrida asignar runId nuevo, preparar usuario y fixture, usar el login de preparación del perfil correcto, que guarda token_admin, token_cliente_a, token_bloqueo, etc.; resolver rolCajeroId y permisoDepositoId con las peticiones de preparación de catálogos autenticadas como admin. No ejecutar todo de forma indiscriminada: cambian los perfiles, los estados de caja y las precondiciones.

Las aserciones incluidas verifican HTTP en los casos simples. Las verificaciones de cuerpo, secuencias compuestas, concurrencia y persistencia siguen los pasos de la ficha y las consultas del Anexo A. Esto evita presentar las plantillas como una automatización completa. El guard de la colección omite solicitudes si las URLs base no son exactamente las dos URLs QA autorizadas.

5.7 Incidencias de preparación y resolución

Durante el arranque se ajustó la inicialización a un archivo SQL montado para evitar la ejecución del shell de inicialización; se corrigió el turno sintético DIURNO al catálogo válido COMPLETO y se reinició una compilación inicial fallida de la API interna. Se verificó de nuevo el estado final. Son ajustes del montaje/fixture QA, no correcciones funcionales del sistema. Una prueba de red Docker internal impedía publicar los puertos en este host y se sustituyó por red exclusiva con puertos loopback; no se declara bloqueo de egreso.

6. Preguntas de reflexión técnica

6.1 ¿Por qué las técnicas formales evitan la sobregeneración?

Partición de equivalencia reduce entradas que comparten el mismo comportamiento a representantes justificados; BVA concentra las pruebas en la frontera y sus vecinos. Las tablas de decisión eliminan combinaciones irrelevantes mediante condiciones indiferentes y los estados evitan repetir rutas que dejan el sistema en la misma situación. El ahorro debe medirse sobre elementos de cobertura definidos, no sobre un número arbitrario de casos. [R1]

En BancoERP, probar centenares de importes dentro del intervalo válido no agrega la información que sí aportan 499.99, 500.00 y 500.01 ante un límite de 500. La matriz Pairwise reduce 16 combinaciones válidas a seis y conserva los 24 pares del modelo. Se mantienen fuera de esa reducción las pruebas de idempotencia concurrente, saldo mínimo y permisos porque responden a riesgos propios. Pairwise tampoco demuestra cobertura de interacciones de tres o cuatro factores. [R2]

6.2 ¿Qué riesgos genera copiar bases reales de producción?

Operacionalmente, una copia puede conservar usuarios, integraciones y credenciales capaces de enviar notificaciones o transacciones reales; una prueba de bloqueo puede afectar accesos compartidos y una fuga exponer saldos e identidades. Además, los datos copiados envejecen, tienen distribuciones difíciles de controlar y vuelven las pruebas menos reproducibles. Una copia que mantiene destinos externos puede convertir un error de prueba en un incidente real.

Desde el punto de vista contractual y legal, usar información personal sin una finalidad, autorización y controles adecuados puede incumplir deberes aplicables de confidencialidad, protección de datos, conservación y secreto de la información financiera. Su alcance concreto depende de la jurisdicción y de los contratos, por lo que no se atribuye aquí una sanción a una norma no analizada. La instrucción académica prohíbe expresamente datos personales reales. Este paquete usa generación desde cero, correos reservados y credenciales QA; no es una copia anonimizada de una base productiva.

6.3 ¿Qué ocurre si falta un resultado esperado exacto?

El ejecutor no puede decidir objetivamente Pass/Fail, dos testers pueden emitir veredictos distintos y una regresión automatizada carece de aserción estable. Una respuesta aparentemente correcta podría ocultar un doble crédito o una fuga. También se encarece la investigación: se discute qué debió ocurrir en vez de demostrar la desviación.

Para el reenvío ACH no basta “mostrar error”: TC-REQ19-002 exige HTTP 409, el mensaje y errors.codigo_confirmacion declarados, saldo final de 1100 desde 1000 y una sola transferencia. Esas condiciones permiten detectar tanto un código incorrecto como una duplicación financiera. El timeout saliente quedó definido en la decisión D-08: 15 s, HTTP 504, estado ERROR y conciliación sin débito ni reintento. La respuesta real del código se contrasta con ese contrato durante la ejecución; la existencia de documentación no basta para emitir un Pass.

6.4 ¿Cómo detecta la RTM requerimientos sin cobertura?

La RTM parte del inventario cerrado de 24 IDs de E1 y exige que cada uno tenga condiciones y casos. Las comprobaciones de integridad calculan el conjunto REQ sin TC y buscan casos o datos huérfanos; el resultado estructural es cero. Un cambio de criterio puede rastrearse a sus CP, TC y DT para revisar exactamente qué se debe modificar.

Sin embargo, tener un vínculo no basta: la RTM también conserva estado, oráculo y bloqueo. En esta revisión, REQ-18 ya tiene casos de timeout, HTTP 500, JSON inválido, duplicado y confirmación válida; REQ-07 cubre el catálogo reconciliado de 20 eventos. No quedan bloqueos de definición, pero sí diferencias de implementación y ejecución pendiente. Esta distinción impide confundir un diseño completo con un producto aprobado.

Anexo A. Consultas iniciales de verificación

Ejecutar mediante psql exclusivamente en bancoerp_e2_qa. Las consultas se incluyen también en scripts/verificar_qa.sql. Capturar los conteos previos y posteriores por referencia/código; no asumir que toda la base masiva debe quedar balanceada por los movimientos históricos de consulta.

```
-- Solo lectura; base exclusiva de esta entrega.
DO $$ BEGIN IF current_database()<>'bancoerp_e2_qa' THEN
  RAISE EXCEPTION 'Solo QA'; END IF; END $$;
SELECT current_database(), current_user, version();

-- Saldos antes/después: ajustar números a la ficha.
SELECT id,numero_cuenta,estado,saldo,saldo_disponible,saldo_minimo,
       limite_retiro_diario,monto_retirado_hoy
FROM cuentas.cuentas
WHERE numero_cuenta IN ('AH000 000 000 001','MO000 000 000 002');

-- Idempotencia: ajustar al código EXACTO del caso y runId.
SELECT codigo_confirmacion,count(*) AS veces,sum(monto) AS monto
FROM ach.transferencias_entrantes
WHERE codigo_confirmacion LIKE 'E2-%'
GROUP BY codigo_confirmacion;

-- Verificar partida doble SOLO de nuevas operaciones, no del histórico mock.
SELECT m.referencia,m.tipo,count(a.id) AS asientos,
 sum(CASE WHEN a.tipo_asiento='DB' THEN a.monto ELSE 0 END) AS debe,
 sum(CASE WHEN a.tipo_asiento='CR' THEN a.monto ELSE 0 END) AS haber
FROM transacciones.movimientos m
LEFT JOIN contabilidad.asientos a ON a.movimiento_id=m.id
WHERE m.referencia NOT LIKE 'E2-HIST-%'
  AND m.referencia NOT LIKE 'E2-PAG-%'
GROUP BY m.referencia,m.tipo ORDER BY m.referencia;

SELECT username,estado,bloqueado,intentos_fallidos,bloqueado_hasta
FROM seguridad.usuarios WHERE username LIKE 'qa_%';
SELECT u.username,count(*) FILTER(WHERE s.estado) AS sesiones_activas
FROM seguridad.usuarios u LEFT JOIN seguridad.sesiones s
 ON s.usuario_id=u.id GROUP BY u.username;

SELECT evento,username_intentado,ip_address,user_agent,exitoso,created_at
FROM seguridad.eventos_seguridad ORDER BY created_at DESC LIMIT 30;
SELECT estado,fondo_inicial,total_depositos,total_retiros,
 efectivo_contado,diferencia,hora_cierre FROM cajas.sesiones_caja;
SELECT codigo,activo FROM contabilidad.plan_cuentas
WHERE codigo IN ('1101','2101');


-- ACH saliente: contrastar estado y saldo con D-08; código del request real.
SELECT id,banco_codigo,codigo_confirmacion,estado,monto,mensaje_error,
 cuenta_origen_id,fecha_creacion,fecha_actualizacion
FROM ach.transferencias WHERE banco_codigo LIKE 'E2_QA_%'
ORDER BY fecha_creacion DESC;

-- Catálogo de eventos ejecutados; el diseño usa 20 códigos reconciliados.
SELECT evento,count(*) FROM seguridad.eventos_seguridad GROUP BY evento;
```

Anexo B. Inventario de la entrega

Ubicación	Contenido / uso
01_Documento_Consolidado_QA	E2_Documento_Diseno_Implementacion_EquipoXX.docx y .pdf; contenido consolidado completo.
02_Matrices_y_Casos_Prueba	E2_Matrices_y_CasosPrueba_EquipoXX.xlsx: RTM, Casos_Prueba, Data_Matrix, Condiciones, Pairwise, Decision_Deposito, Transiciones, Readiness, Bloqueos, Decisiones, Eventos y Mock_y_Compuerta.
03_Evidencias_y_MockData	MockData_Sintetica.json/.sql, MockData_Cuentas.csv, colección/entorno Postman, Casos_Diseno.json, Pairwise_Cobertura.json, Decisiones_Aprobadas.json, Catalogo_Eventos_QA.json y Validacion_Artefactos.json.
Capturas_Readiness_Entorno	Capturas PNG, Readiness_Resultados.json, Readiness_Log.txt y contratos OpenAPI capturados.

Ubicación	Contenido / uso
scripts	Generadores, inicialización de esquema, fixtures, verificaciones SQL y validación.
docker-compose.qa.yml	Servicios y persistencia propios; conexiones locales QA.
LEEME_ENTREGA.txt	Orden de uso, reproducción del entorno y restricciones.
MANIFIESTO_SHA256.txt	Huella de los archivos finales para comprobar integridad.

Anexo C. Fuentes y referencias

[R1] ISTQB. Certified Tester Foundation Level Syllabus v4.0.1 (15-09-2024), §4.2. Técnicas de caja negra. https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf. Consultado el 06-10-2026.

[R2] NIST. SP 800-142: Practical Combinatorial Testing (2010). Modelo de interacciones y límites de Pairwise. https://csrc.nist.gov/pubs/sp/800/142/final. Consultado el 06-10-2026.

[R3] Equipo QA XX. E1_MasterTestPlan_EquipoXX.docx (15-09-2026), §§5, 6, 14 y 17. Fuente original de 24 requerimientos, criterios, datos y brechas.

[R4] Repositorio BancoERP. AuthDtos.cs, AuthService.cs, CajaService.cs, TransaccionService.cs, PortalClienteService.cs, CuentaService.cs, ExternalAchPublicValidators.cs y EventosSeguridad.cs; scripts SQL 001–008, 012–016. HEAD 0453d06 y modificaciones locales presentes al revisar. Fuente de oráculos de caracterización técnica.

[R5] Actividad de clase: plan de pruebas de la Public API (03-10-2026) y fuentes actuales de la Public API. Apoyo de contrato ACH entrante; no reutilizado como prueba de ejecución de esta entrega.

[R6] Guía de actividad adjunta por el docente: Entregable 2, Diseño e Implementación STLC. Fuente de estructura, campos mínimos, técnicas, mock sintético y reflexiones solicitadas.